

Python Companion

Vibe Coding for
Medical Professionals

Complete Programs, Small Inputs,
and the Large-Data Exercises

George Grätzer

Copy-editing manuscript edition
September 23, 2026

Edition: `medical-copyedit-2026-09-23`

About This Edition

This companion follows the 23 chapters of *Vibe Coding for Medical Professionals*. It contains the complete reference programs and all the small input listings needed to run the examples. The main book develops the questions, prompts, checks, and interpretations without printing the programs.

There are 27 distinct complete reference programs: the 23 chapter applications, two integrated-capstone programs, `journey_matching_inputs.py` in Chapter 8, and `journey_large_tests.py` in Chapter 20. The separate `journey_large.csv` download contains 12,000 fictional events. Small fixtures, capstone inputs, and four independent practice fixtures are printed here with their exact filenames.

The two companion PDFs and that one CSV are the reader's downloads. No separate teaching-folder archive or reference-output folder is required. A PDF listing is not already a runnable file: create the named plain-text files from the complete listings, preserving program indentation and data. Line numbers, continuation arrows, and page furniture are not file contents.

All examples use invented records and explicit teaching rules. Their execution checks do not establish clinical validity or permission to use real patient records. No example diagnoses, treats, sends reminders, or changes appointments. The final section states what was actually run for this edition.

Edition identifier: `medical-copyedit-2026-09-23`.

Contents

How to Use This Companion	vii
1 Patient-Journey Timeline	1
2 Timeline Checker	9
3 From Eighteen to 12,000	15
4 Appointment Report	21
5 Measurement Catalogue	27
6 Measurement Checks	31
7 Weekly Teaching Extract	35
8 Follow-Up Matching	39
9 Cohort Summary	47
10 Monthly Waiting-Time Report	53
11 Weekly Follow-Up Monitor	59
12 Clinic Schedules	63
13 Appointment Letters	69

14 Rule-and-Boundary Lab	75
15 Predictive-Value Explorer	79
16 Literature-Review Organizer	85
17 Reproducible Research Table	91
18 Reminder Simulation	97
19 District Outreach	103
20 Journey Tests	107
21 Demonstration Package	119
22 Review-Status Check	125
23 Follow-Up Worklist	129
The Integrated Capstone	137
Larger Practice Files	155
A Small Python Reference	165
Checks for This Edition	167

How to Use This Companion

Find the Complete Listing, Not Just the Excerpt

Each numbered chapter has a run-and-check guide, a short explanation of key lines, a *Try One Change* task, and a *Complete Working Files* section. The key-line excerpts explain ideas but are not runnable programs. Use the complete named listing when creating a working file. Required modules from earlier chapters are identified explicitly.

Prompt

From Chapter 1 of this companion, save the complete `journey_view.py` listing as an actual Python file and the complete `journey_demo.csv` listing as an actual CSV file. Keep all original instructions, indentation, and records. Remove printed line numbers and continuation arrows. Return the two files with exactly those names. Do not substitute a newly written program or newly generated data.

Save the returned files in a working folder. Ask for a syntax check before running a reconstructed Python file and compare it with the complete listing when the check fails. Also check spaces in wrapped quoted strings: a lost space can pass a syntax check while changing a label or layout rule. Use a fresh working copy for each trial; several programs refuse to replace existing outputs. Their displayed commands assume that the terminal is open in the folder containing the program and input files. The reference programs use Python's standard library, without third-party packages.

Reference Run or Independent Construction?

A reference run uses the complete supplied program unchanged. The main book separately labels prompts for *Build Your Own Version*. A new implementation must meet the specified rules and record-level expectations, but need not have identical source code or formatting. For a simulation, even a shared seed need not produce identical trials if algorithms draw random numbers differently.

Chapter 1 of the main book gives the continuous first-run route and a recovery table. Check that the current session accepts files, executes Python, and returns actual output files; capabilities vary. A folder on your computer is not automatically visible to that session. Attach the required files, or run them with local Python using the printed command. Save returned outputs locally and open the actual HTML, not a picture of it. Reopening that saved page and repeating in a fresh folder are separate checks.

Run the Small Example Before the Large One

Chapter 1 uses the clean eighteen-record journey input. Chapter 2 supplies a different eighteen-record file with six review rows. Chapter 3 runs the reusable tool on that problem file, then on the separate downloaded `journey_large.csv`. Keep the two sets of expectations distinct. The large input has 12,000 events, 1,000 people, and 3,000 encounters. P0001 belongs to the large file; P01 belongs to the small fixtures.

The public download page is:

<https://github.com/mertnem/Vibe-Coding-for-Medical-Professionals-Companions-and-Data>

It supplies this volume, `Exercise_Help_and_Solutions.pdf`, and `journey_large.csv`. Save the actual CSV rather than a webpage preview. The large file is directly suitable for the Chapter 3 journey runner; Chapter 8 specifies a conversion before matching. Other programs need their own declared fields and small input files.

Keep Evidence of the Actual Run

A session that executes Python can run the saved files and return its outputs. Ask it to state whether execution occurred; a predicted result is not an execution record. Open the actual CSV, browser page, or report. Compare counts and named records before requesting any change. Read the command-line examples as exact run instructions, not as another program to reconstruct.

The four larger practice fixtures at the end are independent small extensions. They are not subsets of `journey_large.csv`. Their R2, R8, R9, and R23 tasks remain in both parts of *Exercise Help and Solutions*. The main book's large-data exercises are covered there too.

Patient-Journey Timeline

The first application turns a small event file into a page that you can open in a browser. A person may have several encounters, and an encounter may have several events. The program draws the dated events and keeps a table underneath them. The table is not an optional decoration: it lets you check what each marker represents.

Start by running the supplied file; you need not understand its browser-page code first.

1.1 Run the Reference Version

Working files: complete listings in Chapter 1 below.

Inputs: `journey_demo.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 journey_view.py
```

Rule to preserve. One input row is one event. The six field names and their order are fixed. Event IDs must be distinct; required values must be present; dates must be exact calendar dates in YYYY-MM-DD form. This first program expects a clean file. It does not perform the encounter-anchor checks introduced in Chapter 2. Same-day positions do not imply a time-of-day order.

1.2 Check the Result

Outputs: `journey_events.csv`, `journey_timeline.html`.

Check the meaning

The CSV must contain all 18 events: three people and six encounters. Find E02 and check P01, encounter C01, June 2, 2026, and the detail “Sample collected.” Selecting P01 in the page should show six records. A marker can represent more than one event of the same date and type; do not use the number of circles as the record count.

1.3 Read the Key Lines

```
12 def read_events(path):
13     with Path(path).open(newline="", encoding="utf-8-sig") as stream:
14         reader = csv.DictReader(stream, strict=True)
15         if reader.fieldnames != FIELDS:
16             raise ValueError("Expected header: " + ",".join(FIELDS))
17         rows = list(reader)
18     if any(None in row or None in row.values() for row in rows):
19         raise ValueError("Malformed CSV row")
20     return rows
```

Excerpt from journey_view.py, source lines 12–20.

Named fields rather than column numbers. `csv.DictReader` makes each row a dictionary. Thus `row["event_id"]` retrieves a named field rather than “column 1.” `FIELDS` is the declared header; comparing the received header with it makes a mistaken input fail visibly.

Lists, sets, and groups. The list `rows` retains every record. A set of person references counts distinct people, whereas `len(rows)` counts events. `defaultdict(list)` collects events sharing a date and type. These three operations answer different questions; none licenses deleting a source event.

A page is also an output file. `timeline_svg` constructs vector drawing instructions. `render_page` combines them with tables and a small JavaScript filter. Python creates the page once; JavaScript changes what the browser displays afterward. `escape` ensures that supplied detail text is displayed as text rather than interpreted as markup.

1.4 Try One Change

In a fresh copy, change only E02’s detail to “Sample collected: teaching revision.” Run again. The new text must appear in both the CSV and browser page, while the totals remain three people, six encounters, and 18 events. This separates a presentation change from a change in the population being displayed.

1.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

journey_demo.csv

Save the following complete contents as `journey_demo.csv`.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
E01,P01,C01,2026-06-01,visit,Visit recorded
E02,P01,C01,2026-06-02,test,Sample collected
E03,P01,C01,2026-06-05,followup,Follow-up recorded
E04,P01,C02,2026-06-12,visit,Visit recorded
E05,P01,C02,2026-06-12,measurement,Temperature 37.0 C
E06,P01,C02,2026-06-18,followup,Follow-up recorded
E07,P02,C03,2026-06-03,visit,Visit recorded
E08,P02,C03,2026-06-04,test,Sample collected
E09,P02,C03,2026-06-10,followup,Follow-up recorded
E10,P02,C04,2026-06-15,visit,Visit recorded
E11,P02,C04,2026-06-15,measurement,Temperature 36.8 C
E12,P02,C04,2026-06-22,followup,Follow-up recorded
E13,P03,C05,2026-06-06,visit,Visit recorded
E14,P03,C05,2026-06-07,test,Sample collected
E15,P03,C05,2026-06-13,followup,Follow-up recorded
E16,P03,C06,2026-06-20,visit,Visit recorded
E17,P03,C06,2026-06-20,measurement,Temperature 37.1 C
E18,P03,C06,2026-06-26,followup,Follow-up recorded
```

journey_view.py

Complete program. Save this listing as `journey_view.py`.

```
1 import csv
2 from collections import defaultdict
3 from datetime import date, timedelta
4 from html import escape
5 from pathlib import Path
6
7 FIELDS = ["event_id", "person_ref", "encounter_id",
8           "event_date", "event_type", "detail"]
9 KINDS = ["visit", "test", "measurement", "followup"]
10 COLORS = ["#175078", "#6b4697", "#936019", "#267054"]
11
12 def read_events(path):
13     with Path(path).open(newline="", encoding="utf-8-sig") as stream:
```

```

14     reader = csv.DictReader(stream, strict=True)
15     if reader.fieldnames != FIELDS:
16         raise ValueError("Expected header: " + ",".join(FIELDS))
17     rows = list(reader)
18     if any(None in row or None in row.values() for row in rows):
19         raise ValueError("Malformed CSV row")
20     return rows
21
22 def save_csv(path, rows, fields):
23     with Path(path).open("x", newline="", encoding="utf-8") as stream:
24         writer = csv.DictWriter(stream, fieldnames=fields)
25         writer.writeheader()
26         writer.writerows(rows)
27
28 def clean_events(rows):
29     # Chapter 1 starts with a well-formed file, not a repair task.
30     ids = [r["event_id"] for r in rows]
31     if len(set(ids)) != len(ids):
32         raise ValueError("Duplicate event ID; use the Chapter 2 checker")
33     for row in rows:
34         for key in ["event_id", "person_ref", "encounter_id"]:
35             if row[key] != row[key].strip():
36                 raise ValueError("Padded identifier: " + key)
37         if any(not row[k].strip() for k in FIELDS):
38             raise ValueError("Blank field; use the Chapter 2 checker")
39         day = date.fromisoformat(row["event_date"])
40         if day.isoformat() != row["event_date"]:
41             raise ValueError("Use YYYY-MM-DD")
42         if row["event_type"] not in KINDS:
43             raise ValueError("Unrecognized event type")
44     return [dict(r, data_row=i, owner_ref=r["person_ref"],
45                 status="placed", issues="")
46             for i, r in enumerate(rows, 1)]
47
48 def event_key(row):
49     return (row["event_date"], KINDS.index(row["event_type"]),
50           row["event_id"], row["data_row"])
51
52 def timeline_svg(rows, first, last):
53     # One marker per date and type; coincident events share a marker.
54     span = max((last-first).days, 1)
55     x = lambda day: 125 + 715*(day-first).days/span
56     groups = defaultdict(list)
57     for row in rows:
58         groups[(row["event_date"], row["event_type"])].append(row)
59     parts = ['<svg xmlns="http://www.w3.org/2000/svg" '
60             'viewBox="0 0 900 230" role="img" style="font:14px sans-serif" '

```

```

61         'aria-label="Recorded events by date and type">']
62     for offset in sorted({round(span*i/5) for i in range(6)}):
63         day = first + timedelta(days=offset)
64         xx = x(day)
65         parts.append(f'<path d="M{xx} 25 V190" stroke="#dde3e8"/>'
66                     f'<text x="{xx}" y="214" text-anchor="middle">'
67                     f'{day:%d %b}</text>')
68     for lane, kind in enumerate(KINDS):
69         yy = 40 + 44*lane
70         parts.append(f'<text x="5" y="{yy+5}">{kind}</text>')
71         parts.append(f'<path d="M125 {yy} H840" stroke="#d4dce4"/>')
72     for (day, kind), events in sorted(groups.items()):
73         lane = KINDS.index(kind)
74         xx, yy = x(date.fromisoformat(day)), 40+44*lane
75         label = events[0]["event_id"] if len(events)==1 else f'{len(events)}
↳ events"
76         tip = "; ".join(f'{r["event_id"]}: {r["detail"]}'
77                         f'({r["encounter_id"]})' for r in events)
78         parts.append(f'<a tabindex="0" href="#event-{events[0]["data_row"]}">'
79                     f'<title>{escape(tip)}</title>'
80                     f'<circle cx="{xx}" cy="{yy}" r="7" fill="{COLORS[lane]}
↳ ]"/>'
81                     f'<text x="{xx}" y="{yy-12}" text-anchor="middle">'
82                     f'{escape(label)}</text></a>')
83     return "".join(parts) + "</svg>"
84
85 def render_page(report, title="Patient-journey timeline"):
86     people = sorted({r["owner_ref"] for r in report})
87     placed = [r for r in report if r["status"]=="placed"]
88     dates = [date.fromisoformat(r["event_date"]) for r in placed]
89     first, last = (min(dates), max(dates)) if dates else (date(2026,6,1), date
↳ (2026,6,2))
90     page = ['<!doctype html><html lang="en"><meta charset="utf-8">'
91            '<meta name="viewport" content="width=device-width,initial-scale
↳ =1">'
92            '<title>'+escape(title)+'</title><style>'
93            'body{font:17px system-ui;color:#243544;background:#f1f5f8;margin
↳ :0}'
94            'main{max-width:1100px;margin:auto;padding:28px}'
95            'h1{color:#175078}section{background:white;padding:22px;margin:22px
↳ :0;'
96            'border:1px solid #d4dce4;border-radius:10px}'
97            'svg{width:100%;min-width:650px;font:14px system-ui}'
98            '.scroll{overflow-x:auto}table{border-collapse:collapse;width:100%}
↳ '
99            'th,td{text-align:left;padding:9px;border-bottom:1px solid #dde3e8}
↳ '

```

```

100         '.review{background:#fff1dc}tr:target{outline:2px solid #175078}'
101         'select{font:inherit;padding:7px}.note{max-width:850px}'
102         '@media print{body{background:white}select{display:none}}'
103         '</style><main><h1>'+escape(title)+'</h1>'
104         '<p class="note">SYNTHETIC TEACHING RECORDS. '
105         'Events are recorded information, not diagnoses or recommendations.
    ↪ '
106         'Select a person; hover over or select a marker for details. '
107         'Same-day positions do not establish time-of-day order.</p>'
108         '<label>Person <select id="person"><option value="">All people</
    ↪ option>']
109 page.extend('<option>'+escape(p)+'</option>' for p in people)
110 page.append('</select></label><p id="count" aria-live="polite"></p>')
111 if not people:
112     page.append("<p>No event records were supplied.</p>")
113 for person in people:
114     group = [r for r in report if r["owner_ref"]==person]
115     good = sorted((r for r in group if r["status"]=="placed"),key=event_key
    ↪ )
116     bad = [r for r in group if r["status"]!="placed"]
117     page.append('<section data-person="'+escape(person,quote=True)+
118                 f'" data-total="{len(group)}" data-placed="{len(good)}" '
119                 f'data-review="{len(bad)}"><h2>'+escape(person)+'</h2>')
120     page.append('<div class="scroll">'+timeline_svg(good,first,last)+'</div>
    ↪ >'
121         if good else '<p>No dated events can be placed.</p>')
122     page.append('<h3>Recorded events</h3><div class="scroll"><table>'
123                 '<tr><th>ID</th><th>Date</th><th>Type</th>'
124                 '<th>Encounter</th><th>Detail</th></tr>')
125     for row in good:
126         page.append(f'<tr id="event-{row["data_row"]}>'+
127                     '".join("<td>"+escape(row[k])+"</td>"
128                             for k in ["event_id","event_date","event_type","
    ↪ encounter_id","detail"])+</tr>")
129     page.append("</table></div>")
130     if bad:
131         page.append('<h3>Not placed: review data</h3><div class="scroll"><
    ↪ table>'
132                 '<tr><th>Row</th><th>ID</th><th>Supplied person</th>'
133                 '<th>Supplied date</th><th>Issue</th></tr>')
134     for row in bad:
135         page.append('<tr class="review">'+
136                     '".join("<td>"+escape(str(row[k]))+"</td>"
137                             for k in ["data_row","event_id","person_ref",
    ↪ event_date","issues"])+</tr>")
138     page.append("</table></div>")
139     page.append("</section>")

```

```

140     page.append("""<script>
141     const picker=document.getElementById("person");
142     function selectPerson(){
143         let total=0,placed=0,review=0;
144         document.querySelectorAll("section[data-person]").forEach(s=>{
145             s.hidden=picker.value!==" && s.dataset.person!==picker.value;
146             if(!s.hidden){total+=Number(s.dataset.total);
147                 placed+=Number(s.dataset.placed);review+=Number(s.dataset.review);}
148         });
149         document.getElementById("count").textContent=
150         total+" records | "+placed+" placed | "+review+" for review";
151     }
152     picker.addEventListener("change",selectPerson);selectPerson();
153     </script></main></html>""")
154     return "".join(page)
155
156 def main():
157     root = Path(__file__).resolve().parent
158     report = clean_events(read_events(root/"journey_demo.csv"))
159     fields = FIELDS + ["data_row","owner_ref","status","issues"]
160     save_csv(root/"journey_events.csv",sorted(report,key=lambda r:
161         (r["owner_ref"],)+event_key(r)),fields)
162     with (root/"journey_timeline.html").open("x",encoding="utf-8") as stream:
163         stream.write(render_page(report))
164     print("People:",len({r["person_ref"] for r in report}))
165     print("Encounters:",len({r["encounter_id"] for r in report}))
166     print("Events:",len(report))
167     print("Saved journey_timeline.html and journey_events.csv")
168
169 if __name__ == "__main__":
170     main()

```


Timeline Checker

The second program deals with records that cannot safely be placed on a timeline. It does not make troublesome rows disappear. Instead, it creates a checking report with the original values, source-row numbers, dispositions, and reasons, and passes that report to the display functions from Chapter 1.

2.1 Run the Reference Version

Working files: complete listings in Chapter 2 below.

Inputs: `journey_problem.csv`.

Local modules: `journey_view.py`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 journey_check.py
```

Rule to preserve. Every input row must survive in the report. Each encounter needs exactly one visit row that itself passes the checks. Other events must agree with that visit’s person and must not precede its date. A duplicated event ID affects both occurrences. An event whose person disagrees with its visit is grouped under the visit owner, while its supplied person remains visible.

2.2 Check the Result

Outputs: `journey_checked.html`, `journey_review.csv`.

Check the meaning

The baseline retains 18 rows: 12 placed and six for review. Review source rows 2, 3, 8, 11, 14, and 18. The two occurrences of E02 are rows 2 and 14; neither is silently selected as the “real” one. In row 18, E18 retains supplied person P01 but has owner P03 and the reason “person differs from visit.”

2.3 Read the Key Lines

```

8 def check_events(rows):
9     ids = Counter(r["event_id"].strip() for r in rows)
10    issues = [[] for _ in rows]
11    days = []
12    visits = defaultdict(list)
13    for i, row in enumerate(rows):
14        for field in FIELDS:
15            if not row[field].strip():
16                issues[i].append("missing " + field)
17        for field in ["event_id", "person_ref", "encounter_id"]:
18            if row[field] != row[field].strip():
19                issues[i].append("padded " + field)
20        key = row["event_id"].strip()
21        if key and ids[key] > 1:
22            issues[i].append("duplicate event_id")

```

Excerpt from journey_check.py, source lines 8–22.

Count before deciding. `Counter` counts all supplied IDs before any row is classified. This avoids a common asymmetric mistake: accepting the first duplicate and rejecting only the second. The parallel `issues` lists allow several reasons to be retained for one row.

Separate passes have different jobs. The first pass checks fields and collects visit positions. The anchor-building pass admits only an unambiguous, otherwise valid visit. The final pass compares each event with that anchor. Reading the passes in that order is more useful than treating the function as one long loop.

Derived and supplied fields can coexist. `dict(row, ...)` preserves the original dictionary fields and adds checking fields. `owner_ref` serves the display; it does not overwrite `person_ref`. That distinction is what makes the mismatch auditable.

2.4 Try One Change

Change only the event ID in source row 14 from E02 to E14. Both duplicate-ID reasons should clear, leaving 14 placed rows and four review rows. The row count stays 18. A result of 13 placed rows would suggest that only one occurrence was reconsidered.

2.5 Complete Working Files

Save this chapter's complete program below and supply these unchanged earlier listings: `journey_view.py` from Chapter 1.

`journey_problem.csv`

Save the following complete contents as `journey_problem.csv`.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
E01,P01,C01,2026-06-01,visit,Visit recorded
E02,P01,C01,2026-06-02,test,Sample collected
E03,P01,C01,2026-05-31,followup,Follow-up recorded
E04,P01,C02,2026-06-12,visit,Visit recorded
E05,P01,C02,2026-06-12,measurement,Temperature 37.0 C
E06,P01,C02,2026-06-18,followup,Follow-up recorded
E07,P02,C03,2026-06-03,visit,Visit recorded
E08,P02,C03,,test,Sample collected
E09,P02,C03,2026-06-10,followup,Follow-up recorded
E10,P02,C04,2026-06-15,visit,Visit recorded
E11,P02,C04,2026-06-31,measurement,Temperature 36.8 C
E12,P02,C04,2026-06-22,followup,Follow-up recorded
E13,P03,C05,2026-06-06,visit,Visit recorded
E02,P03,C05,2026-06-07,test,Sample collected
E15,P03,C05,2026-06-13,followup,Follow-up recorded
E16,P03,C06,2026-06-20,visit,Visit recorded
E17,P03,C06,2026-06-20,measurement,Temperature 37.1 C
E18,P01,C06,2026-06-26,followup,Follow-up recorded
```

`journey_check.py`

Complete program. Save this listing as `journey_check.py`.

```
1 from collections import Counter, defaultdict
2 from datetime import date
3 from pathlib import Path
4 from journey_view import FIELDS, KINDS, read_events, save_csv, render_page
5
6 REPORT_FIELDS = FIELDS + ["data_row", "owner_ref", "status", "issues"]
7
8 def check_events(rows):
9     ids = Counter(r["event_id"].strip() for r in rows)
10    issues = [[] for _ in rows]
11    days = []
12    visits = defaultdict(list)
13    for i, row in enumerate(rows):
```

```

14     for field in FIELDS:
15         if not row[field].strip():
16             issues[i].append("missing " + field)
17     for field in ["event_id", "person_ref", "encounter_id"]:
18         if row[field] != row[field].strip():
19             issues[i].append("padded " + field)
20     key = row["event_id"].strip()
21     if key and ids[key] > 1:
22         issues[i].append("duplicate event_id")
23     try:
24         day = date.fromisoformat(row["event_date"])
25         if day.isoformat() != row["event_date"]:
26             raise ValueError()
27     except ValueError:
28         day = None
29         if row["event_date"].strip():
30             issues[i].append("invalid event_date")
31     days.append(day)
32     if row["event_type"] not in KINDS:
33         issues[i].append("unrecognized event_type")
34     if row["event_type"]=="visit" and row["encounter_id"].strip():
35         visits[row["encounter_id"]].append(i)
36 # An encounter needs one unambiguous, dated, otherwise valid visit.
37 anchors = {}
38 for encounter, positions in visits.items():
39     if len(positions)!=1:
40         for i in positions:
41             issues[i].append("multiple visits for encounter")
42     else:
43         i = positions[0]
44         if not issues[i]:
45             anchors[encounter] = i
46 report = []
47 for i, row in enumerate(rows):
48     anchor = anchors.get(row["encounter_id"])
49     owner = row["person_ref"] or "(unassigned)"
50     if anchor is None:
51         if row["event_type"]!="visit" or not issues[i]:
52             issues[i].append("no valid visit for encounter")
53     else:
54         owner = rows[anchor]["person_ref"]
55         if row["person_ref"] != owner:
56             issues[i].append("person differs from visit")
57         if days[i] is not None and days[i] < days[anchor]:
58             issues[i].append("event before visit")
59     report.append(dict(row, data_row=i+1, owner_ref=owner,
60                       status="review" if issues[i] else "placed",

```

```
61         issues=""; ".join(issues[i]))
62     return report
63
64 def main():
65     root = Path(__file__).resolve().parent
66     report = check_events(read_events(root/"journey_problem.csv"))
67     save_csv(root/"journey_review.csv",report,REPORT_FIELDS)
68     with (root/"journey_checked.html").open("x",encoding="utf-8") as stream:
69         stream.write(render_page(report,"Checked patient-journey timeline"))
70     print("Input records:",len(report))
71     print("Placed:",sum(r["status"]=="placed" for r in report))
72     print("For review:",sum(r["status"]=="review" for r in report))
73     print("Saved journey_checked.html and journey_review.csv")
74
75 if __name__ == "__main__":
76     main()
```


From Eighteen to 12,000

The reusable journey tool accepts an input filename, a new output-folder name, and an optional person reference. It saves the report and page together with a record of the input, program versions, interpreter, rules, and counts.

3.1 Run the Reference Version

Working files: complete listings in Chapter 3 below.

Inputs: `journey_problem.csv`.

Local modules: `journey_check.py`, `journey_view.py`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 journey_report.py journey_problem.csv first_run
```

Rule to preserve. Validate the complete input before selecting a person. Selection uses the derived visit owner, not an unverified supplied person. The destination folder must be new, and its parent must already exist. A run record identifies the bytes processed; it does not prove that the rules are appropriate.

3.2 Check the Result

Outputs: `first_run/journey_report.csv`, `first_run/journey_timeline.html`, `first_run/run_record.json`.

Check the meaning

The all-person run retains 18 selected records, 12 placed and six for review. In `run_record.json`, check the input filename, the three program hashes, the stated rule, and the counts. A P03 run must report 18 input records but only six selected records, of which four are placed and two require review. E18 must remain in that selection with its supplied P01 value visible.

3.3 Read the Key Lines

```

11 def build_report(source, destination, person=None):
12     source, destination = Path(source), Path(destination)
13     report = check_events(read_events(source))
14     # Validate the complete file BEFORE selecting a person's records.
15     selected = [r for r in report if person is None or r["owner_ref"]==person]
16     if person is not None and not selected:
17         raise ValueError("No records for selected person: " + person)
18     counts = dict(input_records=len(report), selected_records=len(selected),
19                   placed=sum(r["status"]=="placed" for r in selected),
20                   review=sum(r["status"]=="review" for r in selected))

```

Excerpt from journey_report.py, source lines 11–20.

Arguments are inputs to a function. `build_report(source, destination, person=None)` names three inputs. The default `None` means no person filter. It does not mean that missing person values in the data are acceptable.

Filtering comes after validation. The call to `check_events` precedes the list comprehension that selects records. Reversing those operations could hide a duplicate or an inconsistent anchor outside the chosen person's apparent rows.

Exclusive folder creation. `destination.mkdir(parents=False, exist_ok=False)` refuses a reused output folder. The function then writes all three result files there. If a run fails after folder creation, preserve that partial folder and use a new name for the next attempt.

3.4 Try One Change

Run `python3 journey_report.py journey_problem.csv P03_run --person P03` in a fresh chapter copy. Check the P03 counts above. Then repeat the identical command: it must fail rather than overwrite `P03_run`. Verify that the earlier files are unchanged. For another successful run, choose a different destination name.

3.5 Run the Separate Large Input

Keep the same three programs and the downloaded `journey_large.csv` together. These commands create two different new output folders:

```

python3 journey_report.py journey_large.csv journey_large_01
python3 journey_report.py journey_large.csv journey_large_P0001 --person
    ↪ P0001

```

The complete run has 12,000 input and selected records, 12,000 placed, and zero review rows. The selected run still has 12,000 input records, but twelve selected and

placed records. Its event IDs are E000001 through E000012. Compare all six original fields, not only the number of rows. P0001's visits are January 1, March 7, and April 16, 2026; its follow-ups are January 12, March 21, and May 6. Use the main book's named checks and controlled missing-date trial before adding a feature.

The reference runner checks the complete input before applying the person selection. The large suite in Chapter 20 tests that ordering with a duplicate shared by P0001 and P0002. These commands do not establish a maximum input size or test browser interaction.

3.6 Complete Working Files

Save this chapter's complete program below and supply these unchanged earlier listings: `journey_check.py` from Chapter 2; `journey_view.py` from Chapter 1.

`journey_problem.csv`

Save the following complete contents as `journey_problem.csv`.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
E01,P01,C01,2026-06-01,visit,Visit recorded
E02,P01,C01,2026-06-02,test,Sample collected
E03,P01,C01,2026-05-31,followup,Follow-up recorded
E04,P01,C02,2026-06-12,visit,Visit recorded
E05,P01,C02,2026-06-12,measurement,Temperature 37.0 C
E06,P01,C02,2026-06-18,followup,Follow-up recorded
E07,P02,C03,2026-06-03,visit,Visit recorded
E08,P02,C03,,test,Sample collected
E09,P02,C03,2026-06-10,followup,Follow-up recorded
E10,P02,C04,2026-06-15,visit,Visit recorded
E11,P02,C04,2026-06-31,measurement,Temperature 36.8 C
E12,P02,C04,2026-06-22,followup,Follow-up recorded
E13,P03,C05,2026-06-06,visit,Visit recorded
E02,P03,C05,2026-06-07,test,Sample collected
E15,P03,C05,2026-06-13,followup,Follow-up recorded
E16,P03,C06,2026-06-20,visit,Visit recorded
E17,P03,C06,2026-06-20,measurement,Temperature 37.1 C
E18,P01,C06,2026-06-26,followup,Follow-up recorded
```

journey_report.py

Complete program. Save this listing as `journey_report.py`.

```

1  import argparse
2  import hashlib
3  import json
4  import platform
5  from pathlib import Path
6  import journey_view
7  import journey_check
8  from journey_view import read_events, save_csv, render_page
9  from journey_check import check_events, REPORT_FIELDS
10
11 def build_report(source, destination, person=None):
12     source, destination = Path(source), Path(destination)
13     report = check_events(read_events(source))
14     # Validate the complete file BEFORE selecting a person's records.
15     selected = [r for r in report if person is None or r["owner_ref"]==person]
16     if person is not None and not selected:
17         raise ValueError("No records for selected person: " + person)
18     counts = dict(input_records=len(report), selected_records=len(selected),
19                   placed=sum(r["status"]=="placed" for r in selected),
20                   review=sum(r["status"]=="review" for r in selected))
21     modules = [Path(__file__), Path(journey_view.__file__), Path(journey_check.
22     ↪ __file__)]
23     record = dict(input_file=source.name,
24                   input_sha256=hashlib.sha256(source.read_bytes()).hexdigest(),
25                   program_sha256={p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p
26     ↪ in modules},
27                   python=platform.python_version(), person=person or "all",
28                   rules="Exactly one visit row, which must pass checks, per encounter;
29     ↪ validate before filtering; "
30                   "review rows retained but not plotted", **counts)
31     # The whole report must have a new directory, including on repeat runs.
32     destination.mkdir(parents=False, exist_ok=False)
33     save_csv(destination/"journey_report.csv", selected, REPORT_FIELDS)
34     with (destination/"journey_timeline.html").open("x", encoding="utf-8") as
35     ↪ stream:
36         stream.write(render_page(selected, "Checked patient-journey timeline"))
37     with (destination/"run_record.json").open("x", encoding="utf-8") as stream:
38         json.dump(record, stream, indent=2)
39     return counts
40
41 def main():
42     parser = argparse.ArgumentParser(description="Build a synthetic journey
43     ↪ report")
44     parser.add_argument("input_csv")

```

```
40     parser.add_argument("new_output_folder")
41     parser.add_argument("--person")
42     args = parser.parse_args()
43     try:
44         counts = build_report(args.input_csv,args.new_output_folder,args.person
45         ↩ )
46     except (ValueError,OSError) as error:
47         parser.exit(1,str(error)+"\n")
48     for key,value in counts.items():
49         print(key + ":",value)
50     print("Saved journey_report.csv, journey_timeline.html, run_record.json")
51 if __name__ == "__main__":
52     main()
```


Appointment Report

This report illustrates a subtle programming error: making a percentage look better by dropping an eligible record whose outcome is unresolved. The program records eligibility, confirmed completion, and unresolved information as separate fields. They are not mutually exclusive categories.

4.1 Run the Reference Version

Working files: complete listings in Chapter 4 below.

Inputs: `appointments.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 appointment_report.py
```

Rule to preserve. Eligibility requires an acceptable appointment ID, scheduled date, and status. A missing completion date does not by itself remove an otherwise eligible completed appointment from the denominator. A confirmed completion additionally needs a valid completion date that does not precede the appointment. Both occurrences of a duplicated appointment ID are unresolved and ineligible under this teaching rule.

4.2 Check the Result

Outputs: `appointment_report.csv`.

Check the meaning

The baseline is 2 confirmed completions among 5 eligible records, or 40.0%. A03 has `eligible=1`, `completed=0`, and `unresolved=1`. A04 and A05 remain eligible despite cancelled and no-show statuses. Both A07 rows remain in the output. Four unresolved rows are not the complement of the five eligible rows.

4.3 Read the Key Lines

```
52     eligible = not issues
53     completed = False
54     if status == "completed":
55         try:
56             finished = parse_day(row["completed_date"])
57             if eligible and finished < scheduled:
58                 issues.append("completion precedes appointment")
59             elif eligible:
60                 completed = True
61         except ValueError:
62             issues.append("missing or invalid completion date")
63     output.append(dict(row, data_row=number,
64                       eligible=int(eligible), completed=int(completed),
65                       unresolved=int(bool(issues)), reason="; ".join(issues)))
```

Excerpt from appointment_report.py, source lines 52–65.

Order of assignment matters. The assignment `eligible = not issues` occurs before completion-date checks. This is intentional: those later checks affect confirmation and unresolved status without retroactively changing the defined denominator.

Boolean values support counting. The program stores `int(eligible)` and `int(completed)` as 0 or 1. Summing these fields counts rows meeting the respective rules. It is still necessary to inspect which rows supplied the ones.

An undefined proportion needs a branch. The final expression calculates a proportion only when the denominator is nonzero. The other branch prints “not calculable.” An empty eligible population is not a 0% completion rate.

4.4 Try One Change

In the synthetic A03 row, supply completion date 2026-09-01. The confirmed numerator becomes 3 while the eligible denominator remains 5: 60.0%. Seven or eight eligible rows would indicate that unrelated ID or status rules changed as well.

4.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

appointments.csv

Save the following complete contents as `appointments.csv`.

```
appointment_id,scheduled_date,status,completed_date
A01,2026-09-01,completed,2026-09-01
A02,2026-09-01,completed,2026-09-01
A03,2026-09-01,completed,
A04,2026-09-01,cancelled,
A05,2026-09-01,no_show,
A06,2026-09-01,unknown,
A07,2026-09-01,completed,2026-09-01
A07,2026-09-01,completed,2026-09-01
```

appointment_report.py

Complete program. Save this listing as `appointment_report.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'appointments.csv': ['appointment_id', 'scheduled_date', 'status', '
    ↪ completed_date']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20 def parse_day(value):
21     day = _date.fromisoformat(value)
22     if day.isoformat() != value:
```

```

23         raise ValueError("Use exact YYYY-MM-DD dates")
24     return day
25
26
27 def save_rows(name, fields, rows):
28     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
29         writer = csv.DictWriter(stream, fieldnames=fields)
30         writer.writeheader()
31         writer.writerows(rows)
32
33 from collections import Counter
34 from datetime import date
35
36 rows = read_rows("appointments.csv")
37 ids = Counter(r["appointment_id"].strip() for r in rows)
38 output = []
39 allowed = {"completed", "cancelled", "no_show"}
40 for number, row in enumerate(rows, 1):
41     key = row["appointment_id"].strip()
42     status = row["status"].strip().lower()
43     issues = []
44     if not key or ids[key] > 1 or row["appointment_id"] != key:
45         issues.append("blank, padded, or duplicate appointment_id")
46     try:
47         scheduled = parse_day(row["scheduled_date"])
48     except ValueError:
49         issues.append("invalid scheduled date")
50     if status not in allowed:
51         issues.append("unrecognized status")
52     eligible = not issues
53     completed = False
54     if status == "completed":
55         try:
56             finished = parse_day(row["completed_date"])
57             if eligible and finished < scheduled:
58                 issues.append("completion precedes appointment")
59             elif eligible:
60                 completed = True
61         except ValueError:
62             issues.append("missing or invalid completion date")
63     output.append(dict(row, data_row=number,
64                       eligible=int(eligible), completed=int(completed),
65                       unresolved=int(bool(issues)), reason="; ".join(issues)))
66 save_rows("appointment_report.csv", SCHEMAS["appointments.csv"] + ["data_row",
67                               ↪ "eligible", "completed", "unresolved", "reason"], output)
68 denominator = sum(r["eligible"] for r in output)
69 numerator = sum(r["completed"] for r in output)

```

```
69 print("Rows:", len(output))
70 print("Eligible:", denominator)
71 print("Completed with a valid date:", numerator)
72 print("Unresolved:", sum(r["unresolved"] for r in output))
73 print("Proportion:", f"{numerator}/{denominator} = {numerator/denominator:.1%}"
74       if denominator else "not calculable")
```


Measurement Catalogue

A measurement catalogue tells you what is present before you attempt calculations. Here an observation, a person, and an encounter are three different units. The program groups observations by the pair consisting of the measure name and its supplied unit, and creates a second file linking every observation to its catalogue group.

5.1 Run the Reference Version

Working files: complete listings in Chapter 5 below.

Inputs: `measurements.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 measurement_catalogue.py
```

Rule to preserve. Observation IDs must be distinct and required labels must be nonblank and unpadded. Repeated person or encounter references are allowed. Temperature in C and temperature in F are separate catalogue groups. A blank value counts as missing, not as a numerical zero.

5.2 Check the Result

Outputs: `measurement_catalogue.csv`, `observation_lineage.csv`.

Check the meaning

Six observations represent three people and four encounters. There are three measure-unit groups. The pulse group has three supplied observations. Temperature in C has two observations, one supplied and one missing; temperature in F has one supplied observation. The lineage file must still have all six observation IDs.

5.3 Read the Key Lines

```
42 groups = defaultdict(list)
43 for row in rows:
44     groups[(row["measure"], row["unit"])].append(row)
45 catalogue = []
46 for (measure, unit), group in sorted(groups.items()):
47     catalogue.append(dict(measure=measure, unit=unit,
48                           observations=len(group),
49                           supplied=sum(bool(r["value"].strip()) for r in group),
50                           missing=sum(not r["value"].strip() for r in group),
51                           sources="; ".join(sorted({r["source"] for r in group}))))
```

Excerpt from measurement_catalogue.py, source lines 42–51.

A tuple is a grouping key. `(row["measure"], row["unit"])` is one key with two components. Grouping on the measure name alone would merge incompatible supplied units before any conversion rule had been stated.

Distinct counts differ from row counts. A set of person references has three elements, while the list of observations has six. `require_ids` is applied to observation IDs, not to person references. Applying it to the latter would reject legitimate repeated observations.

Presence is a modest claim. `bool(r["value"].strip())` tests whether text is supplied. It does not show that the text is numeric or meaningful. The separate measurement checker in Chapter 6 illustrates a stronger claim.

5.4 Try One Change

Copy the first observation as a new synthetic observation O07, changing its value to 72 but keeping the same person, encounter, measure, unit, and source. Expect seven observations, still three people and four encounters, still three groups, and four observations in the pulse group. The lineage file must gain O07.

5.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

measurements.csv

Save the following complete contents as `measurements.csv`.

```
observation_id,person_ref,encounter_id,measure,value,unit,source
001,S01,E01,pulse,72,beats/min,teaching_A
002,S01,E01,temperature,37.0,C,teaching_A
003,S01,E02,pulse,80,beats/min,teaching_B
004,S02,E03,temperature,,C,teaching_B
005,S02,E03,pulse,75,beats/min,teaching_B
006,S03,E04,temperature,98.6,F,teaching_A
```

measurement_catalogue.py

Complete program. Save this listing as `measurement_catalogue.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'measurements.csv': ['observation_id', 'person_ref', 'encounter_id',
7     ↪ 'measure', 'value', 'unit', 'source']}
8
9 def read_rows(name):
10     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
11         reader = csv.DictReader(stream, strict=True)
12         if reader.fieldnames != SCHEMAS[name]:
13             raise ValueError("Wrong header in " + name)
14         rows = list(reader)
15         if any(None in row or None in row.values() for row in rows):
16             raise ValueError("Malformed CSV row in " + name)
17         return rows
18
19 from datetime import date as _date
20
21 def require_ids(rows, field):
22     keys = [r[field] for r in rows]
23     if any(not k.strip() or k != k.strip() for k in keys):
24         raise ValueError("IDs must be nonblank and unpadded: " + field)
```

```

25     if len(set(keys)) != len(keys):
26         raise ValueError("Duplicate ID: " + field)
27
28 def save_rows(name, fields, rows):
29     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
30         writer = csv.DictWriter(stream, fieldnames=fields)
31         writer.writeheader()
32         writer.writerows(rows)
33
34 from collections import defaultdict
35
36 rows = read_rows("measurements.csv")
37 require_ids(rows, "observation_id")
38 for row in rows:
39     for key in ["person_ref", "encounter_id", "measure", "unit", "source"]:
40         if not row[key].strip() or row[key] != row[key].strip():
41             raise ValueError("Blank or padded label: " + key)
42 groups = defaultdict(list)
43 for row in rows:
44     groups[(row["measure"], row["unit"])].append(row)
45 catalogue = []
46 for (measure, unit), group in sorted(groups.items()):
47     catalogue.append(dict(measure=measure, unit=unit,
48                          observations=len(group),
49                          supplied=sum(bool(r["value"].strip()) for r in group),
50                          missing=sum(not r["value"].strip() for r in group),
51                          sources="; ".join(sorted({r["source"] for r in group}))))
52 save_rows("measurement_catalogue.csv", ["measure", "unit", "observations", "
    ↳ supplied", "missing", "sources"], catalogue)
53 lineage = [dict(observation_id=r["observation_id"],
54                person_ref=r["person_ref"], encounter_id=r["encounter_id"],
55                catalogue_group=r["measure"] + " | " + r["unit"],
56                source=r["source"]) for r in rows]
57 save_rows("observation_lineage.csv", ["observation_id", "person_ref", "
    ↳ encounter_id", "catalogue_group", "source"], lineage)
58 print("Observations:", len(rows))
59 print("People:", len({r["person_ref"] for r in rows}))
60 print("Encounters:", len({r["encounter_id"] for r in rows}))
61 print("Measure-unit groups:", len(catalogue))

```

Measurement Checks

This checker records several kinds of data problem without stopping at the first problematic row. It distinguishes malformed dates, missing values, nonnumeric values, non-finite numbers, unrecognized units, and ID problems. The output says “passes checks,” deliberately not “clinically valid.”

6.1 Run the Reference Version

Working files: complete listings in Chapter 6 below.

Inputs: `measurements_to_check.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 measurement_check.py
```

Rule to preserve. Each of the eight source rows must remain visible. A numeric value must be finite; a date must be an exact valid calendar date; units must be one of the supplied labels C and F. Both duplicate-ID occurrences are flagged. No physiological range, conversion, or treatment rule is implemented.

6.2 Check the Result

Outputs: `measurement_check.csv`.

Check the meaning

M01 and M02 pass the stated checks. The remaining six rows require review: M03 has an impossible date, M04 a missing value, M05 an unrecognized unit, M06 nonnumeric text, and both M07 occurrences a duplicate ID. Count rows with reasons, not the number of semicolon-separated reasons.

6.3 Read the Key Lines

```
50     if not row["value"].strip():
51         reasons.append("missing value")
52     else:
53         try:
54             if not math.isfinite(float(row["value"])):
55                 reasons.append("non-finite value")
56         except ValueError:
57             reasons.append("not numeric")
58     if row["unit"] not in {"C", "F"}:
59         reasons.append("unrecognized unit")
```

Excerpt from measurement_check.py, source lines 50–59.

Conversion can fail. `float` and date parsing can raise `ValueError`. The `try/except` blocks turn these particular failures into row-level reasons, allowing the checker to continue with later rows.

Numeric does not imply finite. Values such as a non-finite floating-point token may convert successfully. The extra `math.isfinite` check makes the numeric contract explicit. This is a different check from testing whether text was supplied.

Schema failure differs from a review row. A wrong header or malformed CSV row raises an exception in `read_rows`. The program can retain a row-level value problem only after it knows which fields that row actually contains.

6.4 Try One Change

For this invented fixture only, stipulate that M05's supplied unit label K was a transcription error and replace it with C. Expect three rows passing the checks and five review rows. This is a stated source correction, not an automatic conversion between units; the program performs no such conversion.

6.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

measurements_to_check.csv

Save the following complete contents as `measurements_to_check.csv`.

```
record_id,event_date,value,unit
M01,2026-09-01,37.0,C
M02,2026-09-01,98.6,F
M03,2026-02-30,37.5,C
M04,2026-09-01,,C
M05,2026-09-01,38.0,K
M06,2026-09-01,warm,C
M07,2026-09-01,37.2,C
M07,2026-09-02,37.3,C
```

measurement_check.py

Complete program. Save this listing as `measurement_check.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'measurements_to_check.csv': ['record_id', 'event_date', 'value', '
    ↪ unit' ]}]
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20 def parse_day(value):
21     day = _date.fromisoformat(value)
22     if day.isoformat() != value:
```

```

23         raise ValueError("Use exact YYYY-MM-DD dates")
24     return day
25
26
27 def save_rows(name, fields, rows):
28     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
29         writer = csv.DictWriter(stream, fieldnames=fields)
30         writer.writeheader()
31         writer.writerows(rows)
32
33 from collections import Counter
34 from datetime import date
35 import math
36
37 rows = read_rows("measurements_to_check.csv")
38 ids = Counter(r["record_id"].strip() for r in rows)
39 report = []
40 for number, row in enumerate(rows, 1):
41     reasons = []
42     if not row["record_id"].strip() or ids[row["record_id"].strip()] > 1:
43         reasons.append("blank or duplicate ID")
44     if row["record_id"] != row["record_id"].strip():
45         reasons.append("padded ID")
46     try:
47         parse_day(row["event_date"])
48     except ValueError:
49         reasons.append("invalid date")
50     if not row["value"].strip():
51         reasons.append("missing value")
52     else:
53         try:
54             if not math.isfinite(float(row["value"])):
55                 reasons.append("non-finite value")
56             except ValueError:
57                 reasons.append("not numeric")
58         if row["unit"] not in {"C", "F"}:
59             reasons.append("unrecognized unit")
60     report.append(dict(row, data_row=number,
61                       status="review" if reasons else "passes checks",
62                       reason="; ".join(reasons)))
63 save_rows("measurement_check.csv", SCHEMAS["measurements_to_check.csv"] + [
64     ↪ data_row", "status", "reason"], report)
65 print("Rows:", len(report))
66 print("Passes checks:", sum(not r["reason"] for r in report))
67 print("Review:", sum(bool(r["reason"]) for r in report))

```

Weekly Teaching Extract

The weekly teaching extract demonstrates selecting a period and exporting only the fields needed for a particular count. The synthetic input contains names, email addresses, and free text. None belongs in the requested output.

7.1 Run the Reference Version

Working files: complete listings in Chapter 7 below.

Inputs: `synthetic_schedule.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 minimal_extract.py
```

Rule to preserve. Include events dated in September 2026, and group them by Monday week start and supplied status. Allowed statuses are completed, cancelled, and no_show. Event IDs must be distinct. The exported columns are exactly `week_start`, `status`, and `count`; names, contacts, and free text are excluded.

7.2 Check the Result

Outputs: `extract_receipt.csv`, `weekly_counts.csv`.

Check the meaning

Five of six source events are in September; one is outside the period. The first grouped week is labelled 2026-08-31 because that Monday begins the week containing early September dates. Its two counts are one completed and one no-show. The week beginning September 7 has two completed and one cancelled event.

7.3 Read the Key Lines

```
45 counts = Counter()
46 included = 0
47 for row in rows:
48     day = parse_day(row["event_date"])
49     if row["status"] not in allowed:
50         raise ValueError("Unexpected status")
51     if day.year == 2026 and day.month == 9:
52         monday = day - timedelta(days=day.weekday())
53         counts[(monday.isoformat(), row["status"])] += 1
54         included += 1
55 report = [dict(week_start=week, status=status, count=count)
56           for (week, status), count in sorted(counts.items())]
```

Excerpt from minimal_extract.py, source lines 45–56.

Filter by event date, label by week. The month test uses the event’s date. Only afterward does `day - timedelta(days=day.weekday())` compute the Monday label. Filtering on the week label instead would incorrectly lose September events from the August 31 week.

Count a pair of categories. `Counter` uses the pair of week and status as its key. Each included row increments exactly one cell. The sorted keys produce a stable output order independent of source order.

Construct an allowed output. The output dictionaries contain only the three requested fields. This is different from copying each source row and attempting to remove sensitive-looking columns afterward.

7.4 Try One Change

Reverse the order of the six input records and run in another fresh copy. The grouped-count CSV should be unchanged byte for byte. Input order is not part of this report’s grouping rule, although a future source-row ledger would need to record the new row positions.

7.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

synthetic_schedule.csv

Save the following complete contents as `synthetic_schedule.csv`.

```
event_id,person_name,email,event_date,status,free_text
E01,Invented One,one@example.invalid,2026-09-01,completed,example note
E02,Invented Two,two@example.invalid,2026-09-02,no_show,example note
E03,Invented Three,three@example.invalid,2026-09-08,completed,example note
E04,Invented Four,four@example.invalid,2026-09-09,cancelled,example note
E05,Invented Five,five@example.invalid,2026-08-31,completed,example note
E06,Invented Six,six@example.invalid,2026-09-08,completed,example note
```

minimal_extract.py

Complete program. Save this listing as `minimal_extract.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'synthetic_schedule.csv': ['event_id', 'person_name', 'email', '
    ↪ event_date', 'status', 'free_text']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20 def parse_day(value):
21     day = _date.fromisoformat(value)
22     if day.isoformat() != value:
23         raise ValueError("Use exact YYYY-MM-DD dates")
24     return day
```

```

25
26 def require_ids(rows, field):
27     keys = [r[field] for r in rows]
28     if any(not k.strip() or k != k.strip() for k in keys):
29         raise ValueError("IDs must be nonblank and unpadded: " + field)
30     if len(set(keys)) != len(keys):
31         raise ValueError("Duplicate ID: " + field)
32
33 def save_rows(name, fields, rows):
34     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
35         writer = csv.DictWriter(stream, fieldnames=fields)
36         writer.writeheader()
37         writer.writerows(rows)
38
39 from collections import Counter
40 from datetime import date, timedelta
41
42 rows = read_rows("synthetic_schedule.csv")
43 require_ids(rows, "event_id")
44 allowed = {"completed", "cancelled", "no_show"}
45 counts = Counter()
46 included = 0
47 for row in rows:
48     day = parse_day(row["event_date"])
49     if row["status"] not in allowed:
50         raise ValueError("Unexpected status")
51     if day.year == 2026 and day.month == 9:
52         monday = day - timedelta(days=day.weekday())
53         counts[(monday.isoformat(), row["status"])] += 1
54         included += 1
55 report = [dict(week_start=week, status=status, count=count)
56           for (week, status), count in sorted(counts.items())]
57 save_rows("weekly_counts.csv", ["week_start", "status", "count"], report)
58 receipt = [dict(item="input_rows", value=len(rows)),
59            dict(item="included_rows", value=included),
60            dict(item="outside_period", value=len(rows)-included),
61            dict(item="export_columns", value="week_start; status; count")]
62 save_rows("extract_receipt.csv", ["item", "value"], receipt)
63 print("Input rows:", len(rows))
64 print("September rows:", included)
65 print("Outside period:", len(rows)-included)
66 print("Exported columns: week_start, status, count")

```

Follow-Up Matching

This application links candidate follow-ups to encounters. The relationship is defined by encounter ID, not by a shared person reference or a guess based on proximity. The program retains a summary for every encounter, a ledger of candidates with known encounters, and a separate file for unknown references.

8.1 Run the Reference Version

Working files: complete listings in Chapter 8 below.

Inputs: `encounters.csv`, `followups.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 followup_match.py
```

Rule to preserve. With the default window, elapsed days 0 through 14 are inclusive. Among eligible candidates for one encounter, choose the smallest pair of elapsed days and follow-up ID. Both input files require distinct record IDs. An unknown encounter reference goes to the separate unmatched file, not into a newly invented encounter.

8.2 Check the Result

Outputs: `followup_candidates.csv`, `followup_matches.csv`, `unmatched_followups.csv`.

Check the meaning

E01 selects F01 at day 0; E02 selects F02 at day 14; E05 selects F05 at day 3 rather than F06 at day 5. E03's F03 is at day 15 and E04's F04 is at day -1, so neither is selected. E06 has no candidate. F07 refers to unknown E99 and must appear in the unmatched file.

8.3 Read the Key Lines

```

65     for event in followups:
66         if event["encounter_id"] != encounter["encounter_id"]:
67             continue
68         days = (parse_day(event["followup_date"]) - start).days
69         eligible = 0 <= days <= args.window_days
70         candidates.append(dict(encounter_id=encounter["encounter_id"],
71                               followup_id=event["followup_id"], elapsed_days=days,
72                               within_window=int(eligible)))
73         if eligible:
74             valid.append((days, event["followup_id"]))
75     valid.sort()
76     matches.append(dict(encounter_id=encounter["encounter_id"],
77                       candidates_in_window=len(valid),
78                       selected=valid[0][1] if valid else "",
79                       elapsed_days=valid[0][0] if valid else "",
80                       status="matched" if valid else "no match in window"))

```

Excerpt from followup_match.py, source lines 65–80.

Boundary comparisons carry the rule. `0 <= days <= args.window_days` includes both endpoints. The date library supplies a difference in days; the comparison supplies the inclusion decision.

Tuple sorting supplies a tie-break. Each valid candidate is stored as `(days, followup_id)`. Sorting these pairs gives an earliest-day choice and then an ID tie-break. Source order does not become an accidental priority rule.

Not selected is not the same as unmatched. A known candidate can be eligible but not selected, outside the window, or before the index date. The final unmatched file has the narrower meaning of an unknown encounter reference. These categories should not be collapsed.

8.4 Try One Change

Run `python3 followup_match.py --window-days 15` in a fresh copy. E03 should now select F03 and the matched encounter count should rise from three to four. E04 remains before the index date; F07 remains an unknown reference. The changed result belongs to a different, explicitly stated teaching contract.

8.5 Convert and Match the Large Journey File

The additional complete converter printed below is `journey_matching_inputs.py`. Keep it with the reader and checker from Chapters 1 and 2 and the downloaded `journey_large.csv`. It validates every journey row, rejects review records, and

writes the matching inputs into a new folder.

```
python3 journey_matching_inputs.py journey_large.csv large_matching_14
```

The result contains `encounters.csv` with 3,000 visits and `followups.csv` with 3,000 follow-ups. The `journey_conversion_ledger.csv` preserves all 12,000 events, including 6,000 tests and measurements not used in matching. `conversion_record.json` saves the conversion rule, counts, and file identities. No fourteen-day condition is imposed during conversion.

Put an unchanged copy of this chapter's `followup_match.py` in the new `large_matching_14` folder, open a terminal there, and run:

```
python3 followup_match.py --window-days 14
```

Expect 3,000 encounters, 1,684 matches, 1,316 without an in-window match, and zero unknown encounter references. For an independent comparison, repeat the conversion into `large_matching_21`, put the same matcher there, and run it in that folder with `--window-days 21`. All 3,000 encounters then match. Save both window choices and their outputs.

The converter's selection is by event type under a validated encounter relationship, not by guessing from the detail text. It preserves the original event ID as the follow-up ID. C000003's follow-up, E000012, is twenty days after its visit: excluded from the fourteen-day selection, retained as a candidate, and selected at twenty-one days.

8.6 Complete Working Files

The complete program and small inputs are printed below. Keep the named files together.

encounters.csv

Save the following complete contents as `encounters.csv`.

```
encounter_id,person_ref,index_date
E01,S01,2026-09-01
E02,S02,2026-09-01
E03,S03,2026-09-01
E04,S04,2026-09-01
E05,S05,2026-09-01
E06,S06,2026-09-01
```

followups.csv

Save the following complete contents as `followups.csv`.

```
followup_id,encounter_id,followup_date
F01,E01,2026-09-01
F02,E02,2026-09-15
F03,E03,2026-09-16
F04,E04,2026-08-31
F05,E05,2026-09-04
F06,E05,2026-09-06
F07,E99,2026-09-03
```

followup_match.py

Complete program. Save this listing as `followup_match.py`.

```
1  import argparse
2  import csv
3  from pathlib import Path
4
5  ROOT = Path(__file__).resolve().parent
6  DEFAULT_WINDOW_DAYS = 14
7
8  SCHEMAS = {'followups.csv': ['followup_id', 'encounter_id', 'followup_date'], '
    ↪ encounters.csv': ['encounter_id', 'person_ref', 'index_date']}
9
10 def read_rows(name):
11     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
12         reader = csv.DictReader(stream, strict=True)
13         if reader.fieldnames != SCHEMAS[name]:
14             raise ValueError("Wrong header in " + name)
15         rows = list(reader)
16         if any(None in row or None in row.values() for row in rows):
17             raise ValueError("Malformed CSV row in " + name)
18         return rows
19
20 from datetime import date as _date
21
22 def parse_day(value):
23     day = _date.fromisoformat(value)
24     if day.isoformat() != value:
25         raise ValueError("Use exact YYYY-MM-DD dates")
26     return day
27
28 def require_ids(rows, field):
29     keys = [r[field] for r in rows]
```

```

30     if any(not k.strip() or k != k.strip() for k in keys):
31         raise ValueError("IDs must be nonblank and unpadded: " + field)
32     if len(set(keys)) != len(keys):
33         raise ValueError("Duplicate ID: " + field)
34
35     def save_rows(name, fields, rows):
36         with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
37             writer = csv.DictWriter(stream, fieldnames=fields)
38             writer.writeheader()
39             writer.writerows(rows)
40
41     from datetime import date
42
43     parser = argparse.ArgumentParser(description="Match synthetic encounter follow-
44         ↪ ups")
45     parser.add_argument("--window-days", type=int, default=DEFAULT_WINDOW_DAYS)
46     args = parser.parse_args()
47     if args.window_days < 0:
48         parser.error("Window must be nonnegative")
49     encounters = read_rows("encounters.csv")
50     followups = read_rows("followups.csv")
51     require_ids(encounters, "encounter_id")
52     require_ids(followups, "followup_id")
53     for row in encounters:
54         if not row["person_ref"].strip() or row["person_ref"] != row["person_ref"].
55             ↪ strip():
56             raise ValueError("Nonblank unpadded person reference required")
57     for row in followups:
58         if not row["encounter_id"].strip() or row["encounter_id"] != row["
59             ↪ encounter_id"].strip():
60             raise ValueError("Nonblank unpadded encounter reference required")
61         parse_day(row["followup_date"])
62     known = {r["encounter_id"] for r in encounters}
63     matches = []
64     candidates = []
65     for encounter in encounters:
66         start = parse_day(encounter["index_date"])
67         valid = []
68         for event in followups:
69             if event["encounter_id"] != encounter["encounter_id"]:
70                 continue
71             days = (parse_day(event["followup_date"]) - start).days
72             eligible = 0 <= days <= args.window_days
73             candidates.append(dict(encounter_id=encounter["encounter_id"],
74                 followup_id=event["followup_id"], elapsed_days=days,
75                 within_window=int(eligible)))
76             if eligible:

```

```

74         valid.append((days, event["followup_id"]))
75     valid.sort()
76     matches.append(dict(encounter_id=encounter["encounter_id"],
77         candidates_in_window=len(valid),
78         selected=valid[0][1] if valid else "",
79         elapsed_days=valid[0][0] if valid else "",
80         status="matched" if valid else "no match in window"))
81     save_rows("followup_matches.csv", ["encounter_id", "candidates_in_window", "
    ↳ selected", "elapsed_days", "status"], matches)
82     chosen = {r["selected"] for r in matches if r["selected"]}
83     for row in candidates:
84         row["selected"] = int(row["followup_id"] in chosen)
85         row["reason"] = ("selected" if row["selected"] else
86             "eligible; not selected by day/ID rule" if row["
    ↳ within_window"] else
87                 "before index date" if row["elapsed_days"] < 0 else
88                 "after window")
89     save_rows("followup_candidates.csv",
90         ["encounter_id", "followup_id", "elapsed_days", "within_window", "
    ↳ selected", "reason"], candidates)
91     orphans = [r for r in followups if r["encounter_id"] not in known]
92     save_rows("unmatched_followups.csv",
93         ["followup_id", "encounter_id", "followup_date"], orphans)
94     print("Encounters:", len(matches))
95     print("Matched:", sum(r["status"] == "matched" for r in matches))
96     print("No match in window:", sum(r["status"] != "matched" for r in matches))
97     print("Unknown encounter references:", len(orphans))

```

journey_matching_inputs.py

Complete program. Save this listing as journey_matching_inputs.py.

```

1  """Prepare Chapter 8 inputs from a checked synthetic journey file.
2
3  This conversion does not apply a matching window. It preserves all source
4  rows in a ledger and leaves the original file unchanged.
5  """
6  import argparse
7  import csv
8  import hashlib
9  import json
10     from collections import Counter
11     from pathlib import Path
12
13     import journey_check
14     import journey_view

```

```

15 from journey_check import check_events
16 from journey_view import FIELDS, read_events
17
18
19 def save_rows(path: Path, fields: list[str], rows: list[dict]) -> None:
20     with path.open("x", newline="", encoding="utf-8") as stream:
21         writer = csv.DictWriter(stream, fieldnames=fields)
22         writer.writeheader()
23         writer.writerows(rows)
24
25
26 def prepare(source: Path, destination: Path) -> dict[str, int]:
27     source, destination = Path(source), Path(destination)
28     rows = read_events(source)
29     checked = check_events(rows)
30     review = [row for row in checked if row["status"] != "placed"]
31     if review:
32         raise ValueError("Resolve journey review rows before this conversion")
33
34     encounters, followups, ledger = [], [], []
35     for number, row in enumerate(rows, 1):
36         target, identifier, use = "", "", "not used for matching"
37         if row["event_type"] == "visit":
38             target, identifier, use = "encounters.csv", row["encounter_id"], "
↳ encounter"
39             encounters.append(dict(encounter_id=row["encounter_id"],
40                                   person_ref=row["person_ref"], index_date=row["event_date"]))
41         elif row["event_type"] == "followup":
42             target, identifier, use = "followups.csv", row["event_id"], "follow
↳ -up"
43             followups.append(dict(followup_id=row["event_id"],
44                                  encounter_id=row["encounter_id"], followup_date=row["event_date"]
↳ ))
45             ledger.append(dict(row, data_row=number, used_as=use,
46                               output_file=target, output_id=identifier))
47
48     encounters.sort(key=lambda row: row["encounter_id"])
49     followups.sort(key=lambda row: row["followup_id"])
50     counts = dict(input_events=len(rows), encounters=len(encounters),
51                   followups=len(followups),
52                   not_used=len(rows)-len(encounters)-len(followups))
53     assert sum(Counter(row["used_as"] for row in ledger).values()) == len(rows)
54     # Validate first; never reuse an existing output directory.
55     destination.mkdir(parents=False, exist_ok=False)
56     save_rows(destination / "encounters.csv",
57               ["encounter_id", "person_ref", "index_date"], encounters)
58     save_rows(destination / "followups.csv",

```

```

59         ["followup_id", "encounter_id", "followup_date"], followups)
60     save_rows(destination / "journey_conversion_ledger.csv",
61             FIELDS+["data_row", "used_as", "output_file", "output_id"],
62             ↪ ledger)
63     modules = [Path(__file__), Path(journey_view.__file__),
64             Path(journey_check.__file__)]
65     record = dict(input_file=source.name,
66                 input_sha256=hashlib.sha256(source.read_bytes()).hexdigest(),
67                 program_sha256={p.name: hashlib.sha256(p.read_bytes()).hexdigest()
68                               ↪ for p in modules},
69                 rule="Visits become encounters; followup events become candidates; "
70                 ↪ "tests and measurements remain in the source ledger", **counts)
71     with (destination / "conversion_record.json").open("x", encoding="utf-8")
72     ↪ as stream:
73         json.dump(record, stream, indent=2)
74     return counts
75
76 def main() -> None:
77     parser = argparse.ArgumentParser(description=__doc__)
78     parser.add_argument("input_csv")
79     parser.add_argument("new_output_folder")
80     args = parser.parse_args()
81     try:
82         counts = prepare(Path(args.input_csv), Path(args.new_output_folder))
83     except (ValueError, OSError) as error:
84         parser.exit(1, str(error)+"\n")
85     for key, value in counts.items():
86         print(key+":", value)
87
88 if __name__ == "__main__":
89     main()

```

Cohort Summary

The cohort summary separates membership in a population from availability of a particular measurement. The input contains one row per person. The output includes a disposition ledger, a count of dispositions, and summary measures with their respective population sizes.

9.1 Run the Reference Version

Working files: complete listings in Chapter 9 below.

Inputs: `cohort_candidates.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 cohort_summary.py
```

Rule to preserve. A person is included when age is supplied and at least 18, and enrollment is yes. Missing age is unresolved rather than assumed adult. Exclusion reasons are assigned in the order age unresolved, under 18, not enrolled, included. Person references must be unique. Observed waits must be finite and nonnegative.

9.2 Check the Result

Outputs: `cohort_flow.csv`, `cohort_ledger.csv`, `cohort_summary.csv`.

Check the meaning

Twelve candidates yield eight eligible people. Six of the eight have recorded follow-up, and six have observed waits. The median of those six waits is 9 days. S04 and S09 are eligible despite missing waits; S07 is not silently included with an invented age. The ledger must contain all twelve person references.

9.3 Read the Key Lines

```
64 waits = [float(r["wait_days"]) for r in eligible if r["wait_days"].strip()]
65 followed = sum(r["followup_recorded"] == "yes" for r in eligible)
66 summary = [
67     dict(measure="candidates", value=len(rows), basis_n=len(rows)),
68     dict(measure="eligible", value=len(eligible), basis_n=len(rows)),
69     dict(measure="followup_recorded", value=followed, basis_n=len(eligible)),
70     dict(measure="observed_waits", value=len(waits), basis_n=len(eligible)),
71     dict(measure="median_wait_days", value=statistics.median(waits) if waits
72         ↪ else "",
73         basis_n=len(waits))]
```

Excerpt from cohort_summary.py, source lines 64–72.

Two selections answer different questions. The eligible list is formed from membership rules. The waits list then selects observed measurements within that population. Computing the latter directly from all source rows would include waits from excluded candidates.

A median has a basis, not a proportion denominator. `statistics.median(waits)` uses the observed values. The `basis_n` field reports how many values support it. The recorded-follow-up count has the eligible cohort as its basis instead.

Missing information is not zero. The blank wait test prevents a missing value from becoming zero. Zero would be a genuine observed wait and would affect the median. A blank age likewise prevents the membership decision from being made.

9.4 Try One Change

Stipulate an age of 18 for synthetic S07 and change that one field. Expect nine eligible people, seven recorded follow-ups, and seven observed waits. Their median becomes 8 days. This change affects both cohort membership and the observed-value list, so retaining the old median would be a warning sign.

9.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

cohort_candidates.csv

Save the following complete contents as `cohort_candidates.csv`.

```
person_ref,age_years,enrolled,followup_recorded,wait_days
S01,25,yes,yes,4
S02,40,yes,yes,8
S03,17,yes,yes,3
S04,65,yes,no,
S05,50,no,no,6
S06,18,yes,yes,10
S07,,yes,yes,5
S08,73,yes,yes,12
S09,35,yes,no,
S10,29,yes,yes,6
S11,44,no,yes,7
S12,61,yes,yes,14
```

cohort_summary.py

Complete program. Save this listing as `cohort_summary.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'cohort_candidates.csv': ['person_ref', 'age_years', 'enrolled', '
    ↪ followup_recorded', 'wait_days']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
```

```

19
20
21 def require_ids(rows, field):
22     keys = [r[field] for r in rows]
23     if any(not k.strip() or k != k.strip() for k in keys):
24         raise ValueError("IDs must be nonblank and unpadded: " + field)
25     if len(set(keys)) != len(keys):
26         raise ValueError("Duplicate ID: " + field)
27
28 def save_rows(name, fields, rows):
29     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
30         writer = csv.DictWriter(stream, fieldnames=fields)
31         writer.writeheader()
32         writer.writerows(rows)
33
34 import statistics
35 from collections import Counter
36
37 rows = read_rows("cohort_candidates.csv")
38 require_ids(rows, "person_ref")
39 import math
40 for row in rows:
41     if row["age_years"].strip():
42         age = int(row["age_years"])
43         if age < 0:
44             raise ValueError("Age must be a nonnegative integer")
45     if row["wait_days"].strip():
46         wait = float(row["wait_days"])
47         if not math.isfinite(wait) or wait < 0:
48             raise ValueError("Waiting time must be finite and nonnegative")
49 flow = []
50 eligible = []
51 for row in rows:
52     if row["enrolled"] not in {"yes", "no"} or row["followup_recorded"] not in {
53         ↪ "yes", "no"}:
54         raise ValueError("Unexpected yes/no value")
55     if not row["age_years"].strip():
56         reason = "age unresolved"
57     elif int(row["age_years"]) < 18:
58         reason = "under 18"
59     elif row["enrolled"] != "yes":
60         reason = "not enrolled"
61     else:
62         reason = "included"
63     eligible.append(row)
64     flow.append(dict(person_ref=row["person_ref"], disposition=reason))
65 waits = [float(r["wait_days"]) for r in eligible if r["wait_days"].strip()]

```

```

65 followed = sum(r["followup_recorded"] == "yes" for r in eligible)
66 summary = [
67     dict(measure="candidates", value=len(rows), basis_n=len(rows)),
68     dict(measure="eligible", value=len(eligible), basis_n=len(rows)),
69     dict(measure="followup_recorded", value=followed, basis_n=len(eligible)),
70     dict(measure="observed_waits", value=len(waits), basis_n=len(eligible)),
71     dict(measure="median_wait_days", value=statistics.median(waits) if waits
    ↪ else "",
72         basis_n=len(waits))]
73 counts = Counter(r["disposition"] for r in flow)
74 flow_summary = [dict(disposition=reason, count=counts[reason])
75     for reason in ["age unresolved", "under 18", "not enrolled", "included"]]
76 save_rows("cohort_ledger.csv", ["person_ref", "disposition"], flow)
77 save_rows("cohort_flow.csv", ["disposition", "count"], flow_summary)
78 save_rows("cohort_summary.csv", ["measure", "value", "basis_n"], summary)
79 for row in summary:
80     print(f'{row["measure"]}: {row["value"]} (basis n={row["basis_n"]})')

```


Monthly Waiting-Time Report

The waiting-time report builds a complete twelve-month view even though one month has no supplied observations. It writes a numeric table and an SVG chart from the same summary. The missing month remains a gap, not a zero and not a line interpolated between neighboring months.

10.1 Run the Reference Version

Working files: complete listings in Chapter 10 below.

Inputs: `waiting_times.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 waiting_time_report.py
```

Rule to preserve. The report covers the twelve months of 2026. Month labels must be exact YYYY-MM strings; waits must be finite and nonnegative. Each month gets an output row containing its count and, only when values exist, its median. Missing months break the plotted line.

10.2 Check the Result

Outputs: `monthly_waits.csv`, `waiting_time_report.svg`.

Check the meaning

There are eleven observed months. July has count 0 and an empty median, while June's median is 11 and August's is 10. The chart must not draw a point at zero for July or connect June directly to August. January has two observations and a median of 6.

10.3 Read the Key Lines

```
43 summary = []
44 for m in range(1,13):
45     key = f"2026-{m:02}"
46     values = groups[key]
47     summary.append(dict(month=key, count=len(values),
48                         median_days=statistics.median(values) if values else ""))
49 save_rows("monthly_waits.csv", ["month","count","median_days"], summary)
```

Excerpt from waiting_time_report.py, source lines 43–49.

Enumerate the reporting frame explicitly. The loop over `range(1,13)` creates every month, not just keys that happened to appear in the input. This is how the program makes an absent month visible.

Choose the empty-group representation. An empty values list gives count 0 and an empty median field. The code does not call the median function on that empty list or supply a made-up result.

The chart state has a meaning. `previous` stores the previous plotted point. When the month has no observations it is reset to `None`, preventing a segment from bridging the gap.

10.4 Try One Change

Add one invented July observation with wait 0. July should now have count 1 and median 0, and the chart should have an actual July point. This is deliberately different from the baseline's missing July. The exercise tests representation of supplied zero, not a proposal to fill missing real records with zero.

10.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

waiting_times.csv

Save the following complete contents as `waiting_times.csv`.

```
month,wait_days
2026-01,4
2026-01,8
2026-02,5
2026-02,9
2026-03,6
2026-03,10
2026-04,6
2026-04,8
2026-05,8
2026-05,10
2026-06,10
2026-06,12
2026-08,9
2026-08,11
2026-09,8
2026-09,10
2026-10,7
2026-10,9
2026-11,6
2026-11,8
2026-12,5
2026-12,7
```

waiting_time_report.py

Complete program. Save this listing as `waiting_time_report.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'waiting_times.csv': ['month', 'wait_days']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
```

```

10     reader = csv.DictReader(stream, strict=True)
11     if reader.fieldnames != SCHEMAS[name]:
12         raise ValueError("Wrong header in " + name)
13     rows = list(reader)
14     if any(None in row or None in row.values() for row in rows):
15         raise ValueError("Malformed CSV row in " + name)
16     return rows
17
18 from datetime import date as _date
19
20
21
22 def save_rows(name, fields, rows):
23     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
24         writer = csv.DictWriter(stream, fieldnames=fields)
25         writer.writeheader()
26         writer.writerows(rows)
27
28 from collections import defaultdict
29 from datetime import datetime
30 import math, statistics
31
32 groups = defaultdict(list)
33 for row in read_rows("waiting_times.csv"):
34     month = row["month"]
35     if datetime.strptime(month, "%Y-%m").strftime("%Y-%m") != month:
36         raise ValueError("Use YYYY-MM")
37     if not month.startswith("2026-"):
38         raise ValueError("This report covers 2026")
39     value = float(row["wait_days"])
40     if not math.isfinite(value) or value < 0:
41         raise ValueError("Waiting times must be finite and nonnegative")
42     groups[month].append(value)
43 summary = []
44 for m in range(1,13):
45     key = f"2026-{m:02}"
46     values = groups[key]
47     summary.append(dict(month=key, count=len(values),
48         median_days=statistics.median(values) if values else ""))
49 save_rows("monthly_waits.csv", ["month", "count", "median_days"], summary)
50 maximum = max([r["median_days"] for r in summary if r["count"]] or [1])
51 top = max(5, math.ceil(maximum / 5) * 5)
52 svg = ['<svg xmlns="http://www.w3.org/2000/svg" width="720" height="360"
53     ↪ viewBox="0 0 720 360">',
54     '<rect width="720" height="360" fill="white"/>',
55     '<text x="50" y="25">Synthetic median waiting time: 2026</text>']
56 for value in range(0,top+1,5):

```

```

56     y = 280 - 220*value/top
57     svg.append(f'<path d="M50 {y} H690" stroke="#ddd"/>')
58     svg.append(f'<text x="15" y="{y+4}">{value}</text>')
59 previous = None
60 for i,row in enumerate(summary):
61     x = 65 + 55*i
62     svg.append(f'<text x="{x-7}" y="305">{i+1}</text>')
63     svg.append(f'<text x="{x-10}" y="325">n={row["count"]}</text>')
64     if not row["count"]:
65         previous = None
66         continue
67     y = 280 - 220*row["median_days"]/top
68     if previous:
69         svg.append(f'<path d="M{previous[0]} {previous[1]} L{x} {y}" stroke
    ↪="#175078" stroke-width="2"/>')
70         svg.append(f'<circle cx="{x}" cy="{y}" r="4" fill="#175078"/>')
71         previous = (x,y)
72 svg.append('<text x="270" y="350">Month; vertical scale in days</text></svg>')
73 (ROOT/"waiting_time_report.svg").write_text("\n".join(svg),encoding="utf-8")
74 print("Observed months:", sum(r["count"] > 0 for r in summary))
75 print("Missing months:", ", ".join(r["month"] for r in summary if not r["count"]
    ↪ ))
76 print("Saved monthly_waits.csv and waiting_time_report.svg")

```


Weekly Follow-Up Monitor

The weekly monitor starts with supplied aggregate counts. It preserves provisional weeks but excludes them from the final pooled result. Its main calculation is a ratio of totals, not the unweighted average of weekly percentages.

11.1 Run the Reference Version

Working files: complete listings in Chapter 11 below.

Inputs: `weekly_followup.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 weekly_followup.py
```

Rule to preserve. Week numbers must be distinct positive integers. Counts must be nonnegative integers, and completions cannot exceed the eligible count. A zero eligible count produces an empty weekly rate. Only rows with `provisional=no` contribute to the final pooled numerator and denominator.

11.2 Check the Result

Outputs: `weekly_followup_report.csv`, `weekly_summary.csv`.

Check the meaning

The ten final weeks contribute 105 completions among 130 eligible records: 80.8% at one decimal place. Weeks 11 and 12 remain in the detailed report as provisional. The unresolved counts and the provisional flags are supplied information, not values inferred from today's date.

11.3 Read the Key Lines

```

45 final=[r for r in report if r["provisional"]=="no"]
46 num=sum(r["completed"] for r in final); den=sum(r["eligible"] for r in final)
47 summary=[dict(measure="final_weeks",value=len(final)),
48           dict(measure="final_completed",value=num),
49           dict(measure="final_eligible",value=den),
50           dict(measure="pooled_final_percent",value=round(100*num/den,1) if den
    ↪ else ""),
51           dict(measure="provisional_weeks",value=sum(r["provisional"]=="yes" for
    ↪ r in report))]

```

Excerpt from weekly_followup.py, source lines 45–51.

Keep the detailed report before pooling. The full report is saved first. The `final` list is a second view selected from it. Excluding provisional weeks from a final summary therefore does not remove them from the handover.

Pool counts, not rounded percentages. The program sums `completed` and `eligible` and divides once. Averaging `rate_percent` would give each week equal weight and would also use already-rounded values.

Do not infer meanings absent from the input. The program checks that unresolved counts are nonnegative, but does not derive them or equate them with `eligible` minus `completed`. The source contract must say what they count.

11.4 Try One Change

Change only week 11's provisional flag to no. The final totals become 113 completions among 142 eligible records, or 79.6%; one provisional week remains. Do not change the counts merely to keep the original percentage.

11.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

weekly_followup.csv

Save the following complete contents as `weekly_followup.csv`.

```
week,completed,eligible,unresolved,provisional
1,8,10,0,no
2,9,12,1,no
3,7,10,0,no
4,11,14,1,no
5,12,15,0,no
6,10,12,0,no
7,9,11,1,no
8,13,16,0,no
9,12,14,0,no
10,14,16,0,no
11,8,12,2,yes
12,5,10,3,yes
```

weekly_followup.py

Complete program. Save this listing as `weekly_followup.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'weekly_followup.csv': ['week', 'completed', 'eligible', 'unresolved',
    ↪ 'provisional']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
```

```

19
20
21
22 def save_rows(name, fields, rows):
23     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
24         writer = csv.DictWriter(stream, fieldnames=fields)
25         writer.writeheader()
26         writer.writerows(rows)
27
28 import statistics
29
30 report=[]
31 seen=set()
32 for row in read_rows("weekly_followup.csv"):
33     week=int(row["week"]); complete=int(row["completed"])
34     eligible=int(row["eligible"]); unresolved=int(row["unresolved"])
35     if week in seen or week<1 or min(complete,eligible,unresolved)<0 or
    ↪ complete>eligible:
36         raise ValueError("Invalid weekly counts or duplicate week")
37     seen.add(week)
38     if row["provisional"] not in {"yes","no"}:
39         raise ValueError("Invalid provisional flag")
40     report.append(dict(week=week,completed=complete,eligible=eligible,
41                       rate_percent=round(100*complete/eligible,1) if eligible else "",
42                       unresolved=unresolved,provisional=row["provisional"]))
43 report.sort(key=lambda r:r["week"])
44 save_rows("weekly_followup_report.csv",["week", "completed", "eligible", "
    ↪ rate_percent", "unresolved", "provisional"],report)
45 final=[r for r in report if r["provisional"]=="no"]
46 num=sum(r["completed"] for r in final); den=sum(r["eligible"] for r in final)
47 summary=[dict(measure="final_weeks",value=len(final)),
48           dict(measure="final_completed",value=num),
49           dict(measure="final_eligible",value=den),
50           dict(measure="pooled_final_percent",value=round(100*num/den,1) if den
    ↪ else ""),
51           dict(measure="provisional_weeks",value=sum(r["provisional"]=="yes" for
    ↪ r in report))]
52 save_rows("weekly_summary.csv",["measure","value"],summary)
53 for row in summary: print(f'{row["measure"]}: {row["value"]}')

```

Clinic Schedules

The schedule comparison uses one invented arrival file to run two non-preemptive service rules. One chooses the earliest waiting arrival; the other gives precedence to already-waiting records bearing the supplied urgent label. The program does not assign that label or implement clinical triage.

12.1 Run the Reference Version

Working files: complete listings in Chapter 12 below.

Inputs: `clinic_arrivals.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 schedule_comparison.py
```

Rule to preserve. There is one server, positive integer service duration, and no interruption of a visit already in service. Only arrived visits are eligible to be chosen. Ties are resolved by arrival time and visit ID. A server with nobody waiting advances to the next arrival time.

12.2 Check the Result

Outputs: `schedule_detail.csv`, `schedule_summary.csv`.

Check the meaning

First-come order is A, B, C, D, E, F. The alternative order is A, C, E, B, D, F. Mean waits are 18 and 15.5 minutes, but maximum waits are 25 and 28. D waits 23 minutes under the first rule and 28 under the second. Those figures describe this invented scenario, not a generally preferable policy.

12.3 Read the Key Lines

```

47     ready=[r for r in pending if r["arrival_minute"]<=clock]
48     if not ready:
49         clock=min(r["arrival_minute"] for r in pending)
50         continue
51     if rule=="urgent_first":
52         key=lambda r:(r["priority"]!="urgent",r["arrival_minute"],r["
↪ visit_id"])
53     else:
54         key=lambda r:(r["arrival_minute"],r["visit_id"])
55     chosen=min(ready,key=key)

```

Excerpt from schedule_comparison.py, source lines 47–55.

Simulation state is explicit. `pending` holds unserved visits and `clock` records the next available service time. A visit is removed exactly once after its finish time is calculated.

Do not choose a future arrival. The `ready` list is formed before the selection key is applied. Sorting the entire original file by priority would let a future arrival jump ahead of someone who could already be served.

Boolean keys can express precedence. For the alternative rule, `priority != "urgent"` sorts supplied urgent records before routine ones. Arrival time and ID complete the key and make ties deterministic.

12.4 Try One Change

Set all six supplied priority labels to routine in a fresh copy. The two rules should now produce the same order and the same overall mean of 18 minutes. Both urgent-group means should be empty. This checks that the difference between the rules really comes from the supplied labels.

12.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

clinic_arrivals.csv

Save the following complete contents as `clinic_arrivals.csv`.

```
visit_id,arrival_minute,service_minutes,priority
A,0,20,routine
B,5,10,routine
C,10,5,urgent
D,12,10,routine
E,20,5,urgent
F,25,10,routine
```

schedule_comparison.py

Complete program. Save this listing as `schedule_comparison.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'clinic_arrivals.csv': ['visit_id', 'arrival_minute', '
    ↪ service_minutes', 'priority']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20
21 def require_ids(rows, field):
22     keys = [r[field] for r in rows]
23     if any(not k.strip() or k != k.strip() for k in keys):
24         raise ValueError("IDs must be nonblank and unpadded: " + field)
```

```

25     if len(set(keys)) != len(keys):
26         raise ValueError("Duplicate ID: " + field)
27
28 def save_rows(name, fields, rows):
29     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
30         writer = csv.DictWriter(stream, fieldnames=fields)
31         writer.writeheader()
32         writer.writerows(rows)
33
34 import statistics
35
36 visits=read_rows("clinic_arrivals.csv")
37 require_ids(visits, "visit_id")
38 for row in visits:
39     row["arrival_minute"]=int(row["arrival_minute"])
40     row["service_minutes"]=int(row["service_minutes"])
41     if row["arrival_minute"]<0 or row["service_minutes"]<=0 or row["priority"]
↳ not in {"routine","urgent"}:
42         raise ValueError("Invalid visit")
43 def simulate(rule):
44     pending=[dict(r) for r in visits]
45     clock=0; records=[]
46     while pending:
47         ready=[r for r in pending if r["arrival_minute"]<=clock]
48         if not ready:
49             clock=min(r["arrival_minute"] for r in pending)
50             continue
51         if rule=="urgent_first":
52             key=lambda r:(r["priority"]!="urgent",r["arrival_minute"],r["
↳ visit_id"])
53         else:
54             key=lambda r:(r["arrival_minute"],r["visit_id"])
55         chosen=min(ready,key=key)
56         start=clock; clock+=chosen["service_minutes"]
57         records.append(dict(rule=rule,visit_id=chosen["visit_id"],
58             priority=chosen["priority"],arrival=chosen["arrival_minute"],
59             start=start,finish=clock,wait=start-chosen["arrival_minute"]))
60         pending.remove(chosen)
61     return records
62 def mean_or_blank(values):
63     values = list(values)
64     return round(statistics.mean(values), 2) if values else ""
65
66 details=[]; summary=[]
67 for rule in ["first_come","urgent_first"]:
68     result=simulate(rule); details.extend(result)
69     summary.append(dict(rule=rule,

```

```
70     mean_wait=mean_or_blank(r["wait"] for r in result),
71     maximum_wait=max((r["wait"] for r in result), default=""),
72     urgent_mean=mean_or_blank(r["wait"] for r in result if r["priority"]=="
↪ urgent"),
73     routine_mean=mean_or_blank(r["wait"] for r in result if r["priority"]=="
↪ "routine"))))
74 save_rows("schedule_detail.csv",["rule", "visit_id", "priority", "arrival", "
↪ start", "finish", "wait"],details)
75 save_rows("schedule_summary.csv",["rule", "mean_wait", "maximum_wait", "
↪ urgent_mean", "routine_mean"],summary)
76 for row in summary:
77     print(f'{row["rule"]}: mean {row["mean_wait"]}; max {row["maximum_wait"]};
↪ urgent {row["urgent_mean"]}; routine {row["routine_mean"]}')

```


Appointment Letters

This program creates a local HTML pack of appointment letters from invented requests. It checks the required details, holds incomplete or invalid requests, and records every request in a build report. It does not connect to an email service, print automatically, or send messages.

13.1 Run the Reference Version

Working files: complete listings in Chapter 13 below.

Inputs: `letter_requests.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 appointment_letters.py
```

Rule to preserve. Appointment IDs are distinct. A letter requires a name, exact date and time, location, and contact. Incomplete requests are held with reasons rather than completed by invented details. Supplied text is escaped before being inserted into HTML.

13.2 Check the Result

Outputs: `appointment_letters.html`, `letter_build_report.csv`.

Check the meaning

L01, L02, and L03 generate letters. L04 is held because its contact field is blank. The build report contains all four request IDs. Check L01's October 1, 2026, 09:00 appointment and supplied location against the input; do not infer that the generated letter has been sent.

13.3 Read the Key Lines

```

49     status="hold" if issues else "generated"
50     report.append(dict(appointment_id=row["appointment_id"],status=status,
51                       reason="; ".join(issues)))
52     if issues:
53         continue
54     fields={k:escape(v) for k,v in row.items()}
55     sections.append("<section><p>SYNTHETIC TEACHING EXAMPLE</p><h1>Appointment
    ↳ details</h1>" +
56         "<p>"+fields["display_name"]+"</p><p>"+fields["date"]+" at "+fields["
    ↳ time"]+
57         "</p><p>Location: "+fields["location"]+"</p><p>Questions or changes: "+
58         fields["contact"]+"</p><p>Please check these details and contact the
    ↳ office "+
59         "if anything is incorrect.</p><p>Reference: "+fields["appointment_id"]+
    ↳ "</p></section>")

```

Excerpt from appointment_letters.py, source lines 49–59.

A list comprehension can collect missing fields. The `issues` list initially contains the names of blank required fields. Date-time parsing can add another reason. A nonempty list controls both the hold status and the decision not to generate a section.

Escape data, not the template. The fixed HTML template contains markup. The supplied fields are passed through `escape` before insertion so that names or locations containing angle brackets are not interpreted as instructions.

One report row for each request. The build-report row is appended before `continue` skips a held letter. Reversing that order could make held requests vanish from the report.

13.4 Try One Change

Supply the invented contact 555-0101 for L04. Four letters should now be generated and no request held. Check the added letter's details. The change must not add message-sending behavior.

13.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

letter_requests.csv

Save the following complete contents as `letter_requests.csv`.

```
appointment_id,display_name,date,time,location,contact
L01,Sample Person One,2026-10-01,09:00,Teaching Clinic A,555-0101
L02,Sample Person Two,2026-10-01,10:30,Teaching Clinic B,555-0102
L03,Sample Person Three,2026-10-02,14:00,Teaching Clinic A,555-0101
L04,Sample Person Four,2026-10-02,15:00,Teaching Clinic A,
```

appointment_letters.py

Complete program. Save this listing as `appointment_letters.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'letter_requests.csv': ['appointment_id', 'display_name', 'date', '
    ↪ time', 'location', 'contact']}]
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20
21 def require_ids(rows, field):
22     keys = [r[field] for r in rows]
23     if any(not k.strip() or k != k.strip() for k in keys):
24         raise ValueError("IDs must be nonblank and unpadded: " + field)
25     if len(set(keys)) != len(keys):
26         raise ValueError("Duplicate ID: " + field)
```

```

27
28 def save_rows(name, fields, rows):
29     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
30         writer = csv.DictWriter(stream, fieldnames=fields)
31         writer.writeheader()
32         writer.writerows(rows)
33
34 from datetime import datetime
35 from html import escape
36
37 rows=read_rows("letter_requests.csv")
38 require_ids(rows, "appointment_id")
39 sections=[]; report=[]
40 for row in rows:
41     issues=[name for name in ["display_name","date","time","location","contact"
42 ↪ ] if not row[name].strip()]
43     try:
44         supplied = row["date"] + " " + row["time"]
45         day=datetime.strptime(supplied,"%Y-%m-%d %H:%M")
46         if day.strftime("%Y-%m-%d %H:%M") != supplied:
47             raise ValueError("Use YYYY-MM-DD and HH:MM")
48     except ValueError:
49         issues.append("invalid date or time")
50     status="hold" if issues else "generated"
51     report.append(dict(appointment_id=row["appointment_id"],status=status,
52 ↪ reason="; ".join(issues)))
53     if issues:
54         continue
55     fields={k:escape(v) for k,v in row.items()}
56     sections.append("<section><p>SYNTHETIC TEACHING EXAMPLE</p><h1>Appointment
57 ↪ details</h1>"+
58     "<p>"+fields["display_name"]+"</p><p>"+fields["date"]+" at "+fields["
59 ↪ time"]+
60     "</p><p>Location: "+fields["location"]+"</p><p>Questions or changes: "+
61     fields["contact"]+"</p><p>Please check these details and contact the
62 ↪ office "+
63     "if anything is incorrect.</p><p>Reference: "+fields["appointment_id"]+
64     "</p></section>")
65 save_rows("letter_build_report.csv",["appointment_id", "status", "reason"],
66 ↪ report)
67 page='<!doctype html><html lang="en"><meta charset="utf-8"><meta name="viewport
68 ↪ " content="width=device-width,initial-scale=1"><title>Teaching
69 ↪ appointment pack</title>'
70 page+='<style>body{font:20px system-ui;max-width:720px;margin:auto}section{
71 ↪ padding:30px;border-bottom:1px solid #aaa;overflow-wrap:anywhere}h1{
72 ↪ line-height:1.2}@page{size:letter;margin:18mm}@media print{body{font-
73 ↪ size:14pt;max-width:none}section{padding:12mm;break-after:page;break-

```

```
    ↪ inside:avoid}section:last-child{break-after:auto}}</style>'
63 with (ROOT/"appointment_letters.html").open("x",encoding="utf-8") as stream:
64     stream.write(page+("\n".join(sections) if sections else "<p>No letters
    ↪ generated.</p>")+ "</html>")
65 print("Letters generated:",len(sections))
66 print("Held for missing or invalid details:",sum(r["status"]=="hold" for r in
    ↪ report))
67 print("No messages sent.")
```


Rule-and-Boundary Lab

The rule-and-boundary lab implements a deliberately fictional classroom score. Its purpose is to make boundary conditions and component calculations visible. It does not represent a validated clinical score and should not be renamed to resemble one.

14.1 Run the Reference Version

Working files: complete listings in Chapter 14 below.

Inputs: `score_cases.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 teaching_score.py
```

Rule to preserve. Duration below 5 gives 0 points, duration from 5 up to but not including 10 gives 1, and duration at least 10 gives 2. Task counts through 2 give 0, through 5 give 1, and higher counts give 2. Supplied support=no adds 1. Invalid input prevents calculation.

14.2 Check the Result

Outputs: `teaching_score_report.csv`.

Check the meaning

C01 totals 0, C02 totals 1, C03 totals 5, and C04 totals 2. C05 has missing duration; C06 has negative duration. Notice the exact boundaries: C02's duration is 5 and C03's is 10. A program using the wrong comparison could look correct on most other inputs.

14.3 Read the Key Lines

```
56     d = t = support = total = ""
57     if not issues:
58         d = 0 if duration < 5 else 1 if duration < 10 else 2
59         t = 0 if tasks <= 2 else 1 if tasks <= 5 else 2
60         support = int(row["support_present"] == "no")
61         total = d+t+support
62     report.append(dict(case_id=row["case_id"],duration_points=d,
63                       task_points=t,support_points=support,total=total,
64                       status="review" if issues else "calculated",
65                       reason="; ".join(issues)))
```

Excerpt from teaching_score.py, source lines 56–65.

Validate before assigning components. The component variables start as empty strings. They receive numeric values only when `issues` is empty. This prevents a missing duration from accidentally behaving like duration 0.

Nested conditional expressions encode intervals. The duration expression tests the lower interval first, then the middle interval, then the remaining upper interval. Read it as three disjoint cases rather than as a compact formula to admire.

Components make a total inspectable. Saving the separate points lets you locate an incorrect comparison without reverse-engineering the sum. A matching total alone can hide two compensating errors.

14.4 Try One Change

Change C02's duration from 5 to 4.9. Its duration component and total should fall from 1 to 0; every other case should remain unchanged. This is a test of the stated interval boundary, not a new scoring recommendation.

14.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

score_cases.csv

Save the following complete contents as `score_cases.csv`.

```
case_id,duration,task_count,support_present
C01,4,2,yes
C02,5,2,yes
C03,10,6,no
C04,9.9,5,yes
C05,,3,yes
C06,-1,3,yes
```

teaching_score.py

Complete program. Save this listing as `teaching_score.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'score_cases.csv': ['case_id', 'duration', 'task_count', '
    ↪ support_present']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20
21 def require_ids(rows, field):
22     keys = [r[field] for r in rows]
23     if any(not k.strip() or k != k.strip() for k in keys):
24         raise ValueError("IDs must be nonblank and unpadded: " + field)
```

```

25     if len(set(keys)) != len(keys):
26         raise ValueError("Duplicate ID: " + field)
27
28 def save_rows(name, fields, rows):
29     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
30         writer = csv.DictWriter(stream, fieldnames=fields)
31         writer.writeheader()
32         writer.writerows(rows)
33
34 import math
35
36 report = []
37 rows = read_rows("score_cases.csv")
38 require_ids(rows, "case_id")
39 for row in rows:
40     issues = []
41     duration = tasks = None
42     try:
43         duration = float(row["duration"])
44         if not math.isfinite(duration) or duration < 0:
45             issues.append("invalid duration")
46     except ValueError:
47         issues.append("missing or nonnumeric duration")
48     try:
49         tasks = int(row["task_count"])
50         if tasks < 0:
51             issues.append("negative task count")
52     except ValueError:
53         issues.append("invalid task count")
54     if row["support_present"] not in {"yes", "no"}:
55         issues.append("invalid support value")
56     d = t = support = total = ""
57     if not issues:
58         d = 0 if duration < 5 else 1 if duration < 10 else 2
59         t = 0 if tasks <= 2 else 1 if tasks <= 5 else 2
60         support = int(row["support_present"] == "no")
61         total = d+t+support
62     report.append(dict(case_id=row["case_id"], duration_points=d,
63                       task_points=t, support_points=support, total=total,
64                       status="review" if issues else "calculated",
65                       reason="; ".join(issues)))
66 save_rows("teaching_score_report.csv", ["case_id", "duration_points", "
    ↳ task_points", "support_points", "total", "status", "reason"], report)
67 for row in report:
68     print(row["case_id"], row["total"] if row["status"]=="calculated" else row[
    ↳ "reason"])
69 print("Fictional classroom rule; no clinical interpretation.")

```

Predictive-Value Explorer

The predictive-value explorer is a fictional expected-count calculation, not an evaluation of a real diagnostic test. Python writes a scenario CSV and a browser page. The browser then lets a reader try other prevalence values under the same supplied model.

15.1 Run the Reference Version

Working files: complete listings in Chapter 15 below.

Inputs: `test_scenarios.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 predictive_value_explorer.py
```

Rule to preserve. The baseline assumes 10,000 people, sensitivity 0.90, and specificity 0.95. Prevalence must be between 0 and 100 percent. Expected cells may be fractional. The positive predictive value is calculated from expected true positives divided by all expected positives, and is left undefined when that denominator is zero.

15.2 Check the Result

Outputs: `predictive_value.html`, `test_scenarios_report.csv`.

Check the meaning

At 1% prevalence, expected true positives are 90 and expected false positives 495, giving 15.4%. The remaining cells are 10 and 9,405, and all four sum to 10,000. At 10% and 30% prevalence the saved percentages are 66.7% and 88.5%. Changing the browser control does not rewrite the separately saved scenario CSV.

Inspect both the saved CSV and the interactive page; a correct CSV does not test the browser controls.

15.3 Read the Key Lines

```
31 def expected(prevalence):  
32     condition = MODEL["n"] * prevalence  
33     tp = condition * MODEL["sensitivity"]  
34     fn = condition - tp  
35     fp = (MODEL["n"] - condition) * (1 - MODEL["specificity"])  
36     tn = MODEL["n"] - condition - fp  
37     return tp, fp, fn, tn
```

Excerpt from predictive_value_explorer.py, source lines 31–37.

One model, two execution environments. The Python `expected` function computes the saved scenarios. `json.dumps(MODEL)` embeds the same assumptions in the page’s JavaScript. The two calculations still need comparison; sharing assumptions does not itself prove identical behavior.

Expected counts precede rounding. The code calculates the four cells from unrounded quantities and forms the ratio before rounding its displayed percentage. Rounding cells too early can change the reported proportion.

Undefined is not zero. The branch on the positive total protects the ratio when no positives are expected. A blank CSV value and the page’s “undefined” label express absence of a defined ratio, not a 0% estimate.

15.4 Try One Change

Add scenario rows with prevalence 0 and 100. Under the unchanged baseline assumptions, the corresponding saved positive predictive values are 0% and 100%. Check all four expected cells and their total as well. Compare the same values in the browser. Changing its control does not rewrite the saved CSV.

15.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

test_scenarios.csv

Save the following complete contents as `test_scenarios.csv`.

```
prevalence_percent
1
10
30
```

predictive_value_explorer.py

Complete program. Save this listing as `predictive_value_explorer.py`.

```
1  """Build a fictional expected-count explorer; no real test is evaluated."""
2  import csv
3  import html
4  import json
5  import math
6  from pathlib import Path
7
8  ROOT = Path(__file__).resolve().parent
9  N = 10000
10 sensitivity = 0.90
11 specificity = 0.95
12 # This one model supplies the CSV calculation, page labels, and JavaScript.
13 MODEL = dict(n=N, sensitivity=sensitivity, specificity=specificity)
14 if (isinstance(N, bool) or not isinstance(N, int) or N <= 0 or
15     any(not math.isfinite(x) or not 0 <= x <= 1
16         for x in (sensitivity, specificity))):
17     raise ValueError("Use a positive population and probabilities from 0 to 1")
18
19
20 def read_rows(name):
21     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
22         reader = csv.DictReader(stream, strict=True)
23         if reader.fieldnames != ["prevalence_percent"]:
24             raise ValueError("Wrong header in " + name)
25         rows = list(reader)
26         if any(None in row or None in row.values() for row in rows):
27             raise ValueError("Malformed CSV row in " + name)
28     return rows
```

```

29
30
31 def expected(prevalence):
32     condition = MODEL["n"] * prevalence
33     tp = condition * MODEL["sensitivity"]
34     fn = condition - tp
35     fp = (MODEL["n"] - condition) * (1 - MODEL["specificity"])
36     tn = MODEL["n"] - condition - fp
37     return tp, fp, fn, tn
38
39
40 report = []
41 for row in read_rows("test_scenarios.csv"):
42     prevalence = float(row["prevalence_percent"]) / 100
43     if not math.isfinite(prevalence) or not 0 <= prevalence <= 1:
44         raise ValueError("Prevalence must be between 0 and 100 percent")
45     tp, fp, fn, tn = expected(prevalence)
46     report.append(dict(prevalence_percent=prevalence * 100,
47                       tp=round(tp, 6), fp=round(fp, 6), fn=round(fn, 6), tn=round(tn, 6),
48                       ppv_percent=round(100 * tp / (tp + fp), 1) if tp + fp else ""))
49
50 page = '''<!doctype html><html lang="en"><meta charset="utf-8">
51 <meta name="viewport" content="width=device-width,initial-scale=1">
52 <title>Predictive-value classroom explorer</title>
53 <style>
54 body{font:20px system-ui;max-width:800px;margin:32px auto;padding:0 20px}
55 h1{font-size:30px;line-height:1.2}p{line-height:1.5}
56 input,button{font:inherit;padding:6px}input{width:110px}
57 button{margin:8px 0}:a:focus,input:focus,button:focus{outline:3px solid #175078}
58 .scroll{overflow-x:auto}table{border-collapse:collapse;width:100%}
59 td,th{padding:12px;border:1px solid #aaa;text-align:right}
60 th:first-child{text-align:left}thead,tfoot{background:#f1f5f8}
61 .small{font-size:16px}.ppv{font-size:25px;font-weight:600}
62 </style><main><h1>Predictive-value classroom explorer</h1>
63 <p id="assumptions">Fictional expected counts in __N__ people.<br>
64 Sensitivity __SENS__%; specificity __SPEC__%. These are assumptions,
65 not evidence about a real test.</p>
66 <label for="p">Prevalence (%): </label>
67 <input id="p" type="number" min="0" max="100" step="any" value="1">
68 <button id="go" type="button">Calculate</button>
69 <div id="result" aria-live="polite"></div>
70 <p class="small">Expected counts need not be whole numbers. Changing this
71 page does not change the separately saved scenario CSV.</p></main>
72 <script>
73 const model = __MODEL__;
74 function update(){
75     const raw=document.getElementById("p").value.trim();

```

```

76 const p=Number(raw)/100;
77 const result=document.getElementById("result");
78 if(raw==""||!Number.isFinite(p)||p<0||p>1){
79     result.textContent="Enter 0 to 100.";return;
80 }
81 const condition=model.n*p;
82 const tp=condition*model.sensitivity, fn=condition-tp;
83 const fp=(model.n-condition)*(1-model.specificity);
84 const tn=model.n-condition-fp;
85 const ppv=tp+fp ? (100*tp/(tp+fp)).toFixed(1)+"% : "undefined";
86 const number=x=>x.toFixed(1);
87 const cell=(id,x)=><td id="+id+">'+number(x)+'</td>';
88 result.innerHTML='<p class="ppv">Positive predictive value: <span id="ppv">'+
89 ppv+'</span></p><div class="scroll"><table><caption>Expected counts</caption>
    ↪ +
90 '<thead><tr><th scope="col">Test result</th><th scope="col">Condition present
    ↪ </th>'+
91 '<th scope="col">Condition absent</th><th scope="col">Total</th></tr></thead>'
    ↪ +
92 '<tbody><tr><th scope="row">Positive</th>'+cell('tp',tp)+cell('fp',fp)+
93 cell('positive-total',tp+fp)+'</tr><tr><th scope="row">Negative</th>'+
94 cell('fn',fn)+cell('tn',tn)+cell('negative-total',fn+tn)+'</tr></tbody>'+
95 '<tfoot><tr><th scope="row">Total</th>'+cell('condition-total',tp+fn)+
96 cell('absent-total',fp+tn)+cell('population-total',model.n)+
97 '</tr></tfoot></table></div>';
98 }
99 document.getElementById("go").addEventListener("click",update);
100 document.getElementById("p").addEventListener("input",update);update();
101 </script></html>'''
102 page = (page.replace("__N__", f'{MODEL["n"]:.}')
103         .replace("__SENS__", f'{100 * MODEL["sensitivity"]:g}')
104         .replace("__SPEC__", f'{100 * MODEL["specificity"]:g}')
105         .replace("__MODEL__", json.dumps(MODEL, allow_nan=False)))
106 outputs = ["test_scenarios_report.csv", "predictive_value.html"]
107 if any((ROOT / name).exists() for name in outputs):
108     raise FileExistsError("Use a fresh copy of the chapter folder")
109 with (ROOT / outputs[0]).open("x", newline="", encoding="utf-8") as stream:
110     writer = csv.DictWriter(stream, fieldnames=["prevalence_percent", "tp",
111         "fp", "fn", "tn", "ppv_percent"])
112     writer.writeheader()
113     writer.writerows(report)
114 with (ROOT / outputs[1]).open("x", encoding="utf-8") as stream:
115     stream.write(page)
116 for row in report:
117     value = (str(row["ppv_percent"]) + "%") if row["ppv_percent"] != "" else "
    ↪ undefined"
118     print(f'Prevalence {row["prevalence_percent"]:g}%: PPV {value}')

```


Literature-Review Organizer

The literature-review organizer groups invented retrieved records by a supplied study ID. Multiple records can concern one study. The program preserves those record identities and highlights disagreements instead of silently keeping the first record and discarding the others.

16.1 Run the Reference Version

Working files: complete listings in Chapter 16 below.

Inputs: `citations.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 review_organizer.py
```

Rule to preserve. Record IDs are distinct and study IDs are present. Decisions must be include, exclude, or pending; exclusions require reasons. A disagreement in decision, title, or year within a study group produces review conflict, together with the differing values and their source record IDs.

16.2 Check the Result

Outputs: `study_screening.csv`.

Check the meaning

Eight retrieved records form six study groups. ST01 contains R01 and R02; ST03 contains R04 and R06. The baseline has three included, two excluded, one pending, and no conflicting groups. The two additional retrieved records remain listed within their study groups rather than being erased.

16.3 Read the Key Lines

```
49     conflict_fields = [field for field in ["decision", "title", "year"]
50                        if len({r[field] for r in group}) > 1]
51     conflict = bool(conflict_fields)
52     differences = "; ".join(field + ": " + " + ", ".join(
53         r["record_id"] + "=" + r[field] for r in group)
54         for field in conflict_fields)
55     decision = "review conflict" if conflict else group[0]["decision"]
```

Excerpt from review_organizer.py, source lines 49–55.

Group first, inspect differences second. `defaultdict(list)` retains all records for each study. Sets of supplied values then reveal whether a field agrees within that group. This is not the same as deduplicating whole rows.

A conflict has named fields. `conflict_fields` records which of decision, title, and year differ. The program then constructs strings showing each record ID and supplied value, making the flag inspectable.

Sorting can stabilize a report. Sorting study IDs and sets of reasons gives predictable output order. It does not decide which conflicting value is correct.

16.4 Try One Change

Change only R02's year from 2024 to 2023. ST01 should become review conflict with year named as the conflicting field; both R01 and R02 must remain. Expect two included groups, two excluded, one pending, and one conflict. Do not resolve the conflict by deleting R02.

16.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

citations.csv

Save the following complete contents as `citations.csv`.

```
record_id,study_id,title,year,decision,reason
R01,ST01,Reminder demo A,2024,include,
R02,ST01,Reminder demo A,2024,include,
R03,ST02,Reminder demo B,2025,exclude,wrong setting
R04,ST03,Reminder demo C,2025,include,
R05,ST04,Reminder demo D,2023,pending,full text needed
R06,ST03,Reminder demo C,2025,include,
R07,ST05,Reminder demo E,2024,exclude,wrong outcome
R08,ST06,Reminder demo F,2025,include,
```

review_organizer.py

Complete program. Save this listing as `review_organizer.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'citations.csv': ['record_id', 'study_id', 'title', 'year', '
    ↪ decision', 'reason']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20
21 def require_ids(rows, field):
22     keys = [r[field] for r in rows]
```

```

23     if any(not k.strip() or k != k.strip() for k in keys):
24         raise ValueError("IDs must be nonblank and unpadded: " + field)
25     if len(set(keys)) != len(keys):
26         raise ValueError("Duplicate ID: " + field)
27
28 def save_rows(name, fields, rows):
29     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
30         writer = csv.DictWriter(stream, fieldnames=fields)
31         writer.writeheader()
32         writer.writerows(rows)
33
34 from collections import defaultdict, Counter
35
36 rows = read_rows("citations.csv")
37 require_ids(rows, "record_id")
38 groups = defaultdict(list)
39 for row in rows:
40     if not row["study_id"].strip() or row["study_id"] != row["study_id"].strip()
41     ↪ () or row["decision"] not in {"include", "exclude", "pending"}:
42         raise ValueError("Study ID and approved decision required")
43     if row["decision"]=="exclude" and not row["reason"].strip():
44         raise ValueError("Exclusions require a reason")
45     groups[row["study_id"]].append(row)
46 report = []
47 for study, group in sorted(groups.items()):
48     decisions = {r["decision"] for r in group}
49     titles = {(r["title"], r["year"]) for r in group}
50     conflict_fields = [field for field in ["decision", "title", "year"]
51                       if len({r[field] for r in group}) > 1]
52     conflict = bool(conflict_fields)
53     differences = "; ".join(field + ": " + r[field] for r in group)
54     for field in conflict_fields:
55         decision = "review conflict" if conflict else group[0]["decision"]
56     report.append(dict(study_id=study,
57                       record_ids="; ".join(r["record_id"] for r in group),
58                       retrieved_records=len(group), decision=decision,
59                       conflict_fields="; ".join(conflict_fields),
60                       supplied_differences=differences,
61                       source_locator="; ".join("citations.csv:" + r["record_id"] for r in
62                       ↪ group),
63                       reason="; ".join(sorted({r["reason"] for r in group if r["reason"]})) +
64                       ↪ (";" if differences and any(r["reason"] for r in group) else "") +
65                       ↪ differences))
66 save_rows("study_screening.csv", ["study_id", "record_ids", "retrieved_records",
67 ↪ "decision", "conflict_fields", "supplied_differences", "source_locator
68 ↪ ", "reason"], report)

```

```
64 counts = Counter(r["decision"] for r in report)
65 print("Retrieved records:",len(rows))
66 print("Study groups:",len(report))
67 print("Additional records within groups:",len(rows)-len(report))
68 for decision in ["include","exclude","pending","review conflict"]:
69     print(decision+":",counts[decision])
```


Reproducible Research Table

The research table aligns baseline and follow-up measurements by person reference and computes a change only for complete pairs. It saves a person-level table, a small summary, and a run record identifying the source and program.

The calculation is descriptive and uses invented teaching units. A mean change in these records is not evidence of a treatment effect or a substitute for a research design.

17.1 Run the Reference Version

Working files: complete listings in Chapter 17 below.

Inputs: `research_measurements.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 research_table.py
```

Rule to preserve. Each person-timepoint combination must be unique; allowed timepoints are baseline and followup. A supplied number must be finite. Change means follow-up minus baseline. Incomplete pairs remain in the person table but do not contribute to mean or median change.

17.2 Check the Result

Outputs: `derived_changes.csv`, `research_summary.csv`, `run_record.json`.

Check the meaning

Six people yield five complete pairs. Their changes are -2, -3, -3, -4, and -2: mean -2.8 and median -3.0. S04 remains with baseline 11, blank follow-up, and reason incomplete pair. Source row references identify the two contributing positions for each person.

17.3 Read the Key Lines

```

45 for key,values in sorted(people.items()):
46     baseline=values.get("baseline")
47     followup=values.get("followup")
48     complete=baseline is not None and followup is not None
49     derived.append(dict(person_ref=key,
50         baseline="" if baseline is None else baseline,
51         followup="" if followup is None else followup,
52         baseline_row=source_rows.get((key,"baseline"),""),
53         followup_row=source_rows.get((key,"followup"),""),
54         unit="teaching units",paired=int(complete),
55         change=followup-baseline if complete else "",
56         reason="" if complete else "incomplete pair"))

```

Excerpt from research_table.py, source lines 45–56.

Use a compound identity. A nested dictionary stores values by person and timepoint. The duplicate check concerns that pair, not person alone; two records for the same person are expected when the timepoints differ.

Use None for an absent value. The program distinguishes None from a numerical 0. The test `baseline is not None and followup is not None` accepts genuine zero measurements while excluding missing ones.

A derived record carries its sources. The table stores baseline and follow-up values, source row numbers, the paired flag, and the change. A summary can therefore be checked without guessing which rows were aligned.

17.4 Try One Change

Supply follow-up value 10 for S04's existing blank follow-up record. Expect six complete pairs, with S04's change -1. The mean and median both become -2.5. The original input and its run record must remain available as a separate version.

17.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

research_measurements.csv

Save the following complete contents as `research_measurements.csv`.

```
person_ref,timepoint,value
S01,baseline,10
S01,followup,8
S02,baseline,12
S02,followup,9
S03,baseline,15
S03,followup,12
S04,baseline,11
S04,followup,
S05,baseline,14
S05,followup,10
S06,baseline,13
S06,followup,11
```

research_table.py

Complete program. Save this listing as `research_table.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'research_measurements.csv': ['person_ref', 'timepoint', 'value']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
```

```

20
21
22 def save_rows(name, fields, rows):
23     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
24         writer = csv.DictWriter(stream, fieldnames=fields)
25         writer.writeheader()
26         writer.writerows(rows)
27
28 from collections import defaultdict
29 import hashlib,json,math,platform,statistics
30
31 rows=read_rows("research_measurements.csv")
32 people=defaultdict(dict)
33 source_rows={}
34 for number, row in enumerate(rows, 1):
35     key=row["person_ref"]
36     visit=row["timepoint"]
37     if not key.strip() or key != key.strip() or visit not in {"baseline",
↪ followup"} or visit in people[key]:
38         raise ValueError("Distinct person-timepoint records required")
39     value=float(row["value"]) if row["value"].strip() else None
40     if value is not None and not math.isfinite(value):
41         raise ValueError("Values must be finite")
42     people[key][visit]=value
43     source_rows[(key,visit)]=number
44 derived=[]
45 for key,values in sorted(people.items()):
46     baseline=values.get("baseline")
47     followup=values.get("followup")
48     complete=baseline is not None and followup is not None
49     derived.append(dict(person_ref=key,
50         baseline="" if baseline is None else baseline,
51         followup="" if followup is None else followup,
52         baseline_row=source_rows.get((key,"baseline"),""),
53         followup_row=source_rows.get((key,"followup"),""),
54         unit="teaching units",paired=int(complete),
55         change=followup-baseline if complete else "",
56         reason="" if complete else "incomplete pair"))
57 changes=[r["change"] for r in derived if r["paired"]]
58 summary=[dict(measure="people",value=len(people)),
59     dict(measure="complete_pairs",value=len(changes)),
60     dict(measure="mean_change",value=statistics.mean(changes) if changes
↪ else ""),
61     dict(measure="median_change",value=statistics.median(changes) if
↪ changes else "")]
62 save_rows("derived_changes.csv",["person_ref", "baseline", "followup", "
↪ baseline_row", "followup_row", "unit", "paired", "change", "reason"],

```

```
↪ derived)
63 save_rows("research_summary.csv",["measure","value"],summary)
64 manifest={"input_sha256":hashlib.sha256((ROOT/"research_measurements.csv").
    ↪ read_bytes()).hexdigest(),
65 "program_sha256":hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
66 "python":platform.python_version(),"rule":"followup minus baseline; complete
    ↪ pairs only",
67 "output_files":["derived_changes.csv","research_summary.csv"]}
68 with (ROOT/"run_record.json").open("x",encoding="utf-8") as stream:
69     json.dump(manifest,stream,indent=2)
70 for row in summary:
71     print(f'{row["measure"]}: {row["value"]}')

```


Reminder Simulation

The reminder simulation repeatedly generates two invented groups under supplied attendance probabilities. The saved trial table is the primary simulation record. Its summary, frequency table, and histogram are derived from those same trials rather than from separately generated samples.

The probabilities are assumptions. This program illustrates sampling variation under a model; it does not estimate the effect of a real reminder intervention.

18.1 Run the Reference Version

Working files: complete listings in Chapter 18 below.

Inputs: `simulation_plan.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 reminder_simulation.py
```

Rule to preserve. The input contains exactly one plan: positive group size and repetition count, probabilities between 0 and 1, and a seed. Each repetition independently generates the two groups using the declared model. Difference is reminder minus usual in percentage points. The two reported central bounds use the order-statistic indices written in the program.

18.2 Check the Result

Outputs: `simulation_frequency.csv`, `simulation_histogram.svg`, `simulation_summary.csv`, `simulation_trials.csv`.

Check the meaning

With 80 per group, probabilities 0.70 and 0.80, 1,000 repetitions, and seed 104, the reference run gives mean difference 10.08 points, central bounds -3.75 and 22.5, and 80 trials in which the reminder count is not higher. The frequency counts must sum to 1,000. The central range is a simulation summary, not an automatically justified confidence interval for a real effect.

18.3 Read the Key Lines

```
38 rng=random.Random(int(plan["seed"]))
39 trials=[]
40 for trial in range(1,repetitions+1):
41     usual=sum(rng.random()<p0 for _ in range(n))
42     reminder=sum(rng.random()<p1 for _ in range(n))
43     trials.append(dict(trial=trial,usual=usual,reminder=reminder,
44         difference_points=100*(reminder-usual)/n))
45 values=sorted(r["difference_points"] for r in trials)
46 low=values[int(.025*(len(values)-1))]
47 high=values[int(.975*(len(values)-1))]
```

Excerpt from reminder_simulation.py, source lines 38–47.

A local random generator has explicit state. `random.Random(seed)` creates the generator used by this simulation. Reusing the same implementation, seed, and sequence of draws supports repeatability. Adding an extra random draw can change later simulated results.

A comparison simulates one binary outcome. `rng.random() < p0` is one simulated outcome; summing such Boolean comparisons counts successes. The outer loop repeats the two-group experiment.

Quantile conventions are part of a specification. The code sorts differences and selects the entries at the displayed integer indices. Another percentile convention could give a different bound. The companion explains the implemented rule rather than claiming all percentile methods are interchangeable.

18.4 Try One Change

Set both supplied probabilities to 0 and keep the other plan fields unchanged. Every trial must have zero attendance in both groups, all differences and central bounds must be zero, and reminder-not-higher must count all 1,000 trials. This is a deterministic boundary test of a random program.

18.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

simulation_plan.csv

Save the following complete contents as `simulation_plan.csv`.

```
per_group,usual_probability,reminder_probability,repetitions,seed
80,0.70,0.80,1000,104
```

reminder_simulation.py

Complete program. Save this listing as `reminder_simulation.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'simulation_plan.csv': ['per_group', 'usual_probability', '
    ↪ reminder_probability', 'repetitions', 'seed']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20
21
22 def save_rows(name, fields, rows):
23     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
24         writer = csv.DictWriter(stream, fieldnames=fields)
25         writer.writeheader()
26         writer.writerows(rows)
27
28 import random, statistics
29
```

```

30 plans=read_rows("simulation_plan.csv")
31 if len(plans)!=1:
32     raise ValueError("Exactly one simulation plan row is required")
33 plan=plans[0]
34 n=int(plan["per_group"]); repetitions=int(plan["repetitions"])
35 p0=float(plan["usual_probability"]); p1=float(plan["reminder_probability"])
36 if n<1 or repetitions<1 or not (0<=p0<=1 and 0<=p1<=1):
37     raise ValueError("Invalid simulation plan")
38 rng=random.Random(int(plan["seed"]))
39 trials=[]
40 for trial in range(1,repetitions+1):
41     usual=sum(rng.random()<p0 for _ in range(n))
42     reminder=sum(rng.random()<p1 for _ in range(n))
43     trials.append(dict(trial=trial,usual=usual,reminder=reminder,
44         difference_points=100*(reminder-usual)/n))
45 values=sorted(r["difference_points"] for r in trials)
46 low=values[int(.025*(len(values)-1))]
47 high=values[int(.975*(len(values)-1))]
48 summary=[dict(measure="mean_difference_points",value=round(statistics.mean(
49     ↪ values),2)),
50     dict(measure="central_95_low",value=low),
51     dict(measure="central_95_high",value=high),
52     dict(measure="reminder_not_higher",value=sum(x<=0 for x in values))]
53 save_rows("simulation_trials.csv",["trial", "usual", "reminder", "
54     ↪ difference_points"],trials)
55 save_rows("simulation_summary.csv",["measure","value"],summary)
56 for row in summary: print(f'{row["measure"]}: {row["value"]}')
57 print("Simulated assumptions; not an estimated treatment effect.")
58
59 from collections import Counter
60 import math
61 bins = Counter(5*math.floor(r["difference_points"]/5) for r in trials)
62 frequency = [dict(lower=left, upper=left+5, count=bins[left])
63     for left in range(min(bins), max(bins)+1, 5)]
64 save_rows("simulation_frequency.csv", ["lower", "upper", "count"], frequency)
65
66 # Histogram from the saved frequency table; no second simulation.
67 left = min(f["lower"] for f in frequency)
68 right = max(f["upper"] for f in frequency)
69 peak = max(f["count"] for f in frequency)
70 assumed = 100*(p1-p0)
71 xmin, xmax = min(left, assumed), max(right, assumed)
72 def x(value):
73     return 60+610*(value-xmin)/max(xmax-xmin, 1)
74 def y(count):
75     return 290-220*count/max(peak, 1)
76 svg = ['<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 720 370">',

```

```

75     '<rect width="720" height="370" fill="white"/>',
76     '<g font-family="sans-serif" font-size="13">',
77     '<text x="60" y="24">Simulated attendance differences</text>',
78     '<path d="M60 60 V290 H680" fill="none" stroke="black"/>']
79 for f in frequency:
80     svg.append(f'<rect x="{x(f["lower"])}" y="{y(f["count"])}" '
81               f'width="{x(f["upper"])-x(f["lower"])}" '
82               f'height="{290-y(f["count"])}" fill="none" stroke="black"/>')
83     svg.append(f'<text x="{x(f["lower"])}" y="310">{f["lower"]}</text>')
84 for count in [0, peak//2, peak]:
85     svg.append(f'<text x="12" y="{y(count)+4}">{count}</text>')
86 svg.append(f'<path d="M{x(assumed)} 45 V290" stroke="black" '
87           f'stroke-dasharray="5,4"/>')
88 svg.append(f'<text x="60" y="345">Dashed line: assumed {assumed:g} percentage
89           ↪ points</text>')
89 svg.append(f'<text x="60" y="363">Reminder minus usual; vertical axis: trial
90           ↪ count</text></g></svg>')
91 with (ROOT/"simulation_histogram.svg").open("x",encoding="utf-8") as stream:
92     stream.write("\n".join(svg))

```


District Outreach

The district report contrasts crude percentages with a calculation using the same supplied age-band weights for every district. It makes clear that different population compositions can change crude comparisons even when a common-weight calculation is similar.

The two equal weights are classroom choices. The program does not claim that they are the appropriate standard population for a real public-health report.

19.1 Run the Reference Version

Working files: complete listings in Chapter 19 below.

Inputs: `district_outreach.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 outreach_rates.py
```

Rule to preserve. Each district must have exactly one under65 row and one 65plus row. Each eligible count is positive and reached lies between 0 and eligible. The crude percentage pools the district's counts. The standardized percentage is the equally weighted average of its two age-band percentages.

19.2 Check the Result

Outputs: `district_rates.csv`.

Check the meaning

North has crude 54.0% and standardized 70.0%; South has 78.0% and 70.0%; East has 72.0% and 70.0%; West has 56.0% and 60.0%. North's two supplied band rates are 50% and 90%, whose equal-weight average is 70%. The output keeps the weights visible.

19.3 Read the Key Lines

```
45     reached=sum(x[0] for x in bands.values())
46     eligible=sum(x[1] for x in bands.values())
47     rates={band:100*n/d for band,(n,d) in bands.items()}
48     report.append(dict(district=district,reached=reached,eligible=eligible,
49         crude_percent=round(100*reached/eligible,1),
50         standardized_percent=round(.5*rates["under65"]+.5*rates["65plus"],1),
51         weight_under65=.5,weight_65plus=.5))
```

Excerpt from outreach_rates.py, source lines 45–51.

Two formulas answer different questions. The crude rate divides summed reached counts by summed eligible counts. The common-weight rate combines the separately calculated band rates. Replacing one formula with the other changes the question.

Keep assumptions beside results. The output includes both weights as explicit columns. A reader can therefore reconstruct the calculation instead of treating “standardized” as an unexplained label.

Rounding belongs at presentation. The program forms rates from unrounded counts and rounds the final reported percentages. The original counts remain available for a more precise recalculation.

19.4 Try One Change

Double both reached and eligible counts in both North rows. Expect North totals of 1,080 reached among 2,000 eligible, but unchanged crude 54.0% and standardized 70.0%. Scaling the counts should not alter their ratios. Do not apply the multiplier to only a numerator.

19.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

district_outreach.csv

Save the following complete contents as `district_outreach.csv`.

```
district,age_band,reached,eligible
North,under65,450,900
North,65plus,90,100
South,under65,60,100
South,65plus,720,900
East,under65,240,400
East,65plus,480,600
West,under65,350,700
West,65plus,210,300
```

outreach_rates.py

Complete program. Save this listing as `outreach_rates.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'district_outreach.csv': ['district', 'age_band', 'reached', '
    ↪ eligible']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20
21
22 def save_rows(name, fields, rows):
```

```

23     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
24         writer = csv.DictWriter(stream, fieldnames=fields)
25         writer.writeheader()
26         writer.writerows(rows)
27
28 from collections import defaultdict
29
30 groups=defaultdict(dict)
31 for row in read_rows("district_outreach.csv"):
32     band=row["age_band"]; district=row["district"]
33     if not district.strip() or district != district.strip():
34         raise ValueError("Nonblank unpadded district required")
35     reached=int(row["reached"]); eligible=int(row["eligible"])
36     if band not in {"under65","65plus"} or band in groups[district]:
37         raise ValueError("Two distinct age bands per district required")
38     if eligible<=0 or not 0<=reached<=eligible:
39         raise ValueError("Invalid counts")
40     groups[district][band]=(reached,eligible)
41 report=[]
42 for district,bands in sorted(groups.items()):
43     if set(bands)!={"under65","65plus"}:
44         raise ValueError("Missing age band")
45     reached=sum(x[0] for x in bands.values())
46     eligible=sum(x[1] for x in bands.values())
47     rates={band:100*n/d for band,(n,d) in bands.items()}
48     report.append(dict(district=district,reached=reached,eligible=eligible,
49         crude_percent=round(100*reached/eligible,1),
50         standardized_percent=round(.5*rates["under65"]+.5*rates["65plus"],1),
51         weight_under65=.5,weight_65plus=.5))
52 save_rows("district_rates.csv",["district", "reached", "eligible", "
    ↪ crude_percent", "standardized_percent", "weight_under65", "
    ↪ weight_65plus"],report)
53 for row in report:
54     print(f'{row["district"]}: crude {row["crude_percent"]}% , standardized {row
    ↪ ["standardized_percent"]}%')

```

Journey Tests

The journey test runner checks the reusable programs against named expected cases. It tests more than whether the process completes: source conservation, exact review reasons, duplicate correction, date boundaries, visit anchors, input headers, record order, escaping, person selection, overwrite refusal, and repeated runs.

20.1 Run the Reference Version

Working files: complete listings in Chapter 20 below.

Inputs: `journey_demo.csv`, `journey_problem.csv`.

Local modules: `journey_check.py`, `journey_report.py`, `journey_view.py`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 journey_test_runner.py
```

Rule to preserve. Each trial either completes its assertions or records a failure with a detail. The full result CSV is written even when one trial fails. Any failure produces a nonzero process exit code. Tests retain exact source values and identities rather than accepting an unchanged total alone.

20.2 Check the Result

Outputs: `journey_tests.csv`.

Check the meaning

The baseline has 12 PASS rows and no FAIL rows. In particular, `problem_reasons` verifies the six exact source rows and reasons; `person_selection` checks P03's mismatched record; `no_overwrite` compares the earlier files before and after a refused run. Read those tests, not just the final count.

20.3 Read the Key Lines

```

25 def require_retained(source, report):
26     require(len(report) == len(source), "All input records must remain")
27     require({r["data_row"] for r in report} == set(range(1, len(source)+1)),
28             "Source-row identities were lost or duplicated")
29     by_row = {r["data_row"]: r for r in report}
30     for number, original in enumerate(source, 1):
31         require(all(by_row[number][key] == value
32                    for key, value in original.items()),
33                "Source values changed in row " + str(number))

```

Excerpt from journey_test_runner.py, source lines 25–33.

Assert the identity, not only the size. `require_retained` checks row count, source positions, and every supplied field. A report with the right number of rows but altered values must fail.

Temporary folders isolate file tests. `TemporaryDirectory` supplies fresh destinations for tests of selection, overwriting, and repeatability. These tests do not need to erase a reader's previous output folder.

Test the tests with a deliberate defect. A suite that never fails may be checking too little. A controlled mutation of one rule, made only in a disposable copy, can show that at least the relevant check detects that defect. This is evidence about that mutation, not universal test coverage.

20.4 Try One Change

In a disposable copy of `journey_check.py`, change the event-before-visit comparison from `days[i] < days[anchor]` to `days[i] <= days[anchor]`. Run the suite in a fresh folder. It must report failure, including `same_day`; the defective comparison now rejects permitted same-day events. Restore the original program rather than changing expected answers to force a pass.

20.5 Run the Separate Large-File Suite

Keep the additional `journey_large_tests.py` printed below with all three journey programs and the actual `journey_large.csv`. Choose a new output folder whose parent exists:

```
python3 journey_large_tests.py journey_large.csv large_test_run
```

The reference run reports ten passed and zero failed. Its `journey_large_tests.csv` retains one named row per check. The suite checks file shape and identities, population

and event kinds, date range, clean classification, the saved complete report, a P0001 report, row reordering, a missing date, a duplicate across two people, and protected repeat behavior. Several checks create temporary or named test outputs; the original large input remains unchanged.

These ten tests do not replace the twelve small-example tests. The controlled duplicate changes E000014's ID to E000002 in a copy. Full-input checking must flag both occurrences before selecting P0001. The selected report has eleven placed and one review row, not twelve placed rows. Keep the test reports distinct and inspect the actual browser pages separately. The suite generates HTML but does not operate a browser.

20.6 Complete Working Files

Save this chapter's complete program below and supply these unchanged earlier listings: `journey_check.py` from Chapter 2; `journey_report.py` from Chapter 3; `journey_view.py` from Chapter 1.

`journey_demo.csv`

Save the following complete contents as `journey_demo.csv`.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
E01,P01,C01,2026-06-01,visit,Visit recorded
E02,P01,C01,2026-06-02,test,Sample collected
E03,P01,C01,2026-06-05,followup,Follow-up recorded
E04,P01,C02,2026-06-12,visit,Visit recorded
E05,P01,C02,2026-06-12,measurement,Temperature 37.0 C
E06,P01,C02,2026-06-18,followup,Follow-up recorded
E07,P02,C03,2026-06-03,visit,Visit recorded
E08,P02,C03,2026-06-04,test,Sample collected
E09,P02,C03,2026-06-10,followup,Follow-up recorded
E10,P02,C04,2026-06-15,visit,Visit recorded
E11,P02,C04,2026-06-15,measurement,Temperature 36.8 C
E12,P02,C04,2026-06-22,followup,Follow-up recorded
E13,P03,C05,2026-06-06,visit,Visit recorded
E14,P03,C05,2026-06-07,test,Sample collected
E15,P03,C05,2026-06-13,followup,Follow-up recorded
E16,P03,C06,2026-06-20,visit,Visit recorded
E17,P03,C06,2026-06-20,measurement,Temperature 37.1 C
E18,P03,C06,2026-06-26,followup,Follow-up recorded
```

journey_problem.csv

Save the following complete contents as `journey_problem.csv`.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
E01,P01,C01,2026-06-01,visit,Visit recorded
E02,P01,C01,2026-06-02,test,Sample collected
E03,P01,C01,2026-05-31,followup,Follow-up recorded
E04,P01,C02,2026-06-12,visit,Visit recorded
E05,P01,C02,2026-06-12,measurement,Temperature 37.0 C
E06,P01,C02,2026-06-18,followup,Follow-up recorded
E07,P02,C03,2026-06-03,visit,Visit recorded
E08,P02,C03,,test,Sample collected
E09,P02,C03,2026-06-10,followup,Follow-up recorded
E10,P02,C04,2026-06-15,visit,Visit recorded
E11,P02,C04,2026-06-31,measurement,Temperature 36.8 C
E12,P02,C04,2026-06-22,followup,Follow-up recorded
E13,P03,C05,2026-06-06,visit,Visit recorded
E02,P03,C05,2026-06-07,test,Sample collected
E15,P03,C05,2026-06-13,followup,Follow-up recorded
E16,P03,C06,2026-06-20,visit,Visit recorded
E17,P03,C06,2026-06-20,measurement,Temperature 37.1 C
E18,P01,C06,2026-06-26,followup,Follow-up recorded
```

journey_test_runner.py

Complete program. Save this listing as `journey_test_runner.py`.

```
1 import csv
2 import tempfile
3 from pathlib import Path
4 from journey_view import read_events, render_page
5 from journey_check import check_events
6 from journey_report import build_report
7
8 ROOT = Path(__file__).resolve().parent
9 clean = read_events(ROOT/"journey_demo.csv")
10 problem = read_events(ROOT/"journey_problem.csv")
11 results = []
12
13 def require(condition, message):
14     if not condition:
15         raise AssertionError(message)
16
17 def trial(name, operation):
18     try:
19         operation()
```

```

20     except Exception as error:
21         results.append(dict(test=name,status="FAIL",detail=str(error)))
22     else:
23         results.append(dict(test=name,status="PASS",detail=""))
24
25 def require_retained(source, report):
26     require(len(report) == len(source), "All input records must remain")
27     require({r["data_row"] for r in report} == set(range(1, len(source)+1)),
28             "Source-row identities were lost or duplicated")
29     by_row = {r["data_row"]: r for r in report}
30     for number, original in enumerate(source, 1):
31         require(all(by_row[number][key] == value
32                    for key, value in original.items()),
33                "Source values changed in row " + str(number))
34
35 def counts_test():
36     report = check_events(clean)
37     require_retained(clean, report)
38     require(len(report)==18 and all(r["status"]=="placed" for r in report),
39           "Clean file must place all 18 events")
40
41 def problems_test():
42     report = check_events(problem)
43     require_retained(problem, report)
44     expected = {2:"duplicate event_id",3:"event before visit",
45                8:"missing event_date",11:"invalid event_date",
46                14:"duplicate event_id",18:"person differs from visit"}
47     require(len(report)==18, "All 18 input records must remain")
48     require({r["data_row"] for r in report}==set(range(1,19)),
49           "Input row identities were lost")
50     actual = {r["data_row"]:r["issues"] for r in report if r["status"]=="review
51     ↩ }
52     require(actual==expected,"Wrong review rows or reasons")
53     require(report[17]["owner_ref"]=="P03" and report[17]["person_ref"]=="P01",
54           "Person mismatch must stay visible")
55
56 def duplicate_test():
57     rows = [dict(r) for r in problem]
58     rows[13]["event_id"] = "E14"
59     report = check_events(rows)
60     require_retained(rows, report)
61     require(sum(r["status"]=="review" for r in report)==4,
62           "Correcting duplicate ID should clear two rows")
63
64 def same_day_test():
65     rows = [dict(r) for r in clean]
66     rows[2]["event_date"] = rows[0]["event_date"]

```

```

66     report = check_events(rows)
67     require_retained(rows, report)
68     require(report[2]["status"]=="placed",
69             "Same-day follow-up is permitted")
70
71 def missing_anchor_test():
72     rows = [r for r in clean if r["event_id"]!="E16"]
73     report = check_events(rows)
74     require_retained(rows, report)
75     require(len(report) == 17, "Removing E16 leaves 17 retained records")
76     dependent = [r for r in report if r["encounter_id"] == "C06"]
77     require({r["event_id"] for r in dependent} == {"E17", "E18"},
78             "Both dependent identities E17 and E18 must remain")
79     require(all(r["status"] == "review" for r in dependent),
80             "Dependent events require a visit")
81
82 def multiple_anchor_test():
83     rows = [dict(r) for r in clean] + [dict(clean[15], event_id="E19")]
84     report = check_events(rows)
85     require_retained(rows, report)
86     require(len(report)==19, "All 19 rows must remain")
87     require({r["event_id"] for r in report if r["encounter_id"]=="C06"}==
88             {"E16","E17","E18","E19"}, "All four encounter identities must
89     ↪ remain")
90     require(sum(r["status"]=="review" for r in report if r["encounter_id"]=="
91     ↪ C06")==4,
92             "Ambiguous encounter must withhold all four rows")
93
94 def header_test():
95     with tempfile.TemporaryDirectory() as folder:
96         path = Path(folder)/"bad.csv"
97         path.write_text("wrong,header\n",encoding="utf-8")
98         try:
99             read_events(path)
100         except ValueError:
101             return
102         raise AssertionError("Bad header accepted")
103
104 def order_test():
105     def key(rows):
106         report = check_events(rows)
107         require_retained(rows, report)
108         return {(r["event_id"],r["encounter_id"]):
109                 (r["status"],r["issues"],r["owner_ref"]) for r in report}
110     require(key(problem)==key(list(reversed(problem))),
111             "Classification changed when rows moved")

```

```

111 def html_test():
112     rows = [dict(r) for r in clean]
113     rows[0]["detail"] = "<b>Teaching & example</b>"
114     page = render_page(check_events(rows))
115     require("<b>Teaching & example</b>" in page
116             and "<b>Teaching & example</b>" not in page,
117             "Supplied details interpreted as HTML")
118
119 def selection_test():
120     with tempfile.TemporaryDirectory() as folder:
121         target = Path(folder)/"P03"
122         counts = build_report(ROOT/"journey_problem.csv",target,"P03")
123         require(counts==dict(input_records=18,selected_records=6,placed=4,
124                               ↪ review=2),
125                 "Wrong selected counts")
126         with (target/"journey_report.csv").open() as stream:
127             rows = list(csv.DictReader(stream))
128             require(any(r["event_id"]=="E18" and r["person_ref"]=="P01" for r in
129                               ↪ rows),
130                     "Selection lost the mismatched source record")
131
132 def overwrite_test():
133     with tempfile.TemporaryDirectory() as folder:
134         target = Path(folder)/"existing"
135         build_report(ROOT/"journey_problem.csv",target)
136         before = {p.name:p.read_bytes() for p in target.iterdir()}
137         try:
138             build_report(ROOT/"journey_problem.csv",target)
139         except FileExistsError:
140             pass
141         else:
142             raise AssertionError("Existing output folder accepted")
143         require(before=={p.name:p.read_bytes() for p in target.iterdir()},
144                 "Earlier output changed")
145
146 def repeat_test():
147     with tempfile.TemporaryDirectory() as folder:
148         a,b = Path(folder)/"first",Path(folder)/"second"
149         build_report(ROOT/"journey_problem.csv",a)
150         build_report(ROOT/"journey_problem.csv",b)
151         require({p.name:p.read_bytes() for p in a.iterdir()}==
152                 {p.name:p.read_bytes() for p in b.iterdir()},
153                 "Repeated outputs differ")
154
155 for name,operation in [
156     ("clean_counts",counts_test),("problem_reasons",problems_test),
157     ("duplicate_correction",duplicate_test),("same_day",same_day_test),

```

```

156     ("missing_visit",missing_anchor_test),("multiple_visits",
    ↪ multiple_anchor_test),
157     ("bad_header",header_test),("row_order",order_test),
158     ("escaped_HTML",html_test),("person_selection",selection_test),
159     ("no_overwrite",overwrite_test),("repeat_run",repeat_test)]:
160     trial(name,operation)
161
162 with (ROOT/"journey_tests.csv").open("x",newline="",encoding="utf-8") as stream
    ↪ :
163     writer = csv.DictWriter(stream,fieldnames=["test","status","detail"])
164     writer.writeheader()
165     writer.writerows(results)
166 print("Passed:",sum(r["status"]=="PASS" for r in results))
167 print("Failed:",sum(r["status"]=="FAIL" for r in results))
168 if any(r["status"]=="FAIL" for r in results):
169     raise SystemExit(1)

```

journey_large_tests.py

Complete program. Save this listing as journey_large_tests.py.

```

1  """Ten named checks for the book's unchanged journey_large.csv fixture.
2
3  These fixed expectations belong to this supplied synthetic dataset, not
4  arbitrary journey files. No test edits the original input or program files.
5  Browser interaction and performance are separate acceptance checks.
6  """
7  import argparse
8  from collections import Counter, defaultdict
9  import csv
10 from datetime import date
11 import hashlib
12 import json
13 from pathlib import Path
14
15 from journey_view import FIELDS, read_events
16 from journey_check import check_events
17 from journey_report import build_report
18
19
20 def load_csv(path: Path) -> list[dict[str, str]]:
21     with path.open(newline="", encoding="utf-8-sig") as stream:
22         return list(csv.DictReader(stream, strict=True))
23
24
25 def save_input(path: Path, rows: list[dict[str, str]]) -> None:

```

```

26 with path.open("x", newline="", encoding="utf-8") as stream:
27     writer = csv.DictWriter(stream, fieldnames=FIELDS)
28     writer.writeheader()
29     writer.writerows(rows)
30
31
32 def fingerprint(folder: Path) -> dict[str, str]:
33     return {p.name: hashlib.sha256(p.read_bytes()).hexdigest()
34             for p in folder.iterdir() if p.is_file()}
35
36
37 def run_tests(source: Path, destination: Path) -> list[dict[str, str]]:
38     source, destination = Path(source), Path(destination)
39     rows = read_events(source)
40     before = hashlib.sha256(source.read_bytes()).hexdigest()
41     destination.mkdir(parents=False, exist_ok=False)
42     results = []
43
44     def test(name, action):
45         try:
46             action()
47             results.append(dict(test=name, status="PASS", detail=""))
48         except Exception as error:
49             results.append(dict(test=name, status="FAIL",
50                                 detail=type(error).__name__ + ": " + str(error)))
51
52     def shape_and_ids():
53         assert len(rows) == 12000, "Expected 12,000 source rows"
54         assert len({r["event_id"] for r in rows}) == 12000, "Event IDs not
↪ unique"
55         assert set(rows[0]) == set(FIELDS), "Wrong source fields"
56
57     def populations_and_kinds():
58         people, encounters = defaultdict(set), defaultdict(Counter)
59         for r in rows:
60             people[r["person_ref"]].add(r["encounter_id"])
61             encounters[r["encounter_id"]][r["event_type"]] += 1
62         assert len(people) == 1000 and len(encounters) == 3000
63         assert all(len(group) == 3 for group in people.values())
64         expected = Counter(visits=1, tests=1, measurements=1, followups=1)
65         assert all(group == expected for group in encounters.values())
66
67     def date_range():
68         days = [date.fromisoformat(r["event_date"]) for r in rows]
69         assert min(days) == date(2026, 1, 1)
70         assert max(days) == date(2026, 8, 29)
71

```

```

72 def clean_classification():
73     report = check_events(rows)
74     assert len(report) == 12000
75     assert Counter(r["status"] for r in report) == Counter(placed=12000)
76     for n, (original, result) in enumerate(zip(rows, report), 1):
77         assert {k: result[k] for k in FIELDS} == original
78         assert result["data_row"] == n
79         assert result["owner_ref"] == original["person_ref"]
80         assert result["issues"] == ""
81
82 def full_saved_report():
83     target = destination / "all_people"
84     counts = build_report(source, target)
85     assert counts == dict(input_records=12000, selected_records=12000,
86                           placed=12000, review=0)
87     saved = load_csv(target / "journey_report.csv")
88     assert len(saved) == 12000
89     expected = Counter(tuple(r[k] for k in FIELDS) for r in rows)
90     actual = Counter(tuple(r[k] for k in FIELDS) for r in saved)
91     assert actual == expected, "Lost, duplicated, or changed source values"
92     record = json.loads((target / "run_record.json").read_text())
93     assert record["input_sha256"] == before
94     assert (target / "journey_timeline.html").stat().st_size > 0
95
96 def selected_person():
97     target = destination / "P0001"
98     counts = build_report(source, target, "P0001")
99     assert counts == dict(input_records=12000, selected_records=12,
100                           placed=12, review=0)
101     saved = load_csv(target / "journey_report.csv")
102     assert len(saved) == 12
103     assert {r["event_id"] for r in saved} == {f"E{i:06d}" for i in range(1,
104     ↪ 13)}
105     assert {r["person_ref"] for r in saved} == {"P0001"}
106
107 def row_order():
108     keys = FIELDS+["owner_ref", "status", "issues"]
109     def by_id(report):
110         return {r["event_id"]: tuple(r[k] for k in keys) for r in report}
111     assert by_id(check_events(rows)) == by_id(check_events(list(reversed(
112     ↪ rows))))
113
114 def missing_date():
115     changed = [dict(r) for r in rows]
116     target = next(r for r in changed if r["event_id"] == "E000002")
117     target["event_date"] = ""
118     report = check_events(changed)

```

```

117     assert len(report) == 12000
118     assert Counter(r["status"] for r in report) == Counter(placed=11999,
↪ review=1)
119     review = [r for r in report if r["status"] == "review"]
120     assert len(review) == 1 and review[0]["event_id"] == "E000002"
121     assert review[0]["issues"] == "missing event_date"
122
123     def duplicate_before_selection():
124         changed = [dict(r) for r in rows]
125         next(r for r in changed if r["event_id"] == "E000014")["event_id"] = "
↪ E000002"
126         source_copy = destination / "duplicate.csv"
127         save_input(source_copy, changed)
128         report = check_events(changed)
129         review = [r for r in report if r["status"] == "review"]
130         assert len(report) == 12000 and len(review) == 2
131         assert {r["event_id"] for r in review} == {"E000002"}
132         assert {r["person_ref"] for r in review} == {"P0001", "P0002"}
133         assert all(r["issues"] == "duplicate event_id" for r in review)
134         target = destination / "duplicate_P0001"
135         counts = build_report(source_copy, target, "P0001")
136         assert counts == dict(input_records=12000, selected_records=12,
137                               placed=11, review=1)
138
139     def protected_repeat():
140         target = destination / "protected"
141         build_report(source, target, "P0001")
142         saved = fingerprint(target)
143         try:
144             build_report(source, target, "P0001")
145         except FileExistsError:
146             pass
147         else:
148             raise AssertionError("An existing destination was accepted")
149         assert fingerprint(target) == saved, "Earlier output changed"
150         assert hashlib.sha256(source.read_bytes()).hexdigest() == before
151
152     for name, action in [
153         ("shape_and_ids", shape_and_ids),
154         ("populations_and_kinds", populations_and_kinds),
155         ("date_range", date_range),
156         ("clean_classification", clean_classification),
157         ("full_saved_report", full_saved_report),
158         ("selected_person", selected_person),
159         ("row_order", row_order),
160         ("missing_date", missing_date),
161         ("duplicate_before_selection", duplicate_before_selection),

```

```
162         ("protected_repeat", protected_repeat),
163     ]:
164         test(name, action)
165     with (destination / "journey_large_tests.csv").open("x", newline="",
166         ↪ encoding="utf-8") as stream:
167         writer = csv.DictWriter(stream, fieldnames=["test", "status", "detail"
168         ↪ ])
169         writer.writeheader()
170         writer.writerows(results)
171     return results
172
173 def main() -> None:
174     parser = argparse.ArgumentParser(description=__doc__)
175     parser.add_argument("input_csv")
176     parser.add_argument("new_output_folder")
177     args = parser.parse_args()
178     try:
179         results = run_tests(Path(args.input_csv), Path(args.new_output_folder))
180     except (ValueError, OSError) as error:
181         parser.exit(1, str(error)+"\n")
182     passed = sum(r["status"] == "PASS" for r in results)
183     failed = len(results)-passed
184     print("Passed:", passed)
185     print("Failed:", failed)
186     raise SystemExit(1 if failed else 0)
187
188 if __name__ == "__main__":
189     main()
```

Demonstration Package

The package builder constructs a small demonstration ZIP from an explicit file list. It adds a manifest and instructions. Unrelated files sitting beside the program are not automatically swept into the handover.

21.1 Run the Reference Version

Working files: complete listings in Chapter 21 below.

Inputs: `journey_demo.csv`, `journey_problem.csv`, `package_files.csv`, `expected_summary.txt`.

Files to include: `journey_check.py`, `journey_report.py`, `journey_view.py`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 package_builder.py
```

Rule to preserve. The package list must be nonempty, well formed, and free of duplicate or reserved filenames. Each listed name must be a simple filename from the same folder. Every listed file must exist. The output ZIP is created exclusively, and contains the six source files plus `manifest.json` and `RUN.txt`.

21.2 Check the Result

Outputs: `journey_demo_package.zip`.

Check the meaning

The ZIP has eight members. The source files are the three journey programs, the clean and problem CSVs, and `expected_summary.txt`. Extract it into a fresh folder, run the command in its `RUN.txt`, and check 18 records, 12 placed, and six for review. The manifest's file hashes identify the included bytes.

21.3 Read the Key Lines

```
18 manifest = []
19 for item in items:
20     name = item["filename"]
21     if not name.strip() or name != name.strip() or Path(name).name!=name:
22         raise ValueError("Use simple filenames from this folder")
23     data = (ROOT/name).read_bytes()
24     manifest.append(dict(filename=name,purpose=item["purpose"],
25                          bytes=len(data),sha256=hashlib.sha256(data).hexdigest()))
```

Excerpt from package_builder.py, source lines 18–25.

An explicit list is an inclusion rule. The program reads `package_files.csv`; it does not archive the entire directory. A new file placed in the working folder is therefore excluded unless deliberately added to the list.

Check names before reading files. The simple-filename check prevents the manifest from requesting an arbitrary relative path. Reserved names protect the generated manifest and instructions from collisions with supplied files.

A ZIP container has metadata as well as content. Two runs can produce different ZIP bytes because member timestamps differ. Compare member names and extracted member bytes when checking the packaged content; do not label every container-byte difference a source change.

21.4 Try One Change

Create an unrelated file named `decoy.txt` beside the program, but do not add it to `package_files.csv`. Build the ZIP in a fresh copy. It should still have eight members and no `decoy.txt`. This checks that the explicit inclusion rule controls the package.

21.5 Complete Working Files

Save this chapter's complete program below and supply these unchanged earlier listings: `journey_check.py` from Chapter 2; `journey_report.py` from Chapter 3; `journey_view.py` from Chapter 1.

`expected_summary.txt`

Save the following complete contents as `expected_summary.txt`.

```
input_records: 18
selected_records: 18
placed: 12
review: 6
Saved journey_report.csv, journey_timeline.html, run_record.json
```

`journey_demo.csv`

Save the following complete contents as `journey_demo.csv`.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
E01,P01,C01,2026-06-01,visit,Visit recorded
E02,P01,C01,2026-06-02,test,Sample collected
E03,P01,C01,2026-06-05,followup,Follow-up recorded
E04,P01,C02,2026-06-12,visit,Visit recorded
E05,P01,C02,2026-06-12,measurement,Temperature 37.0 C
E06,P01,C02,2026-06-18,followup,Follow-up recorded
E07,P02,C03,2026-06-03,visit,Visit recorded
E08,P02,C03,2026-06-04,test,Sample collected
E09,P02,C03,2026-06-10,followup,Follow-up recorded
E10,P02,C04,2026-06-15,visit,Visit recorded
E11,P02,C04,2026-06-15,measurement,Temperature 36.8 C
E12,P02,C04,2026-06-22,followup,Follow-up recorded
E13,P03,C05,2026-06-06,visit,Visit recorded
E14,P03,C05,2026-06-07,test,Sample collected
E15,P03,C05,2026-06-13,followup,Follow-up recorded
E16,P03,C06,2026-06-20,visit,Visit recorded
E17,P03,C06,2026-06-20,measurement,Temperature 37.1 C
E18,P03,C06,2026-06-26,followup,Follow-up recorded
```

journey_problem.csv

Save the following complete contents as `journey_problem.csv`.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
E01,P01,C01,2026-06-01,visit,Visit recorded
E02,P01,C01,2026-06-02,test,Sample collected
E03,P01,C01,2026-05-31,followup,Follow-up recorded
E04,P01,C02,2026-06-12,visit,Visit recorded
E05,P01,C02,2026-06-12,measurement,Temperature 37.0 C
E06,P01,C02,2026-06-18,followup,Follow-up recorded
E07,P02,C03,2026-06-03,visit,Visit recorded
E08,P02,C03,,test,Sample collected
E09,P02,C03,2026-06-10,followup,Follow-up recorded
E10,P02,C04,2026-06-15,visit,Visit recorded
E11,P02,C04,2026-06-31,measurement,Temperature 36.8 C
E12,P02,C04,2026-06-22,followup,Follow-up recorded
E13,P03,C05,2026-06-06,visit,Visit recorded
E02,P03,C05,2026-06-07,test,Sample collected
E15,P03,C05,2026-06-13,followup,Follow-up recorded
E16,P03,C06,2026-06-20,visit,Visit recorded
E17,P03,C06,2026-06-20,measurement,Temperature 37.1 C
E18,P01,C06,2026-06-26,followup,Follow-up recorded
```

package_files.csv

Save the following complete contents as `package_files.csv`.

```
filename,purpose
journey_view.py,Timeline display from Chapter 1
journey_check.py,Record checker from Chapter 2
journey_report.py,Reusable runner from Chapter 3
journey_demo.csv,Clean synthetic example
journey_problem.csv,Example with six review rows
expected_summary.txt,Expected problem-file results
```

package_builder.py

Complete program. Save this listing as package_builder.py.

```

1  import csv
2  import hashlib
3  import json
4  import zipfile
5  from pathlib import Path
6
7  ROOT = Path(__file__).resolve().parent
8  with (ROOT/"package_files.csv").open(newline="",encoding="utf-8") as stream:
9      reader = csv.DictReader(stream, strict=True)
10     if reader.fieldnames != ["filename", "purpose"]:
11         raise ValueError("Wrong package-list header")
12     items = list(reader)
13     if not items or any(None in r or None in r.values() for r in items):
14         raise ValueError("A nonempty, well-formed package list is required")
15     names = [r["filename"] for r in items]
16     if len(set(names))!=len(names) or set(names)&{"manifest.json","RUN.txt"}:
17         raise ValueError("Duplicate or reserved package filename")
18     manifest = []
19     for item in items:
20         name = item["filename"]
21         if not name.strip() or name != name.strip() or Path(name).name!=name:
22             raise ValueError("Use simple filenames from this folder")
23         data = (ROOT/name).read_bytes()
24         manifest.append(dict(filename=name,purpose=item["purpose"],
25                             bytes=len(data),sha256=hashlib.sha256(data).hexdigest()))
26     instructions = """SYNTHETIC PATIENT-JOURNEY DEMONSTRATION
27 Release: medical-teaching-2026-09-19-r3.
28 The manifest identifies the exact supplied program versions.
29 Extract all files into a fresh folder.
30 Ask ChatGPT with Python execution enabled to run:
31 python3 journey_report.py journey_problem.csv first_run
32 Compare the counts with expected_summary.txt.
33 Open first_run/journey_timeline.html and inspect all three panels.
34 Check all 18 CSV rows, including the six retained for review.
35 For the clean example, use journey_demo.csv and a different new output folder.
36 The three program files must remain together.
37 These files are teaching examples, not approved clinical software.
38 """
39     with zipfile.ZipFile(ROOT/"journey_demo_package.zip","x",zipfile.ZIP_DEFLATED)
40         ↪ as package:
41         for item in items:
42             package.write(ROOT/item["filename"],arcname=item["filename"])
43             package.writestr("manifest.json",json.dumps(manifest,indent=2))
44             package.writestr("RUN.txt",instructions)

```

```
44 print("Package files:",len(items)+2)
45 print("Source files:",len(items))
46 print("Saved journey_demo_package.zip")
```

Review-Status Check

The review-status checker organizes an invented register of review records. It asks whether the listed version, decision, reviewer role, and validity date meet the supplied completeness rule. It cannot decide whether a review was adequate or grant approval for real use.

22.1 Run the Reference Version

Working files: complete listings in Chapter 22 below.

Inputs: `review_register.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 review_status.py
```

Rule to preserve. The register must contain exactly one row for each of Specification, Program, Tests, Data handling, and User guide. A complete record needs an accepted decision, matching nonblank versions, a reviewer role, and a valid-until date not earlier than September 17, 2026. Several blocker reasons can belong to one artifact.

22.2 Check the Result

Outputs: `review_status_report.csv`.

Check the meaning

Specification and User guide are complete. Program has a version mismatch; Tests has both an unaccepted review and a missing/mismatched version; Data handling has an expired review. Thus three records have blockers but there are four reasons. Every output row must retain `as_of=2026-09-17` and `rule_version=review-register-v1`.

22.3 Read the Key Lines

```
43 for row in rows:
44     issues=[]
45     if row["decision"]!="accepted":
46         issues.append("review not accepted")
47     if (not row["current_version"].strip() or not row["reviewed_version"].strip()
48         ↪ )
49         or row["current_version"]!=row["reviewed_version"]):
50         issues.append("missing or mismatched version")
51     if not row["reviewer_role"].strip():
52         issues.append("reviewer missing")
53     try:
54         if parse_day(row["valid_until"])<AS_OF:
55             issues.append("review expired")
56     except ValueError:
57         issues.append("invalid review date")
```

Excerpt from review_status.py, source lines 43–56.

A set comparison checks the inventory. The combination of a duplicate check and equality with the required artifact set ensures neither omissions nor extra categories. A count of five alone would not establish the correct inventory.

Accumulate independent blockers. Separate conditions append independent issues. Replacing them with an `elif` chain would lose later reasons after the first problem was found.

Use a fixed reporting date for a reproducible example. `AS_OF` is supplied in the program, not taken from the computer clock. Repeating the example tomorrow therefore uses the same contract unless that value is deliberately changed.

22.4 Try One Change

In the invented Program row, change `reviewed_version` from 3 to 4 so it matches `current_version`. Expect three complete records and two blocker records. Tests still has its two reasons; the expired Data handling review is unchanged. This input change does not itself establish that a real review occurred.

22.5 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

review_register.csv

Save the following complete contents as `review_register.csv`.

```
artifact,current_version,reviewed_version,decision,reviewer_role,valid_until
Specification,2,2,accepted,project lead,2026-12-31
Program,4,3,accepted,technical reviewer,2026-12-31
Tests,4,,pending,test reviewer,2026-12-31
Data handling,1,1,accepted,privacy reviewer,2026-09-01
User guide,2,2,accepted,user representative,2026-12-31
```

review_status.py

Complete program. Save this listing as `review_status.py`.

```
1 import csv
2 from pathlib import Path
3
4 ROOT = Path(__file__).resolve().parent
5
6 SCHEMAS = {'review_register.csv': ['artifact', 'current_version', '
    ↪ reviewed_version', 'decision', 'reviewer_role', 'valid_until']}
7
8 def read_rows(name):
9     with (ROOT / name).open(newline="", encoding="utf-8-sig") as stream:
10         reader = csv.DictReader(stream, strict=True)
11         if reader.fieldnames != SCHEMAS[name]:
12             raise ValueError("Wrong header in " + name)
13         rows = list(reader)
14         if any(None in row or None in row.values() for row in rows):
15             raise ValueError("Malformed CSV row in " + name)
16         return rows
17
18 from datetime import date as _date
19
20 def parse_day(value):
21     day = _date.fromisoformat(value)
22     if day.isoformat() != value:
23         raise ValueError("Use exact YYYY-MM-DD dates")
24     return day
25
```

```

26
27 def save_rows(name, fields, rows):
28     with (ROOT / name).open("x", newline="", encoding="utf-8") as stream:
29         writer = csv.DictWriter(stream, fieldnames=fields)
30         writer.writeheader()
31         writer.writerows(rows)
32
33 from datetime import date
34
35 AS_OF=date(2026,9,17)
36 RULE_VERSION = "review-register-v1"
37 required={"Specification","Program","Tests","Data handling","User guide"}
38 rows=read_rows("review_register.csv")
39 names=[r["artifact"] for r in rows]
40 if len(set(names))!=len(names) or set(names)!=required:
41     raise ValueError("Exactly one row for every required artifact")
42 report=[]
43 for row in rows:
44     issues=[]
45     if row["decision"]!="accepted":
46         issues.append("review not accepted")
47     if (not row["current_version"].strip() or not row["reviewed_version"].strip()
48         ↪ ())
49         or row["current_version"]!=row["reviewed_version"]):
50         issues.append("missing or mismatched version")
51     if not row["reviewer_role"].strip():
52         issues.append("reviewer missing")
53     try:
54         if parse_day(row["valid_until"])<AS_OF:
55             issues.append("review expired")
56     except ValueError:
57         issues.append("invalid review date")
58     report.append(dict(artifact=row["artifact"],as_of=AS_OF.isoformat(),
59         ↪ rule_version=RULE_VERSION,
60         ↪ current_version=row["current_version"],reviewed_version=row["
61         ↪ reviewed_version"],
62         ↪ status="blocker" if issues else "record complete",
63         ↪ reason="; ".join(issues)))
64 save_rows("review_status_report.csv",["artifact", "as_of", "rule_version",
65     ↪ "current_version", "reviewed_version", "status", "reason"],report)
66 print("As of:",AS_OF)
67 print("Rule version:",RULE_VERSION)
68 print("Records complete:",sum(r["status"]=="record complete" for r in report))
69 print("Records with blockers:",sum(r["status"]=="blocker" for r in report))
70 print("This checks recorded review completeness, not clinical approval.")

```

Follow-Up Worklist

The final chapter classifies an invented administrative encounter file and writes both a CSV worklist and a browser view. Status and owner controls filter the displayed rows; they do not alter the underlying records. The program sends no messages, changes no appointments, and assigns no clinical priority.

23.1 Run the Reference Version

Working files: complete listings in Chapter 23 below.

Inputs: `worklist_encounters.csv`.

Create the files listed under *Complete Working Files* in a fresh working folder. From that folder, run:

```
python3 followup_worklist.py
```

Rule to preserve. Encounter IDs are distinct. The default reporting date is September 17, 2026, and the window is 14 calendar days. Invalid dates, missing contact/owner details, future index dates, or inconsistent follow-up dates cause review data. Otherwise a supplied valid follow-up is recorded, even if late; without one, the window is passed only when the reporting date is strictly later than its end.

23.2 Check the Result

Outputs: `followup_worklist.csv`, `followup_worklist.html`, `worklist_run.json`.

Check the meaning

E01 and E06 have follow-up recorded. E02 has no follow-up after its window. E03 remains open until September 24 inclusive. E04 has a follow-up before its index date and requires review; E05 requires review for a missing contact route. All six encounter IDs must remain. A recorded late follow-up and a within-window match are different concepts, as the capstone demonstrates.

23.3 Read the Key Lines

```

63         if issues:
64             status = "review data"
65         elif followup:
66             status = "follow-up recorded"
67         elif as_of > due:
68             status = "no follow-up after window"
69         else:
70             status = "window still open"
71         report.append(dict(row, data_row=number, as_of=as_of.isoformat(),
72                             window_days=window, rule_version=VERSION,
73                             due_date=due.isoformat() if due else "", status=status,
74                             reason="; ".join(dict.fromkeys(issues))))

```

Excerpt from followup_worklist.py, source lines 63–74.

Decision precedence prevents misleading labels. The `if issues` branch comes before the recorded, passed-window, and open-window branches. A date or contact problem must not be hidden behind an otherwise plausible administrative label.

A strict comparison keeps the end date inclusive. `as_of > due` means the window is still open on its end date. Changing this to greater-than-or-equal would alter the boundary contract.

Filtering is not deletion. JavaScript hides rows that do not match the selected status and owner. The count and empty-state message describe the visible subset. The CSV and run record remain the complete report.

23.4 Try One Change

Run `python3 followup_worklist.py --as-of 2026-09-25` in a fresh copy. E03 should now have no follow-up after window, giving two recorded, two passed-window, zero open, and two review records. On September 24 itself, E03 is still open under the inclusive-end rule.

23.5 Specify the Large-Data Worklist Extension

The large journey source has encounter and recorded follow-up dates, but no contact route or owner. A proposed conversion must make that absence visible. Using the main book's September 17, 2026 reporting date, all 3,000 derived rows with those administrative fields blank need review. If the classroom explicitly supplies contact route *office* and owner *Teaching Team* for every encounter, all 3,000 become follow-up recorded. Record those additions as fictional configuration, not information extracted from the source.

The current capstone adapter does not accept the journey schema directly. The main book's Stage C is a specified extension, not an undocumented command. Preserve all original event IDs in its conversion ledger, and keep recorded follow-up separate from the 1,684 fourteen-day matches. Weekly final/provisional flags require another supplied decision.

23.6 Complete Working Files

The complete program and small input are printed below. Keep the named files together.

worklist_encounters.csv

Save the following complete contents as `worklist_encounters.csv`.

```
encounter_id,index_date,followup_date,contact_route,owner
E01,2026-09-01,2026-09-05,office,Team A
E02,2026-09-01,,office,Team A
E03,2026-09-10,,office,Team B
E04,2026-09-01,2026-08-31,office,Team B
E05,2026-09-02,,,Team A
E06,2026-09-02,2026-09-16,office,Team B
```

followup_worklist.py

Complete program. Save this listing as `followup_worklist.py`.

```
1  """Synthetic administrative worklist; no messages are sent."""
2  import argparse
3  import csv
4  import hashlib
5  import json
6  from collections import Counter
7  from datetime import date, timedelta
8  from html import escape
9  from pathlib import Path
10
11  ROOT = Path(__file__).resolve().parent
12  FIELDS = ["encounter_id", "index_date", "followup_date", "contact_route", "
    ↪ owner"]
13  EXTRA = ["data_row", "as_of", "window_days", "rule_version", "due_date", "
    ↪ status", "reason"]
14  VERSION = "worklist-2"
15  STATUSES = ["follow-up recorded", "no follow-up after window",
16             "window still open", "review data"]
```

```

17
18 def parse_day(value):
19     day = date.fromisoformat(value)
20     if day.isoformat() != value:
21         raise ValueError("Use exact YYYY-MM-DD dates")
22     return day
23
24 def read_rows(path):
25     with Path(path).open(newline="", encoding="utf-8-sig") as stream:
26         reader = csv.DictReader(stream, strict=True)
27         if reader.fieldnames != FIELDS:
28             raise ValueError("Wrong worklist header")
29         rows = list(reader)
30     if any(None in r or None in r.values() for r in rows):
31         raise ValueError("Malformed CSV row")
32     return rows
33
34 def classify(rows, as_of, window=14, upstream=None):
35     if not isinstance(window, int) or window < 0:
36         raise ValueError("Window must be a nonnegative integer")
37     ids = [r["encounter_id"] for r in rows]
38     if (len(set(ids)) != len(ids) or
39         any(not x.strip() or x != x.strip() for x in ids)):
40         raise ValueError("Distinct nonblank unpadded encounter IDs required")
41     report = []
42     for number, row in enumerate(rows, 1):
43         issues = list((upstream or {}).get(row["encounter_id"], []))
44         start = due = followup = None
45         try:
46             start = parse_day(row["index_date"])
47             due = start + timedelta(days=window)
48         except (ValueError, OverflowError):
49             issues.append("invalid index date or window end")
50         if not row["contact_route"].strip():
51             issues.append("contact route missing")
52         if not row["owner"].strip():
53             issues.append("owner missing")
54         if row["followup_date"].strip():
55             try:
56                 followup = parse_day(row["followup_date"])
57             except ValueError:
58                 issues.append("invalid follow-up date")
59         if followup and ((start and followup < start) or followup > as_of):
60             issues.append("inconsistent follow-up date")
61         if start and start > as_of:
62             issues.append("future index date")
63     if issues:

```

```

64         status = "review data"
65     elif followup:
66         status = "follow-up recorded"
67     elif as_of > due:
68         status = "no follow-up after window"
69     else:
70         status = "window still open"
71     report.append(dict(row, data_row=number, as_of=as_of.isoformat(),
72                       window_days=window, rule_version=VERSION,
73                       due_date=due.isoformat() if due else "", status=status,
74                       reason="; ".join(dict.fromkeys(issues))))
75     return sorted(report, key=lambda r: (r["status"], r["due_date"], r["
    ↪ encounter_id"]))
76
77 def render_page(report, as_of, window=14):
78     counts = Counter(r["status"] for r in report)
79     options = '<option value="">All statuses</option>' + "".join(
80         '<option value="'+escape(s, quote=True)+'">'+escape(s)+
81         f' ({counts[s]})</option>' for s in STATUSES)
82     owners = '<option value="">All owners</option>' + "".join(
83         '<option>'+escape(s)+'</option>' for s in sorted({r["owner"] for r in
    ↪ report})) if s)
84     visible = ["encounter_id", "owner", "due_date", "status", "reason"]
85     body = "".join('<tr data-status="'+escape(r["status"], quote=True)+
86         '" data-owner="'+escape(r["owner"], quote=True)+'">'+
87         '"'.join('<td'+escape(str(r[k]))+'</td>' for k in visible)+'</tr>' for
    ↪ r in report)
88     return f'''<!doctype html><html lang="en"><meta charset="utf-8">
89     <meta name="viewport" content="width=device-width,initial-scale=1">
90     <title>Synthetic follow-up worklist</title>
91     <style>body{{font:18px system-ui;margin:24px}}td,th{{padding:10px;text-align:
    ↪ left;
92     border-bottom:1px solid #aaa}}.scroll{{overflow:auto}}select{{font:inherit}}
93     label{{display:inline-block;margin:8px 16px 8px 0}}</style>
94     <h1>Synthetic follow-up worklist</h1>
95     <p>As of {as_of.isoformat()}; window {window} calendar days; rules {VERSION}</
    ↪ p>
96     <p>Administrative teaching records only. No clinical priority is assigned.
97     No messages are sent and no appointment is changed.</p>
98     <label>Status <select id="filter">{options}</select></label>
99     <label>Owner <select id="owner">{owners}</select></label>
100    <p id="count" aria-live="polite"></p><p id="empty" hidden>No rows match.</p>
101    <div class="scroll"><table><thead><tr><th>Encounter</th><th>Owner</th>
102    <th>Window end</th><th>Status</th><th>Reason</th></tr></thead>
103    <tbody>{body}</tbody></table></div><script>
104    function update(){{
105        const status=document.getElementById("filter").value;

```

```

106 const owner=document.getElementById("owner").value;
107 let n=0;
108 document.querySelectorAll("tbody tr").forEach(r=>{{
109     r.hidden=(status!=="&&r.dataset.status!==status)||
110         (owner!=="&&r.dataset.owner!==owner);
111     if(!r.hidden)n++;
112 }});
113 document.getElementById("count").textContent=n+" of {len(report)} records
    ↪ shown";
114 document.getElementById("empty").hidden=n!==0;
115 }}
116 document.getElementById("filter").addEventListener("change",update);
117 document.getElementById("owner").addEventListener("change",update);update();
118 </script></html>'''
119
120 def save_report(folder, report, as_of, window=14, input_path=None):
121     folder = Path(folder)
122     names = ["followup_worklist.csv", "followup_worklist.html", "worklist_run.
    ↪ json"]
123     if any((folder/n).exists() for n in names):
124         raise FileExistsError("Use a fresh output folder")
125     with (folder/names[0]).open("x", newline="", encoding="utf-8") as stream:
126         writer = csv.DictWriter(stream, fieldnames=FIELDS+EXTRA)
127         writer.writeheader()
128         writer.writerows(report)
129     (folder/names[1]).write_text(render_page(report, as_of, window), encoding="
    ↪ utf-8")
130     record = dict(as_of=as_of.isoformat(), window_days=window, rules=VERSION,
131                 records=len(report), counts=dict(Counter(r["status"] for r in
    ↪ report)),
132                 program_sha256=hashlib.sha256(Path(__file__).read_bytes()).
    ↪ hexdigest())
133     if input_path is not None:
134         record["input_sha256"] = hashlib.sha256(Path(input_path).read_bytes()).
    ↪ hexdigest()
135     (folder/names[2]).write_text(json.dumps(record, indent=2), encoding="utf-8"
    ↪ )
136
137 def main():
138     parser = argparse.ArgumentParser(description=__doc__)
139     parser.add_argument("--as-of", default="2026-09-17")
140     parser.add_argument("--window-days", type=int, default=14)
141     args = parser.parse_args()
142     as_of = parse_day(args.as_of)
143     source = ROOT/"worklist_encounters.csv"
144     report = classify(read_rows(source), as_of, args.window_days)
145     save_report(ROOT, report, as_of, args.window_days, source)

```

```
146     for status in STATUSES:
147         print(status+":", sum(r["status"]==status for r in report))
148
149 if __name__ == "__main__":
150     main()
```


The Integrated Capstone

The capstone connects the matcher, weekly monitor, and worklist through an explicit field map. It uses a new invented dataset, not an assumed connection between unrelated chapter files. The three application programs are unchanged copies from Chapters 8, 11, and 23. An adapter prepares their inputs; a test program checks the integrated results.

Create and Run the Fixed Example

Save the three complete CSV listings and the complete adapter and test program printed below. Supply the unchanged `followup_match.py`, `weekly_followup.py`, and `followup_worklist.py` from Chapters 8, 11, and 23. Keep all eight files together in a fresh working folder. Then run these two commands there, in order:

```
python3 capstone_adapter.py
python3 capstone_tests.py
```

The adapter creates a new `integrated_run` folder. The tests read its outputs and write `capstone_tests.csv`. The text listings below provide the field map, Stage-A specification, and project-card template; create actual text files from them when documenting a revised project.

Keep the Three Meanings Separate

The fixed Stage-A reporting date is September 17, 2026. Matching includes elapsed days 0 through 14. The weekly denominator contains all supplied encounters in that dated week. Twenty encounters and twenty follow-up records make a 40-row source ledger; two weekly status flags are separate inputs.

A within-window match is not the same as a recorded follow-up. C103 has valid follow-up at day 16: recorded, but not matched within the window. C114 has a within-window match but lacks a contact route: it is also a review case. A final weekly flag does not imply that every encounter's window has closed.

Read the Field Map

Keep the source rows. The adapter records the original filename, row number, ID, and values. Both occurrences of F09 remain identifiable. Unknown C999 is not converted to a known encounter. Ambiguous or invalid follow-ups remain in the ledger even when they cannot be passed to the matcher.

Prepare each application's input. The matcher receives encounter IDs, person references, index dates, and usable follow-up candidates. Matching is by encounter ID, never by person alone. The worklist receives the earliest valid recorded follow-up on or after the index date and on or before the reporting date, without imposing the matching window. Upstream problems are passed separately to `classify`.

Make weekly counts traceable. The adapter groups encounters by the Monday of their index date. Its `week_map.csv` links dates to the earlier monitor's week numbers. Within-window selections supply the numerator; the denominator ledger identifies the encounters counted. `integration_record.json` records the date, window, denominator choice, interpreter, hashes, and counts.

The adapter rejects an invalid encounter index date because it cannot assign that encounter to a week. This is stricter than the standalone worklist's row-review rule. Reusing a module does not make every input contract identical.

Check the Result

The baseline retains 40 source records and all 20 encounters. It has 11 within-window matches. The worklist has 11 recorded follow-ups, one passed window, one open window, and seven review cases. The two sets of eleven are not identical: the recorded-follow-up set replaces C114 with C103.

The week beginning August 31 has 8 matches among 16 encounters, including 6 review cases. The provisional week beginning September 7 has 3 among 4, including 1 review case. Only the final week contributes to the pooled 50.0%. The tests should report 12 PASS and no FAIL results.

Check Identities, Not Just Totals

Check the meaning

Matched encounters: C101, C102, C107, C111, C112, C113, C114, C115, C116, C117, C118.

Review encounters: C104, C105, C106, C108, C109, C114, C120.

C110 alone has a passed window; C119 alone has a window still open.

The 12 tests cover the fixed contract, retained source and encounter identities, matching and status decisions, late follow-up, overlapping review and match flags, boundaries, duplicate and unknown references, dates, weekly counts, and dated outputs. Read the named assertions in `capstone_tests.py` when a check fails.

The generated run contains staged inputs and execution transcripts as well as the reports. Save and inspect those actual files. The expected values printed here are comparison targets, not a separately downloadable reference-output folder.

Try a Different Contract

For Stage B, record the new decisions in a copy of `PROJECT_CARD.txt`. For example, run this in a fresh capstone copy:

```
python3 capstone_adapter.py --output changed_run \  
--window-days 21 --denominator closed
```

The backslash continues the command onto the next line. Here `closed` means that the reporting date is strictly later than the window end. Under that denominator choice, the weekly numerator and review count also use only included encounters. The full source ledger and worklist remain complete.

Do not apply the fixed Stage-A expectations to a changed contract and edit `FAIL` into `PASS`. Predict new named cases, justify their expected results, and keep the original tests. The complete adapter, tests, field map, and specification are printed below. Preserve the Stage-A versions before requesting changes. The large-file Stage C in the main book needs a separately specified conversion; this adapter does not accept its schema unchanged.

Complete Capstone Working Files

capstone_encounters.csv

Save the following complete contents as `capstone_encounters.csv`.

```

encounter_id,person_ref,index_date,contact_route,owner
C101,P01,2026-09-01,office,Team A
C102,P01,2026-09-01,office,Team B
C103,P02,2026-09-01,office,Team A
C104,P02,2026-09-01,office,Team B
C105,P03,2026-09-01,office,Team A
C106,P03,2026-09-01,office,Team B
C107,P04,2026-09-01,office,Team A
C108,P04,2026-09-01,office,Team B
C109,P05,2026-09-01,office,Team A
C110,P05,2026-09-01,office,Team B
C111,P06,2026-09-03,office,Team A
C112,P06,2026-09-03,office,Team B
C113,P07,2026-09-03,office,Team A
C114,P07,2026-09-03,,Team B
C115,P08,2026-09-03,office,Team A
C116,P08,2026-09-10,office,Team B
C117,P09,2026-09-10,office,Team A
C118,P09,2026-09-10,office,Team B
C119,P10,2026-09-03,office,Team A
C120,P10,2026-09-10,office,

```

capstone_followups.csv

Save the following complete contents as `capstone_followups.csv`.

```

followup_id,encounter_id,followup_date
F01,C101,2026-09-02
F02,C102,2026-09-15
F03,C103,2026-09-17
F04,C104,2026-08-31
F05,C105,2026-09-18
F06,C106,2026-09-31
F07,C107,2026-09-04
F08,C107,2026-09-06
F09,C108,2026-09-04
F09,C109,2026-09-05
F11,C999,2026-09-04
F12,C101,2026-09-07
F13,C111,2026-09-10
F14,C112,2026-09-16
F15,C113,2026-09-17
F16,C114,2026-09-17

```

```
F17,C115,2026-09-17
F18,C116,2026-09-17
F19,C117,2026-09-17
F20,C118,2026-09-17
```

capstone_weeks.csv

Save the following complete contents as `capstone_weeks.csv`.

```
week_start,provisional
2026-08-31,no
2026-09-07,yes
```

FIELD_MAP.txt

Save the following complete contents as `FIELD_MAP.txt`.

```
CAPSTONE FIELD MAP
Original encounter -> matching/encounters.csv:
    encounter_id, person_ref, index_date (copied, never matched by person alone).
Original usable follow-up -> matching/followups.csv:
    followup_id, encounter_id, followup_date (invalid/duplicate/unknown/future
    rows remain in source_ledger.csv, rather than masquerading as valid input).
Matcher selected IDs + dated encounters -> dated weekly_counts.csv:
    group by Monday of index_date; confirmed counts explicit within-window
    selections; denominator contains all stage-A encounters; review counts
    worklist review flags. A dated week_map.csv preserves the mapping to the
    numeric week field required by the earlier weekly monitor.
Original encounter + earliest valid recorded follow-up -> worklist input:
    encounter_id, index_date, followup_date, contact_route, owner.
Upstream review issues are passed separately to classify. A recorded date
    is nonnegative and on/before the reporting date, not restricted to day 14.
Every original source record -> source_ledger.csv with original supplied values.
Every encounter -> denominator_ledger.csv with inclusion reason and window end.
All run decisions and input/program SHA-256 hashes -> integration_record.json.
All report CSVs have declared schemas; all HTML text is escaped by the released
    renderers. No browser filter changes the separately saved CSV.
```

STAGE_A_SPECIFICATION.txt

Save the following complete contents as STAGE_A_SPECIFICATION.txt.

STAGE A - FIXED TEACHING CONTRACT

Release: medical-teaching-2026-09-19-r3

Purpose: an inspectable synthetic follow-up report and administrative worklist.

No sending, clinical urgency assignment, or operational approval is provided.

Reporting date: 2026-09-17. Window: 14 calendar days, inclusive of days 0 and 14.

Twenty encounter rows and twenty follow-up rows form the 40-record source ledger. The two weekly flag records are separate supplied inputs.

All twenty encounters remain in the worklist and denominator ledger.

All forty original rows remain in the source ledger with source filename, row number, supplied identifier, supplied values, disposition and reason.

A duplicate follow-up identifier withholds all occurrences. Their source-row numbers keep the two F09 records distinct. An unknown encounter stays unknown.

Malformed, future and duplicate follow-ups produce review issues; they are not silently deleted. A valid negative-day candidate is shown to the matcher but cannot be recorded follow-up and causes worklist review.

Within-window matching uses encounter identity, earliest date, then lexical follow-up ID as the tie-break. A valid recorded follow-up can be later than 14 days. C103 therefore has recorded follow-up but no within-window match.

Worklist review issues take precedence over recording or window status.

C114 has a within-window match but missing contact; it is a review case.

A window is open through its end date: C119 is open on 2026-09-17.

The baseline denominator is all supplied encounters in each dated week.

Week commencing 2026-08-31 is final: 8 confirmed of 16, with 6 review cases.

Week commencing 2026-09-07 is provisional: 3 of 4, with 1 review case.

Review and matched flags can overlap. A final flag does not mean every encounter window has closed. C119 demonstrates that distinction.

The adapter records the dated week map beside the chapter monitor's numbered input. It runs the earlier matcher and weekly monitor, not substituted summaries. The worklist reuses the same classify and save functions.

Expected match IDs: C101 C102 C107 C111 C112 C113 C114 C115 C116 C117 C118.

Expected recorded IDs: C101 C102 C103 C107 C111 C112 C113 C115 C116 C117 C118.

Review IDs: C104 C105 C106 C108 C109 C114 C120. Passed: C110. Open: C119.

File-level failures: unreadable/malformed CSV, wrong header, ambiguous encounter IDs, invalid encounter index dates, invalid person references, or absent/conflicting weekly flags stop this integration. Readable follow-up-row problems are retained for review. The standalone worklist has a broader row-review policy for invalid index dates; the adapter needs valid index dates to assign weeks and thus explicitly rejects them at this boundary.

Stage B choices: an explicitly supplied window and all-versus-closed denominator. Closed is `as_of > index_date + window_days`, not `>=`. In that variant, matched and review counts in the weekly aggregate refer only to included denominator encounters. The complete worklist and source ledger are never restricted by that reporting denominator. Record the meaning and limits of every output.

PROJECT_CARD.txt

Save the following complete contents as `PROJECT_CARD.txt`.

PROJECT CARD - EDIT A COPY

Project name:

Author/reviewer:

Release/program identifiers:

Date:

1. Intended user and task

What useful result will this synthetic teaching project deliver?

What will it explicitly not do?

2. Inputs and one-row meanings

List each filename, schema, identifier rules and source.

Which rows can repeat a person? Which identifiers must be unique?

What happens to malformed files and to readable rows with invalid fields?

3. Supplied reporting decisions

Reporting date:

Follow-up window, including its endpoints:

Definition of recorded follow-up:

Definition of within-window match:

Denominator: all encounters or only closed windows? Explain the choice.

Exact meaning of closed at the reporting-date boundary:

Weekly final/provisional flags and their source:

4. Field map and row accounting

Map each source to its derived fields and retained-row ledger.

How do duplicate and unknown-reference records remain visible?

Which issue flags can overlap numerator or denominator membership?

5. Expected cases written before running

C103 late-record distinction:

C114 match/review overlap:

C119 boundary:

Duplicate F09 and unknown C999:

An additional unfamiliar case:

6. Actual execution and checks

Input hashes, code hashes, Python version, command and output folder:

Expected/actual comparisons and result:

Saved failure, smallest exposing row(s), narrow repair and rerun:

Keyboard, narrow-window and print checks actually performed:

Checks not performed and why:

7. Handover

Explicit package file list and release identifier:

Fresh extraction and rerun evidence:

Remaining limitations, reviewer questions and next test:

A successful teaching run is not operational or clinical approval.

capstone_adapter.py

Complete program. Save this listing as `capstone_adapter.py`.

```

1  """Integrate the book's synthetic matcher, weekly monitor, and worklist.
2
3  Stage A: 14 calendar days; all encounters in the dated week's denominator.
4  Stage B options are supplied teaching decisions, not clinical recommendations.
5  The original records always remain in the source ledger.
6  """
7  import argparse
8  from collections import Counter, defaultdict
9  import csv
10 from datetime import date, timedelta
11 import hashlib
12 import json
13 from pathlib import Path
14 import platform
15 import shutil
16 import subprocess
17 import sys
18
19 from followup_worklist import classify, save_report, parse_day, FIELDS
20
21 ROOT = Path(__file__).resolve().parent
22 RELEASE = "medical-teaching-2026-09-19-r3"
23 SCHEMAS = {
24     "capstone_encounters.csv": ["encounter_id", "person_ref", "index_date",
25                                "contact_route", "owner"],
26     "capstone_followups.csv": ["followup_id", "encounter_id", "followup_date"],
27     "capstone_weeks.csv": ["week_start", "provisional"],
28 }
29 LEDGER_FIELDS = ["source_file", "data_row", "supplied_id", "encounter_id",

```

```
30         "person_ref", "index_date", "followup_date", "contact_route",
31         "owner", "disposition", "reason"]
32
33
34 def read_csv(path, fields=None):
35     with Path(path).open(newline="", encoding="utf-8-sig") as stream:
36         reader = csv.DictReader(stream, strict=True)
37         if fields is not None and reader.fieldnames != fields:
38             raise ValueError("Wrong header in " + Path(path).name)
39         rows = list(reader)
40         if any(None in r or None in r.values() for r in rows):
41             raise ValueError("Malformed CSV record in " + Path(path).name)
42     return rows
43
44
45 def save_csv(path, fields, rows):
46     with Path(path).open("x", newline="", encoding="utf-8") as stream:
47         writer = csv.DictWriter(stream, fieldnames=fields)
48         writer.writeheader()
49         writer.writerows(rows)
50
51
52 def checksum(path):
53     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
54
55
56 def require_id(value):
57     return bool(value.strip()) and value == value.strip()
58
59
60 def monday(day):
61     return day - timedelta(days=day.weekday())
62
63
64 def run_chapter(name, folder, *args):
65     """Execute an unchanged copy of the released chapter script."""
66     shutil.copy2(ROOT / name, folder / name)
67     result = subprocess.run([sys.executable, str(folder / name), *args],
68                             cwd=folder, text=True, capture_output=True)
69     (folder / "execution.txt").write_text(
70         result.stdout + result.stderr, encoding="utf-8")
71     if result.returncode:
72         raise RuntimeError(name + " failed: " + result.stderr)
73
74
75 def build(destination, as_of, window=14, denominator="all"):
76     if window < 0 or denominator not in {"all", "closed"}:
```

```

77         raise ValueError("Invalid window or denominator decision")
78     source = {name: read_csv(ROOT / name, fields)
79               for name, fields in SCHEMAS.items()}
80     encounters = source["capstone_encounters.csv"]
81     followups = source["capstone_followups.csv"]
82     flags = source["capstone_weeks.csv"]
83     ids = [r["encounter_id"] for r in encounters]
84     if len(set(ids)) != len(ids) or not all(require_id(x) for x in ids):
85         raise ValueError("Distinct nonblank unpadded encounter IDs required")
86     if not encounters:
87         raise ValueError("The integration requires at least one encounter")
88     starts = {}
89     for r in encounters:
90         if not require_id(r["person_ref"]):
91             raise ValueError("Nonblank unpadded person reference required")
92         starts[r["encounter_id"]] = parse_day(r["index_date"])
93     flag_map = {}
94     for r in flags:
95         day = parse_day(r["week_start"])
96         if day.weekday() != 0 or day in flag_map or r["provisional"] not in {"
↪ yes", "no"}:
97             raise ValueError("Distinct Monday weeks with explicit yes/no flags
↪ required")
98         flag_map[day] = r["provisional"]
99     needed = {monday(x) for x in starts.values()}
100     if set(flag_map) != needed:
101         raise ValueError("Supply exactly one status flag for every encounter
↪ week")
102     known = set(ids)
103     duplicated = Counter(r["followup_id"].strip() for r in followups)
104     upstream = defaultdict(list)
105     matcher_followups = []
106     recorded_dates = defaultdict(list)
107     ledger = []
108     for number, row in enumerate(followups, 1):
109         identity, encounter = row["followup_id"], row["encounter_id"]
110         issues = []
111         if not require_id(identity):
112             issues.append("blank or padded follow-up ID")
113         if identity.strip() and duplicated[identity.strip()] > 1:
114             issues.append("duplicate follow-up ID")
115         if not require_id(encounter):
116             issues.append("blank or padded encounter reference")
117         if encounter not in known:
118             issues.append("unknown encounter reference")
119         day = None
120         try:

```

```

121         day = parse_day(row["followup_date"])
122     except ValueError:
123         issues.append("invalid follow-up date")
124     if day and day > as_of:
125         issues.append("future follow-up")
126     usable = not issues
127     if usable:
128         # A negative-day candidate is visible to the matcher, but is also
129         # an upstream issue for the worklist. It is never a recorded date.
130         matcher_followups.append(dict(row))
131         elapsed = (day - starts[encounter]).days
132         if elapsed >= 0:
133             recorded_dates[encounter].append((day, identity))
134         else:
135             issues.append("follow-up before index date")
136     if encounter in known and issues:
137         upstream[encounter].extend(issues)
138     disposition = ("matcher candidate; review" if usable and issues else
139                   "matcher candidate" if usable else "withheld; review")
140     ledger.append(dict(source_file="capstone_followups.csv", data_row=
↪ number,
141                      supplied_id=identity, encounter_id=encounter, person_ref="",
↪ index_date="",
142                      followup_date=row["followup_date"], contact_route="", owner="",
143                      disposition=disposition, reason="; ".join(issues)))
144     work_input = []
145     for row in encounters:
146         dates = sorted(recorded_dates[row["encounter_id"]])
147         work_input.append({key: (dates[0][0].isoformat() if dates else "")
148                           if key == "followup_date" else row[key] for key in
↪ FIELDS})
149     work_report = classify(work_input, as_of, window, upstream)
150     work_by_id = {r["encounter_id"]: r for r in work_report}
151     for number, row in enumerate(encounters, 1):
152         reviewed = work_by_id[row["encounter_id"]]
153         ledger.append(dict(source_file="capstone_encounters.csv", data_row=
↪ number,
154                           supplied_id=row["encounter_id"], **row, followup_date="",
155                           disposition="encounter retained", reason=reviewed["reason"]))
156     ledger.sort(key=lambda r: (r["source_file"], r["data_row"]))
157     target = Path(destination).resolve()
158     target.mkdir(parents=False, exist_ok=False)
159     matching, weekly, worklist = (target / s for s in ("matching", "weekly", "
↪ worklist"))
160     for folder in (matching, weekly, worklist):
161         folder.mkdir()
162     save_csv(target / "source_ledger.csv", LEDGER_FIELDS, ledger)

```

```

163     save_csv(matching / "encounters.csv", ["encounter_id", "person_ref", "
    ↪ index_date"],
164             [{k: r[k] for k in ["encounter_id", "person_ref", "index_date"]}
    ↪ for r in encounters])
165     save_csv(matching / "followups.csv", SCHEMAS["capstone_followups.csv"],
    ↪ matcher_followups)
166     run_chapter("followup_match.py", matching, "--window-days", str(window))
167     matches = read_csv(matching / "followup_matches.csv")
168     matched = {r["encounter_id"] for r in matches if r["selected"]}
169     save_csv(worklist / "worklist_encounters.csv", FIELDS, work_input)
170     save_report(worklist, work_report, as_of, window, worklist / "
    ↪ worklist_encounters.csv")
171     denominator_ledger = []
172     for row in encounters:
173         eid = row["encounter_id"]
174         due = starts[eid] + timedelta(days=window)
175         included = denominator == "all" or as_of > due
176         denominator_ledger.append(dict(encounter_id=eid,
177                                       week_start=monday(starts[eid]).isoformat(), due_date=due.isoformat
    ↪ (),
178                                       included=int(included), reason=("all supplied encounters" if
    ↪ denominator == "all" else
179                                       "window closed before reporting date" if included else
180                                       "window remains open through its end date")))
181     save_csv(target / "denominator_ledger.csv", ["encounter_id", "week_start",
    ↪ "due_date",
182                                                    "included", "reason"],
    ↪ denominator_ledger)
183     included = {r["encounter_id"] for r in denominator_ledger if r["included"]}
184     week_map, week_counts, monitor_input = [], [], []
185     for number, day in enumerate(sorted(flag_map), 1):
186         group = {eid for eid in ids if monday(starts[eid]) == day} & included
187         positives = group & matched
188         review = {eid for eid in group if work_by_id[eid]["status"] == "review
    ↪ data"}
189         monitor_input.append(dict(week=number, completed=len(positives),
    ↪ eligible=len(group),
190                                unresolved=len(review), provisional=flag_map[
    ↪ day]))
191         week_map.append(dict(week=number, week_start=day.isoformat(),
    ↪ provisional=flag_map[day],
192                                as_of=as_of.isoformat(), window_days=window,
    ↪ denominator=denominator))
193         week_counts.append(dict(week_start=day.isoformat(), confirmed=len(
    ↪ positives),
194                                eligible=len(group), review=len(review), provisional=flag_map[day],
195                                numerator_ids=";".join(sorted(positives)), review_ids=";".join(

```

```

196     ↪ sorted(review)))
197     save_csv(target / "week_map.csv", ["week", "week_start", "provisional", "
198     ↪ as_of",
199     "window_days", "denominator"], week_map)
200     save_csv(target / "weekly_counts.csv", ["week_start", "confirmed", "
201     ↪ eligible", "review",
202     "provisional", "numerator_ids", "
203     ↪ review_ids"], week_counts)
204     save_csv(weekly / "weekly_followup.csv", ["week", "completed", "eligible",
205     ↪ "unresolved",
206     "provisional"], monitor_input)
207     run_chapter("weekly_followup.py", weekly)
208     record = dict(release=RELEASE, python=platform.python_version(), as_of=
209     ↪ as_of.isoformat(),
210     window_days=window, denominator=denominator,
211     closed_definition="as_of is strictly later than index_date plus
212     ↪ window_days",
213     source_records=len(ledger), encounters=len(encounters), followups=len(
214     ↪ followups),
215     flag_records=len(flags), matched=len(matched),
216     worklist_counts=dict(Counter(r["status"] for r in work_report)),
217     inputs={name: checksum(ROOT / name) for name in SCHEMAS},
218     programs={name: checksum(ROOT / name) for name in
219     ↪ ["capstone_adapter.py", "followup_match.py", "weekly_followup
220     ↪ .py", "followup_worklist.py"]})
221     (target / "integration_record.json").write_text(json.dumps(record, indent
222     ↪ =2) + "\n", encoding="utf-8")
223     print("Source ledger:", len(ledger), "records;", len(flags), "separate
224     ↪ weekly flags")
225     print("Within-window matches:", len(matched))
226     for status in ["follow-up recorded", "no follow-up after window", "window
227     ↪ still open", "review data"]:
228         print(status + ":", record["worklist_counts"].get(status, 0))
229     print("Saved", target.name)
230     return target
231
232 def main():
233     parser = argparse.ArgumentParser(description=__doc__)
234     parser.add_argument("--output", default="integrated_run")
235     parser.add_argument("--as-of", default="2026-09-17")
236     parser.add_argument("--window-days", type=int, default=14)
237     parser.add_argument("--denominator", choices=["all", "closed"], default="
238     ↪ all")
239     args = parser.parse_args()
240     try:
241         build(ROOT / args.output, parse_day(args.as_of), args.window_days, args

```

```

↩ .denominator)
230 except (ValueError, OSError, RuntimeError) as error:
231     parser.exit(1, str(error) + "\n")
232
233
234 if __name__ == "__main__":
235     main()

```

capstone_tests.py

Complete program. Save this listing as `capstone_tests.py`.

```

1  """Independent named-case checks for the printed stage-A capstone fixture.
2
3  These expected answers belong to the 14-day/all-encounters contract. A stage-B
4  change requires separately justified expectations, not relabelled PASS results.
5  """
6  import argparse
7  from collections import Counter
8  import csv
9  import json
10 from pathlib import Path
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def read(path):
16     with Path(path).open(newline="", encoding="utf-8") as stream:
17         return list(csv.DictReader(stream))
18
19
20 def check(condition, message):
21     if not condition:
22         raise AssertionError(message)
23
24
25 def main():
26     parser = argparse.ArgumentParser(description=__doc__)
27     parser.add_argument("--output", default="integrated_run")
28     parser.add_argument("--report", default="capstone_tests.csv")
29     args = parser.parse_args()
30     target = ROOT / args.output
31     record = json.loads((target / "integration_record.json").read_text())
32     encounters = read(ROOT / "capstone_encounters.csv")
33     followups = read(ROOT / "capstone_followups.csv")
34     ledger = read(target / "source_ledger.csv")

```

```

35 matches = read(target / "matching/followup_matches.csv")
36 worklist = read(target / "worklist/followup_worklist.csv")
37 weeks = read(target / "weekly_counts.csv")
38 by_id = {r["encounter_id"]: r for r in worklist}
39 matched = {r["encounter_id"] for r in matches if r["status"] == "matched"}
40 expected_matched = {"C101", "C102", "C107", "C111", "C112", "C113", "C114",
    ↪ "C115",
41                      "C116", "C117", "C118"}
42 expected_recorded = (expected_matched - {"C114"}) | {"C103"}
43 results = []
44
45 def trial(name, operation):
46     try:
47         operation()
48     except Exception as error:
49         results.append(dict(test=name, status="FAIL", detail=str(error)))
50     else:
51         results.append(dict(test=name, status="PASS", detail=""))
52
53 def contract():
54     check(record["as_of"] == "2026-09-17" and record["window_days"] == 14
    ↪ and
55         record["denominator"] == "all", "These are stage-A expectations;
    ↪ review changed rules")
56
57 def source_conservation():
58     expected = Counter((name, str(i)) for name, rows in
59                       [("capstone_encounters.csv", encounters), ("
    ↪ capstone_followups.csv", followups)]
60                       for i in range(1, len(rows)+1))
61     check(len(ledger) == 40, "The ledger must contain all forty source rows
    ↪ ")
62     check(Counter((r["source_file"], r["data_row"]) for r in ledger) ==
    ↪ expected,
63           "Source locators are missing or duplicated")
64     indexed = {(r["source_file"], int(r["data_row"])): r for r in ledger}
65     for name, rows in [("capstone_encounters.csv", encounters), ("
    ↪ capstone_followups.csv", followups)]:
66         for number, row in enumerate(rows, 1):
67             actual = indexed[(name, number)]
68             for key, value in row.items():
69                 output_key = "supplied_id" if key == "followup_id" else key
70                 check(actual[output_key] == value, "Changed source value: "
    ↪ + str((name, number, key)))
71
72 def encounter_conservation():
73     expected = Counter(r["encounter_id"] for r in encounters)

```

```

74     check(len(worklist) == 20 and Counter(r["encounter_id"] for r in
↪ worklist) == expected,
75         "Worklist must retain every encounter exactly once")
76     check(Counter(r["encounter_id"] for r in matches) == expected,
77         "Matcher must retain every encounter exactly once")
78
79     def matching_identities():
80         check(matched == expected_matched, "Wrong within-window encounter
↪ identities")
81         selected = {r["encounter_id"]: r["selected"] for r in matches}
82         check(selected["C101"] == "F01" and selected["C107"] == "F07", "
↪ Earliest candidate rule changed")
83
84     def status_identities():
85         groups = {s: {r["encounter_id"] for r in worklist if r["status"] == s}
86                  for s in ["follow-up recorded", "review data", "no follow-up
↪ after window", "window still open"]}
87         check(groups["follow-up recorded"] == expected_recorded, "Wrong
↪ recorded encounter identities")
88         check(groups["review data"] == {"C104", "C105", "C106", "C108", "C109",
↪ "C114", "C120"},
89              "Wrong review encounter identities")
90         check(groups["no follow-up after window"] == {"C110"}, "C110 alone has
↪ a passed window")
91         check(groups["window still open"] == {"C119"}, "C119 alone has an open
↪ window")
92
93     def late_record():
94         check(by_id["C103"]["followup_date"] == "2026-09-17" and
95              by_id["C103"]["status"] == "follow-up recorded" and "C103" not in
↪ matched,
96              "C103 day-16 follow-up must remain recorded, but not matched")
97
98     def review_overlap():
99         check("C114" in matched and by_id["C114"]["status"] == "review data"
↪ and
100              "contact route missing" in by_id["C114"]["reason"],
101              "C114 has a confirmed match and an independently missing contact"
↪ )
102
103     def boundary():
104         check(by_id["C119"]["due_date"] == "2026-09-17" and
105              by_id["C119"]["status"] == "window still open", "The end date is
↪ inclusive")
106         check(by_id["C102"]["followup_date"] == "2026-09-15" and "C102" in
↪ matched,
107              "C102 day-14 match is inclusive")

```

```

108
109 def duplicates_and_orphan():
110     duplicates = [r for r in ledger if r["source_file"] == "
↳ capstone_followups.csv" and r["supplied_id"] == "F09"]
111     check(len(duplicates) == 2 and {r["data_row"] for r in duplicates} == {
↳ "9", "10"},
112         "Both F09 source occurrences must remain")
113     check(all("duplicate" in r["reason"] and "withheld" in r["disposition"]
↳ for r in duplicates),
114         "Ambiguous F09 records must not enter matching")
115     orphan = [r for r in ledger if r["source_file"] == "capstone_followups.
↳ csv" and r["supplied_id"] == "F11"]
116     check(len(orphan) == 1 and orphan[0]["encounter_id"] == "C999" and
117         "unknown encounter" in orphan[0]["reason"], "Unknown reference
↳ must remain explicit")
118
119 def date_issues():
120     for eid, reason in [("C104", "before index"), ("C105", "future"), ("
↳ C106", "invalid")]:
121         check(by_id[eid]["status"] == "review data" and reason in by_id[eid
↳ ]["reason"],
122             "Missing named date issue: " + eid)
123
124 def weekly_values():
125     check([(r["week_start"], r["confirmed"], r["eligible"], r["review"], r[
↳ "provisional"]) for r in weeks] ==
126         [("2026-08-31", "8", "16", "6", "no"), ("2026-09-07", "3", "4", "
↳ 1", "yes")],
127         "Wrong dated week counts or supplied flags")
128     check(set(weeks[0]["numerator_ids"].split(";")) ==
129         {"C101", "C102", "C107", "C111", "C112", "C113", "C114", "C115"},
130         "Wrong final-week numerator identities")
131     summary = {r["measure"]: r["value"] for r in read(target / "weekly/
↳ weekly_summary.csv")}
132     check(summary["final_completed"] == "8" and summary["final_eligible"]
↳ == "16" and
133         float(summary["pooled_final_percent"]) == 50.0, "Final pooled
↳ result must be 8/16")
134
135 def dated_outputs():
136     check(all(r["as_of"] == "2026-09-17" and r["window_days"] == "14" for r
↳ in worklist),
137         "Worklist lost reporting assumptions")
138     denominator = read(target / "denominator_ledger.csv")
139     check(len(denominator) == 20 and all(r["included"] == "1" for r in
↳ denominator),
140         "Stage A retains every encounter in the denominator ledger")

```

```

141     check(record["source_records"] == 40 and record["flag_records"] == 2,
142           "The forty source records and two flag records must remain
↳ distinct")
143
144     for name, operation in [("stage_A_contract", contract), ("
↳ source_conservation", source_conservation),
145                           ("encounter_conservation", encounter_conservation), ("match_identities"
↳ , matching_identities),
146                           ("status_identities", status_identities), ("late_followup", late_record
↳ ),
147                           ("review_overlap", review_overlap), ("inclusive_boundary", boundary),
148                           ("duplicate_and_unknown", duplicates_and_orphan), ("date_issues",
↳ date_issues),
149                           ("dated_weekly_values", weekly_values), ("dated_outputs", dated_outputs
↳ )]):
150         trial(name, operation)
151     with (ROOT / args.report).open("x", newline="", encoding="utf-8") as stream
↳ :
152         writer = csv.DictWriter(stream, fieldnames=["test", "status", "detail"
↳ ])
153         writer.writeheader()
154         writer.writerows(results)
155     print("Passed:", sum(r["status"] == "PASS" for r in results))
156     print("Failed:", sum(r["status"] == "FAIL" for r in results))
157     for r in results:
158         if r["status"] == "FAIL":
159             print(r["test"] + ": " + r["detail"])
160     if any(r["status"] == "FAIL" for r in results):
161         raise SystemExit(1)
162
163
164 if __name__ == "__main__":
165     main()

```

Larger Practice Files

Four independent fixtures extend the small chapter examples. Their programs are unchanged; their inputs and expected results are different. Each fixture has its own complete **EXPECTED.json** listing below. Never compare these runs with the smaller chapter's totals.

Create a separate working folder for each fixture. Save its complete inputs printed below, supply the unchanged programs from its numbered chapter, and run the command in that folder. Do not mix the four files named **EXPECTED.json**; each describes only its own fixture.

Chapter 2: More Journey Records

```
python3 journey_check.py
```

The 36 source records yield 30 placed events and six review rows. The exact review source rows are 5, 8, 20, 27, 29, and 36. Check both the total and those positions. A familiar-looking ID does not link this independent fixture to Chapter 1's events.

Complete Practice Inputs

EXPECTED.json

Save the following complete contents as **EXPECTED.json**.

```
{
  "records": 36,
  "placed": 30,
  "review": 6,
  "review_source_rows": [
    5,
    8,
    20,
    27,
    29,
    36
  ]
}
```

journey_problem.csv

Save the following complete contents as journey_problem.csv.

```
event_id,person_ref,encounter_id,event_date,event_type,detail
Q001,P01,Q01,2026-06-01,visit,Visit recorded
Q002,P01,Q01,2026-06-02,test,Sample collected
Q003,P01,Q01,2026-06-05,followup,Follow-up recorded
Q004,P01,Q02,2026-06-03,visit,Visit recorded
Q005,P01,Q02,2026-06-04,test,Sample collected
Q006,P01,Q02,2026-06-07,followup,Follow-up recorded
Q007,P02,Q03,2026-06-05,visit,Visit recorded
Q008,P02,Q03,,test,Sample collected
Q009,P02,Q03,2026-06-09,followup,Follow-up recorded
Q010,P02,Q04,2026-06-07,visit,Visit recorded
Q011,P02,Q04,2026-06-08,test,Sample collected
Q012,P02,Q04,2026-06-11,followup,Follow-up recorded
Q013,P03,Q05,2026-06-09,visit,Visit recorded
Q014,P03,Q05,2026-06-10,test,Sample collected
Q015,P03,Q05,2026-06-13,followup,Follow-up recorded
Q016,P03,Q06,2026-06-11,visit,Visit recorded
Q017,P03,Q06,2026-06-12,test,Sample collected
Q018,P03,Q06,2026-06-15,followup,Follow-up recorded
Q019,P04,Q07,2026-06-13,visit,Visit recorded
Q020,P04,Q07,2026-06-31,test,Sample collected
Q021,P04,Q07,2026-06-17,followup,Follow-up recorded
Q022,P04,Q08,2026-06-15,visit,Visit recorded
Q023,P04,Q08,2026-06-16,test,Sample collected
Q024,P04,Q08,2026-06-19,followup,Follow-up recorded
Q025,P05,Q09,2026-06-17,visit,Visit recorded
Q026,P05,Q09,2026-06-18,test,Sample collected
Q027,P05,Q09,2026-06-16,followup,Follow-up recorded
Q028,P05,Q10,2026-06-19,visit,Visit recorded
Q005,P05,Q10,2026-06-20,test,Sample collected
Q030,P05,Q10,2026-06-23,followup,Follow-up recorded
Q031,P06,Q11,2026-06-21,visit,Visit recorded
Q032,P06,Q11,2026-06-22,test,Sample collected
Q033,P06,Q11,2026-06-25,followup,Follow-up recorded
Q034,P06,Q12,2026-06-23,visit,Visit recorded
Q035,P06,Q12,2026-06-24,test,Sample collected
Q036,P01,Q12,2026-06-27,followup,Follow-up recorded
```

Chapter 8: Competing Matches

```
python3 followup_match.py
```

There are 16 encounters and 24 follow-up records. Fifteen encounters match; C012 does not. F24 has an unknown encounter reference. Compare the complete selection map in this fixture's `EXPECTED.json` with the actual selected IDs, not just the matched count.

Complete Practice Inputs

EXPECTED.json

Save the following complete contents as **EXPECTED.json**.

```
{
  "encounters": 16,
  "followups": 24,
  "matched": 15,
  "unknown_reference_ids": [
    "F24"
  ],
  "selected": {
    "C001": "F01",
    "C002": "F02",
    "C003": "F19",
    "C004": "F20",
    "C005": "F21",
    "C006": "F06",
    "C007": "F07",
    "C008": "F08",
    "C009": "F09",
    "C010": "F10",
    "C011": "F11",
    "C012": "",
    "C013": "F13",
    "C014": "F14",
    "C015": "F15",
    "C016": "F16"
  }
}
```

encounters.csv

Save the following complete contents as **encounters.csv**.

```
encounter_id,person_ref,index_date
C001,P01,2026-09-01
C002,P01,2026-09-01
C003,P02,2026-09-01
C004,P02,2026-09-01
C005,P03,2026-09-01
C006,P03,2026-09-01
C007,P04,2026-09-01
C008,P04,2026-09-01
C009,P05,2026-09-03
```

```
C010,P05,2026-09-03
C011,P06,2026-09-03
C012,P06,2026-09-03
C013,P07,2026-09-03
C014,P07,2026-09-03
C015,P08,2026-09-03
C016,P08,2026-09-03
```

followups.csv

Save the following complete contents as **followups.csv**.

```
followup_id,encounter_id,followup_date
F01,C001,2026-09-01
F02,C002,2026-09-15
F03,C003,2026-09-16
F04,C004,2026-08-31
F05,C005,2026-09-08
F06,C006,2026-09-09
F07,C007,2026-09-04
F08,C008,2026-09-06
F09,C009,2026-09-05
F10,C010,2026-09-07
F11,C011,2026-09-17
F12,C012,2026-09-18
F13,C013,2026-09-09
F14,C014,2026-09-06
F15,C015,2026-09-05
F16,C016,2026-09-10
F17,C001,2026-09-06
F18,C002,2026-09-15
F19,C003,2026-09-15
F20,C004,2026-09-01
F21,C005,2026-09-02
F22,C006,2026-09-22
F23,C007,2026-09-04
F24,C999,2026-09-04
```

Chapter 9: Different Measurement Populations

```
python3 cohort_summary.py
```

Forty candidates yield 32 eligible people, 24 recorded follow-ups, and 24 observed waits. The median observed wait is 10 days. The two equal counts of 24 do not

establish that the same people supplied both. Inspect the source and disposition ledger.

Complete Practice Inputs

EXPECTED.json

Save the following complete contents as `EXPECTED.json`.

```
{
  "candidates": 40,
  "eligible": 32,
  "followup_recorded": 24,
  "observed_waits": 24,
  "median_wait_days": 10.0
}
```

cohort_candidates.csv

Save the following complete contents as `cohort_candidates.csv`.

```
person_ref,age_years,enrolled,followup_recorded,wait_days
S101,17,yes,no,
S102,17,yes,no,
S103,17,yes,no,
S104,17,yes,no,
S105,25,no,no,
S106,26,no,no,
S107,27,no,no,
S108,28,no,no,
S109,29,yes,yes,4
S110,30,yes,yes,8
S111,31,yes,yes,12
S112,32,yes,yes,16
S113,33,yes,yes,4
S114,34,yes,yes,8
S115,35,yes,yes,12
S116,36,yes,yes,16
S117,37,yes,yes,4
S118,38,yes,yes,8
S119,39,yes,yes,12
S120,40,yes,yes,16
S121,41,yes,yes,4
S122,42,yes,yes,8
S123,43,yes,yes,12
S124,44,yes,yes,16
```

```
S125,45,yes,yes,4
S126,46,yes,yes,8
S127,47,yes,yes,12
S128,48,yes,yes,16
S129,49,yes,yes,4
S130,50,yes,yes,8
S131,51,yes,yes,12
S132,52,yes,yes,16
S133,53,yes,no,
S134,54,yes,no,
S135,55,yes,no,
S136,56,yes,no,
S137,57,yes,no,
S138,58,yes,no,
S139,59,yes,no,
S140,60,yes,no,
```

Chapter 23: Overlapping Review Reasons

```
python3 followup_worklist.py
```

All 36 encounters remain, with nine in each of the four statuses. The recorded group is W001, W005, W009, W013, W017, W021, W025, W029, W033. The passed-window group begins W002 and continues every fourth ID through W034; the open-window group begins W003 and continues through W035; the review group begins W004 and continues through W036.

Use the explicit lists in this fixture's `EXPECTED.json` to check every status. A correct distribution of nine records per group could still conceal incorrect record assignments.

Complete Practice Inputs

`EXPECTED.json`

Save the following complete contents as `EXPECTED.json`.

```
{
  "records": 36,
  "groups": {
    "follow-up recorded": [
      "W001",
      "W005",
      "W009",
      "W013",
      "W017",
```

```
        "W021",
        "W025",
        "W029",
        "W033"
    ],
    "no follow-up after window": [
        "W002",
        "W006",
        "W010",
        "W014",
        "W018",
        "W022",
        "W026",
        "W030",
        "W034"
    ],
    "window still open": [
        "W003",
        "W007",
        "W011",
        "W015",
        "W019",
        "W023",
        "W027",
        "W031",
        "W035"
    ],
    "review data": [
        "W004",
        "W008",
        "W012",
        "W016",
        "W020",
        "W024",
        "W028",
        "W032",
        "W036"
    ]
}
}
```

worklist_encounters.csv

Save the following complete contents as `worklist_encounters.csv`.

```

encounter_id,index_date,followup_date,contact_route,owner
W001,2026-09-01,2026-09-17,office,Team A
W002,2026-09-01,,office,Team B
W003,2026-09-10,,office,Team A
W004,2026-09-01,,,Team B
W005,2026-09-01,2026-09-17,office,Team A
W006,2026-09-01,,office,Team B
W007,2026-09-10,,office,Team A
W008,2026-09-01,2026-09-18,office,Team B
W009,2026-09-01,2026-09-17,office,Team A
W010,2026-09-01,,office,Team B
W011,2026-09-10,,office,Team A
W012,2026-09-01,2026-09-31,office,Team B
W013,2026-09-01,2026-09-17,office,Team A
W014,2026-09-01,,office,Team B
W015,2026-09-10,,office,Team A
W016,2026-09-01,2026-08-31,office,Team B
W017,2026-09-01,2026-09-17,office,Team A
W018,2026-09-01,,office,Team B
W019,2026-09-10,,office,Team A
W020,2026-09-01,,office,
W021,2026-09-01,2026-09-17,office,Team A
W022,2026-09-01,,office,Team B
W023,2026-09-10,,office,Team A
W024,not-a-date,,office,Team B
W025,2026-09-01,2026-09-17,office,Team A
W026,2026-09-01,,office,Team B
W027,2026-09-10,,office,Team A
W028,2026-09-20,,office,Team B
W029,2026-09-01,2026-09-17,office,Team A
W030,2026-09-01,,office,Team B
W031,2026-09-10,,office,Team A
W032,2026-09-01,2026-08-31,,Team B
W033,2026-09-01,2026-09-17,office,Team A
W034,2026-09-01,,office,Team B
W035,2026-09-10,,office,Team A
W036,2026-09-01,,,

```

Make a New Test Deliberately

Keep the supplied fixture and its expected-result file unchanged. In a new copy, choose one record and one field, predict which named outputs should change, and then run. Check records that should remain unchanged as well. The existing expected-result file

is a reference for the original fixture, not an answer key for every modified dataset.

A Small Python Reference

This optional reference explains the notation used in the listings. It is not a separate Python course. Return to the named application when you need a concrete example.

Values and Collections

A name such as `rows` refers to a value. Assignment uses `=`; equality comparison uses `==`. CSV fields are text: `"14"` is not the number 14 until an explicit conversion such as `int` or `float` succeeds. Blank text, zero, and `None` have different meanings.

A dictionary connects names to values: `row["encounter_id"]` retrieves one named field. A list retains a sequence of rows; a set retains distinct values. A tuple such as `(measure, unit)` can be one grouping key. `defaultdict(list)` collects groups; `Counter` counts occurrences. See Chapters 1, 2, and 5. Counting distinct people is not counting source rows.

Conditions, Loops, and Selection

An `if/elif/else` chain chooses one branch, as in the cohort or worklist. Independent `if` statements can collect several reasons, as in the review-status checker. These are different decisions.

A `for` loop visits a sequence. `enumerate(rows, 1)` supplies a row number as well as the row. A comprehension constructs a list, set, or dictionary from selected values. Follow the inclusion condition before admiring the compact notation. Chapter 9 first selects eligible people, then their observed waits; those are not the same population.

A sorting key determines order. Chapter 8 sorts `(days, followup_id)` pairs, so the ID resolves an elapsed-day tie. Changing that key can change the selected follow-up even when no input row is removed.

Functions and Modules

A function groups instructions for a named job and may return a result. Arguments provide its inputs. A default such as `person=None` applies when the caller supplies no value for that argument.

An `import` makes a module available and executes its top-level statements. Reusable modules put their entry point behind `if __name__ == "__main__":`. Several short chapter scripts instead do their work at top level. Keep required local modules together, but do not import every chapter script merely to inspect it.

Files and Failed Runs

`csv.DictReader` reads named fields; `csv.DictWriter` writes them. The supplied programs also check exact headers and malformed rows. Their file paths generally locate inputs beside the script. Chapter 3 additionally accepts command-line paths, interpreted from the working directory when relative. A correct filename in the wrong folder is still missing.

Exclusive creation mode "`x`" refuses an existing file. A failed run can leave partial output. Preserve it, read the error, and use a new destination; do not assume that every multi-file operation either succeeds entirely or leaves nothing behind.

A `try/except` block handles a specified exception. A value problem can become a row-level reason only after the program knows which fields the row contains. The last line of a traceback is a useful starting point, not a reason to change the stated data rule.

Dates and Numerical Summaries

Date arithmetic supplies elapsed days; the comparison supplies the rule. `0 <= days <= 14` includes both endpoints. `as_of > due` keeps the window open through its end date. See Chapters 8 and 23.

Mean and median operate on the values supplied to them. State which records contribute. An empty median is not zero, and an absent denominator does not give a 0% rate. Pool counts before dividing when the question concerns a pooled proportion; do not average rounded weekly percentages.

Tests and Reproduction

A test compares an actual result with an explicit expectation. Check source identities and values as well as counts. The Chapter 20 deliberate defect shows why testing the tests can be useful.

A hash identifies file bytes, not their correctness. For a simulation, record the seed, interpreter, inputs, and program. For a ZIP, distinguish container metadata from member contents. For a browser page, distinguish the saved data from the currently displayed subset. These are different things to compare.

Checks for This Edition

All 23 chapter baselines were executed with their supplied small inputs. The 27 distinct complete program listings are unchanged from the uploaded author source. The small journey suite passed twelve tests; the integrated capstone suite passed twelve. The four independent practice fixtures were also executed and compared with their named expected records and summaries.

The three-program journey tool processed the actual large CSV: 12,000 input and selected records, 12,000 placed, and no review rows. The P0001 report examined the full input and selected twelve records. The separate large suite passed ten tests. Conversion retained all 12,000 events in its ledger and produced 3,000 encounters and 3,000 follow-ups. The matcher returned 1,684 matches at fourteen days and 3,000 at twenty-one days, with no unknown encounter references.

The unchanged worklist classifier was called on the two explicit large-input variants: 3,000 review rows with blank administration, and 3,000 recorded follow-ups with the supplied fictional office/Teaching Team configuration. This verifies the stated consequences, not a general-purpose Stage-C adapter. The eight-member Chapter 21 package was extracted into a fresh location and its printed command ran without borrowing files.

Browser and Print Checks

Separate automated checks operated Chromium on the saved HTML contents. Person selection and E02's marker, retained review records, combined worklist filters and their empty state, and the integrated C103/C114 distinction were checked. The predictive-value controls reproduced the three scenarios and endpoints, rejected invalid entries without stale results, and left the scenario CSV unchanged. A regenerated 80-percent-sensitivity version showed 80 true positives and 13.9-percent predictive value at 1-percent prevalence, agreeing with its CSV. The three supplied letters printed to three letter-size pages, with the held record absent.

The container blocks direct file-URL navigation, so these tests loaded the saved HTML bytes through browser automation. They do not establish a reader's double-click/open experience, all browsers, or accessibility. Those distinctions matter even though the interactive calculations and selectors were actually exercised.

Reconstructing the First Program

The first complete 170-line program and its eighteen-record CSV were reconstructed from the rendered PDF, using the text layer and glyph positions rather than substituting the LaTeX listing. Two wrapped spaces were recovered from their positions; one further missing space in a grouped-marker label was corrected after comparison with the author source. The reconstructed program then passed syntax checking and produced the same CSV and HTML bytes as the reference run.

This is a technical transcription check, not an independent beginner trial. It demonstrates why line numbers, continuation arrows, indentation, and spaces inside quoted strings must be checked. A nonprogrammer should still follow the complete first-run instructions on the intended platform before public release.

These checks apply to the named fixtures, programs, and environment. They are not proof for all inputs or authorization for real records. Open-ended design and interface extensions remain worked tasks unless explicitly identified as executed. A test-suite pass and a separate browser check are different evidence.

The unchanged large file has 942,063 bytes. Its SHA-256 identity is:

```
309ef131fdc5f36eb512e01123c4a8620156ea964aa5c6241501e015e72c5d60
```

A hash identifies these bytes; it does not establish the truth of an event or the correctness of a program. Reordering the file changes its byte identity even when an identity-based comparison of its records agrees.

To typeset this companion, keep `Python_Companion.tex` and `Python_Companion.sty` together. All program and small-input listings are embedded in the LaTeX source; no external listing folders are needed.